

**ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN**



**PBL4: DỰ ÁN HỆ ĐIỀU HÀNH &
MẠNG MÁY TÍNH**

Đề Tài:

**Tối ưu hợp tác bằng Trí tuệ nhân tạo trong Mạng tích hợp
Không gian – Mặt đất – Biển (SAGSINs)**

SINH VIÊN THỰC HIỆN:

**Nguyễn Quang Anh LỚP: 23T_DT1 NHÓM: 23Nh13
Cáp Kim Khánh LỚP: 23T_DT1 NHÓM: 23Nh13**

GIẢNG VIÊN HƯỚNG DẪN: TS. Nguyễn Năng Hùng Vân

Đà Nẵng 12/2025

MỤC LỤC

DANH MỤC HÌNH ẢNH.....	5
DANH MỤC BẢNG BIỂU.....	7
DANH MỤC TỪ VIẾT TẮT.....	8
MỞ ĐẦU.....	10
1. TÍNH CẤP THIẾT CỦA ĐỀ TÀI.....	11
1.1. Tính cấp thiết của đề tài đặt ra.....	11
1.2. Xu hướng phát triển và nhu cầu đổi mới.....	11
1.3. Giải pháp tiếp cận mới.....	11
1.4. Ý nghĩa thực tiễn và cấp thiết của đề tài.....	12
2. MỤC ĐÍCH VÀ Ý NGHĨA CỦA ĐỀ TÀI.....	12
2.1. Mục đích của đề tài.....	12
2.1.1. Mục tiêu tổng quát.....	12
2.1.2. Các nhiệm vụ nghiên cứu chính.....	13
2.2. Ý nghĩa của đề tài.....	13
2.2.1. Ý nghĩa khoa học.....	14
2.2.2. Ý nghĩa thực tiễn.....	14
3. ĐỐI TƯỢNG VÀ PHẠM VI ĐỀ TÀI.....	14
3.1. Đối tượng nghiên cứu.....	14
3.1.1. Cơ sở mô hình QR-DQN và Thuật toán.....	15
3.1.2. Kiến trúc hệ thống và Môi trường mô phỏng.....	15
3.1.3. Không gian trạng thái và hàm phần thưởng.....	16
3.2. Phạm vi nghiên cứu.....	17
3.2.1. Phạm vi lý thuyết.....	17
3.2.2. Phạm vi kỹ thuật và giải pháp.....	17
3.2.3. Phạm vi thực nghiệm.....	17
4. CẤU TRÚC ĐỀ TÀI.....	18
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT.....	19
1.1. Cơ sở lý thuyết về Hệ điều hành và Mạng máy tính.....	19
1.1.1. Các giao thức và kiến trúc tầng Giao vận.....	19
1.1.2. Lý thuyết về Hệ điều hành.....	19
1.2. Tổng quan về Mạng Tích hợp Không gian - Hàng không - Mặt đất - Biển (SAGSINs).....	20
1.2.1. Khái niệm và Kiến trúc.....	20
1.2.2. Đặc điểm và Thách thức.....	21
1.2.3. Bài toán định tuyến đa mục tiêu và QoS	21

1.2.3.1.	Định nghĩa bài toán.....	21
1.2.3.2.	Các chỉ số QoS	21
1.3.	Các công trình liên quan và Khoảng trống nghiên cứu.....	22
1.3.1.	Tổng quan các phương pháp hiện có.....	22
1.3.2.	Khoảng trống nghiên cứu (Research Gap)	22
1.4.	Cơ sở Lý thuyết về Học tăng cường sâu (DRL).....	22
1.4.1.	Giới thiệu chung về DRL	23
1.4.2.	Quy trình Quyết định Markov (MDP).....	23
1.4.4.	Giải pháp đề xuất: Học tăng cường phân bố (Distributional RL)	26
1.5.	Khả năng ứng dụng của DRL vào bài toán Giao tiếp Cộng tác trong SAGSINs....	28
1.5.1.	Ưu thế của QR-DQN	28
1.5.2.	Ứng dụng định tuyến Nhận thức Rủi ro (Risk-Aware Routing)	29
1.6.	Tổng kết chương	29
CHƯƠNG 2: PHƯƠNG PHÁP ĐỀ XUẤT VÀ THIẾT KẾ HỆ THỐNG		31
2.1.	Nguyên tắc thiết kế.....	31
2.2.	Kiến trúc hệ thống đề xuất	31
2.3.	Đặc tả Dữ liệu hệ thống.....	33
2.3.1.	Dữ liệu thực tế (Real-world Simulation Data)	33
2.3.2.	Dữ liệu huấn luyện (Traning Interaction Data)	35
2.4.	Mô hình hóa bài toán (MDP)	35
2.4.1.	Kỹ thuật Tiền xử lý và Chuẩn hóa Dữ liệu (Data Normalization)	35
2.4.2.	Hiện thực hoá Không gian Trạng thái (Observation Space - $\mathcal{S}$).....	36
2.4.3.	Hiện thực hoá Không gian Hành động (Action Space - $\mathcal{A}$)	37
2.4.4.	Thiết kế Hàm phần thưởng cho tác nhân QR – DQN.....	38
2.5.	Thiết kế và huấn luyện Mô hình Học tăng cường (QR-DQN).....	39
2.5.1.	Định nghĩa Bài toán.....	40
2.5.2.	Công nghệ và Thư Viện.....	40
2.5.3.	Kiến trúc Mạng Nơ-ron Sâu	41
2.5.4.	Cơ chế Học tập và Quy trình huấn luyện	42
2.5.5.	Tóm tắt Thông số Kỹ thuật Huấn luyện	43
2.6.	Thực nghiệm và Đánh giá Kết quả.....	44
2.6.1.	Các kịch bản thực nghiệm	44
2.6.2.	Phân tích và Đánh giá Kết quả Huấn luyện.....	45
2.6.3.	Đánh giá Hiệu năng của Tác nhân QR – DQN.....	47
2.6.3.1.	Phương Pháp đánh giá	47
2.6.3.2.	Phân tích đánh giá tổng quan.....	47
2.6.4.	Phân tích ổn định	54

2.6.5.	Phân tích Định tính: Lợi thế của Tác nhân AI.....	54
2.6.6.	Kết luận thực nghiệm	55
2.7.	Tổng kết Chương.....	55
CHƯƠNG 3: TỐI ƯU HỢP TÁC BẰNG TRÍ TUỆ NHÂN TẠO TRONG MẠNG TÍCH HỢP KHÔNG GIAN – MẶT ĐẤT – BIÊN (SAGSINS)		56
3.1.	Mô hình kiến trúc Client – Server.....	56
3.1.1.	Client (Giao diện người dùng).....	56
3.1.2.	Server Gateway.....	57
3.1.3.	Python Service (Flask Backend).....	57
3.2.	Luồng hoạt động và mô tả chương trình	57
3.3.	Tổng kết Chương.....	65
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....		66
1.	Kết quả đạt được	66
2.	Hạn chế của đề tài	67
3.	Hướng phát triển trong tương lai.....	67
4.	Tổng kết.....	67
TÀI LIỆU THAM KHẢO		69

DANH MỤC HÌNH ẢNH

Hình 1: Mô tả sơ bộ mô hình Học tăng cường sâu DRL

Hình 2: Mô tả tương tác giữa Agent – Environment

Hình 3: Mô tả tổng quan về Học tăng cường sâu DRL

Hình 4: Mô tả quy trình Quyết định Markov (Markov Decision Process - MDP)

Hình 5: Mô tả mô hình Deep Q-Network

Hình 6: Mô tả mô hình Deep Deterministic Policy Gradient

Hình 7: Mô tả mô hình Quantile Regression DQN

Hình 8: Mô tả quy trình huấn luyện tác nhân QR – DQN

Hình 9: Kết quả Q-Score theo episode và trung bình trượt

Hình 10: Kết quả Reward theo episode và trung bình trượt

Hình 11: So sánh tổng quan giữa AI Agent và Dijkstra

Hình 12: So sánh tỉ lệ thành công giữa AI Agent và Dijkstra

Hình 13: So sánh chi tiết phân bố tài nguyên

Hình 14: Biểu đồ CDF so sánh độ trễ giữa AI Agent và Dijkstra

Hình 15: Biểu đồ CDF so sánh số hop đã đi giữa AI Agent và Dijkstra

Hình 16: Biểu đồ CDF so sánh băng thông uplink giữa AI Agent và Dijkstra

Hình 17: Biểu đồ CDF so sánh băng thông downlink giữa AI Agent và Dijkstra

Hình 18: Biểu đồ CDF so sánh độ tin cậy giữa AI Agent và Dijkstra

Hình 19: Biểu đồ CDF so sánh mức sử dụng CPU giữa AI Agent và Dijkstra

Hình 20: Biểu đồ CDF so sánh mức sử dụng năng lượng giữa AI Agent và Dijkstra

Hình 21: So sánh độ lệch chuẩn trong phân bố tài nguyên

Hình 22: Mô tả tổng quan hệ thống

Hình 23: Sơ đồ khối mô tả kiến trúc Client-Server và luồng dữ liệu của ứng dụng

Hình 24: Giao diện Client chính với quả địa cầu 3D và nhập liệu mô phỏng

Hình 25: Giao diện Modal hiển thị kết quả đường đi

Hình 26: Giao diện Dashboard giám sát

Hình 27: Giao diện giám sát toàn bộ vệ tinh Satellites

Hình 28: Giao diện giám sát toàn bộ trạm mặt đất GroundStation

Hình 29: Giao diện giám sát toàn bộ trạm biển SeaStation

Hình 30: Giao diện giám sát toàn bộ kết nối Connection tại thời điểm thực

Hình 31: Giao diện chi tiết các thông số AI Agent đạt được ở 1 request ngẫu nhiên

Hình 32: Biểu đồ so sánh chi tiết các thông số AI Agent và Dijkstra đạt được ở 1 request ngẫu nhiên

Hình 33: Biểu đồ so sánh tổng quan AI Agent và Dijkstra đạt được trong toàn bộ quá trình

Hình 34: Biểu đồ so sánh hiệu suất AI Agent và Dijkstra đạt được trong toàn bộ quá trình

Hình 35: Giao diện so sánh chi tiết QoS của AI Agent và Dijkstra

Hình 36: Giao diện so sánh độ lệch chuẩn QoS của AI Agent và Dijkstra

DANH MỤC BẢNG BIỂU

Bảng 1: Chỉ số về yêu cầu dịch vụ

Bảng 2: Chỉ số về trạng thái đường đi

Bảng 3: Chỉ số về hiệu quả sử dụng tài nguyên

Bảng 4: Số lượng nút trong mạng SAGGINS

Bảng 5: Các loại dịch vụ và cấu hình QoS

Bảng 6: Cấu trúc Không gian Trạng thái

Bảng 7: Trọng số chi tiết cho phần thưởng QoS

Bảng 8: Các siêu tham số huấn luyện mô hình

Bảng 9: So sánh Early vs Late đánh giá QR – DQN

Bảng 10: So sánh độ lệch chuẩn giữa AI Agent và Dijkstra

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên tiếng Anh đầy đủ	Giải thích (Nghĩa tiếng Việt)
SAGSINs	Space–Air–Ground Integrated Networks	Mạng tích hợp Không gian - Hàng không - Mặt đất (và Biển). Kiến trúc mạng đa tầng kết hợp vệ tinh (không gian), thiết bị bay (hàng không), trạm mặt đất và trạm biển để cung cấp phủ sóng toàn cầu liên mạch.
GS	Ground Station	Trạm mặt đất. Cơ sở hạ tầng viễn thông trên đất liền, đóng vai trò là cổng kết nối (Gateway) liên kết mạng vệ tinh/UAV với mạng lõi (Internet) mặt đất.
SS	Sea Station	Trạm mặt biển. Các nút mạng triển khai trên biển (tàu, giàn khoan, phao), hoạt động như Gateway hoặc nút chuyển tiếp để mở rộng vùng phủ sóng ra đại dương.
UAV	Unmanned Aerial Vehicle	Phương tiện bay không người lái. Các thiết bị bay tự động hoạt động ở tầng hàng không, đóng vai trò trạm phát sóng di động hoặc nút chuyển tiếp dữ liệu linh hoạt.
GEO	Geostationary Earth Orbit	Quỹ đạo địa tĩnh. Quỹ đạo ở độ cao ~35.786 km. Vệ tinh tại đây có chu kỳ quay trùng với chu kỳ tự quay của Trái Đất, do đó vị trí tương đối đứng yên so với mặt đất.
LEO	Low Earth Orbit	Quỹ đạo Trái Đất tầm thấp. Quỹ đạo ở độ cao từ 160–2.000 km. Vệ tinh LEO di chuyển với tốc độ cao quanh Trái Đất, cung cấp độ trễ thấp cho truyền thông.
QoS	Quality of Service	Chất lượng dịch vụ. Tập hợp các chỉ số kỹ thuật (băng thông, độ trễ, độ rung, tỷ lệ mất gói) dùng để đo lường và cam kết hiệu năng của mạng lưới.
AI	Artificial Intelligence	Trí tuệ nhân tạo. Lĩnh vực khoa học máy tính nghiên cứu mô phỏng các quá trình nhận thức của con người (học tập, lập luận, tự sửa lỗi) bằng hệ thống máy móc.
DRL	Deep Reinforcement Learning	Học tăng cường sâu. Phương pháp kết hợp giữa Học tăng cường (Reinforcement Learning) và Học sâu (Deep Learning), sử dụng mạng nơ-ron để tối ưu hóa chính sách ra quyết định trong không gian trạng thái phức tạp.

Từ viết tắt	Tên tiếng Anh đầy đủ	Giải thích (Nghĩa tiếng Việt)
MDP	Markov Decision Process	Quá trình ra quyết định Markov. Khung toán học dùng để mô hình hóa bài toán ra quyết định, trong đó kết quả phụ thuộc một phần vào hành động của tác nhân và một phần vào yếu tố ngẫu nhiên.
DQN	Deep Q-Network	Mạng Q sâu. Thuật toán DRL sử dụng mạng nơ-ron sâu để xấp xỉ hàm giá trị $Q(s, a)$, nhằm tìm ra hành động tối ưu hóa phần thưởng tích lũy trong không gian trạng thái rời rạc.
QR-DQN	Quantile Regression Deep Q-Network	Mạng Q sâu hồi quy phân vị. Biến thể của DQN, ước lượng toàn bộ phân phối xác suất của phần thưởng thay vì chỉ giá trị trung bình, giúp tác nhân học tốt hơn trong môi trường có tính rủi ro cao.
DDPG	Deep Deterministic Policy Gradient	Gradient chính sách xác định sâu. Thuật toán thuộc họ Actor-Critic, được thiết kế cho không gian hành động liên tục, học một chính sách xác định (deterministic policy) để tối ưu hóa hành động.
PPO	Proximal Policy Optimization	Tối ưu hóa chính sách lân cận. Thuật toán Policy Gradient sử dụng cơ chế cắt (clipping) hàm mục tiêu để giới hạn mức độ thay đổi của chính sách sau mỗi bước, đảm bảo quá trình học ổn định và hội tụ hiệu quả.
MSE	Mean Squared Error	Sai số bình phương trung bình. Hàm mất mát đo lường trung bình của bình phương sai số giữa giá trị dự đoán của mô hình và giá trị thực tế, dùng để đánh giá độ chính xác của việc xấp xỉ hàm.

MỞ ĐẦU

Sự phát triển của mạng di động thế hệ thứ sáu (6G) đang được thúc đẩy mạnh mẽ bởi nhu cầu về một hạ tầng viễn thông siêu kết nối với độ tin cậy cao và độ trễ cực thấp [3, 4]. Mặc dù mạng mặt đất 5G hiện tại đã đạt được những bước tiến đáng kể, chúng vẫn tồn tại những hạn chế vật lý về vùng phủ sóng, đặc biệt tại các khu vực địa hình phức tạp như đại dương, sa mạc hoặc không phận rộng lớn. Nhằm khắc phục các "điểm mù" kết nối này, Mạng tích hợp Không gian - Trên không - Mặt đất - Biển (Space-Air-Ground-Sea Integrated Networks - SAGSINs) đã nổi lên như một kiến trúc tiềm năng [1, 2, 5]. Đây là sự hội tụ của các phân đoạn mạng đa tầng, bao gồm vệ tinh đa quỹ đạo, trạm bay tầng bình lưu (HAP), thiết bị bay không người lái (UAV) kết hợp với hạ tầng mặt đất và trạm biển, tạo nên một hệ sinh thái truyền thông toàn diện [6, 8, 9, 29].

Tuy nhiên, tính không đồng nhất và quy mô của SAGSINs đặt ra những thách thức vận hành chưa từng có [10]. Đặc thù của môi trường này là tính động cao do sự di chuyển liên tục của các vệ tinh LEO và UAV, dẫn đến cấu trúc topo mạng thay đổi theo thời gian thực và các liên kết truyền dẫn thường xuyên bị gián đoạn. Các phương pháp định tuyến truyền thống (như thuật toán đường đi ngắn nhất) hoặc các quy tắc tĩnh thường thiếu khả năng thích ứng linh hoạt để xử lý các yêu cầu đa mục tiêu về Chất lượng Dịch vụ (QoS) như độ trễ, băng thông và hiệu quả năng lượng trong môi trường biến động này [12, 13, 18, 19].

Sự hạn chế của các phương pháp trên đã thúc đẩy nhu cầu ứng dụng Trí tuệ Nhân tạo (AI), đặc biệt là Học tăng cường sâu (Deep Reinforcement Learning - DRL), vào quản trị mạng tự trị [11, 15, 17]. Khác với các kịch bản lập trình sẵn, tác nhân DRL có khả năng tự học chính sách tối ưu thông qua tương tác liên tục với môi trường. Tuy nhiên, phần lớn các nghiên cứu hiện tại áp dụng DRL tiêu chuẩn chỉ tập trung vào tối ưu hóa giá trị kỳ vọng trung bình của phần thưởng, bỏ qua sự phân phối ngẫu nhiên của các biến số môi trường [16, 27]. Để khắc phục điều này, nghiên cứu đề xuất giải pháp định tuyến tài nguyên động sử dụng thuật toán **QR-DQN (Quantile Regression DQN)** [20]. Kiến trúc QR-DQN cho phép mô hình học toàn bộ phân phối xác suất của phần thưởng thay vì chỉ giá trị trung bình, giúp tác nhân nhận thức được rủi ro và đưa ra quyết định với độ tin cậy cao hơn [23, 24].

Cụ thể, nghiên cứu tập trung thiết kế môi trường mô phỏng phản ánh chân thực động học của các nút mạng và ràng buộc tài nguyên. Trên cơ sở đó, tác nhân QR-DQN được huấn luyện để quan sát trạng thái mạng tức thời nhằm tìm ra lộ trình tối ưu, cân bằng giữa tối đa hóa QoS và hiệu quả sử dụng tài nguyên. Kết quả nghiên cứu không chỉ khẳng định tính ưu việt của giải pháp đề xuất so với các thuật toán truyền thống và DRL cơ bản, mà còn góp phần chứng minh tính khả thi của việc vận hành thông minh hạ tầng mạng 6G trong tương lai.

1. TÍNH CẤP THIẾT CỦA ĐỀ TÀI

1.1. Tính cấp thiết của đề tài đặt ra

Trong chiến lược phát triển hạ tầng viễn thông toàn cầu, Mạng Tích hợp Không gian – Hàng không – Mặt đất – Biển (SAGSINs) được xác định là kiến trúc nền tảng cốt lõi cho mạng di động thế hệ thứ sáu (6G) [3, 4]. Mô hình này cho phép mở rộng vùng phủ sóng toàn cầu, xóa bỏ các điểm mù kết nối tại các khu vực địa lý hiểm trở như đại dương hay sa mạc [1, 2]. Tuy nhiên, quy mô vận hành khổng lồ và tính chất không đồng nhất của SAGSINs đã đặt ra những thách thức chưa từng có đối với các phương pháp quản lý mạng truyền thống.

Cụ thể, đặc thù cố hữu của SAGSINs là tính động cao độ (high dynamics) do sự di chuyển liên tục với tốc độ cao của hàng ngàn nút mạng vệ tinh LEO và UAV, dẫn đến cấu trúc topo mạng thay đổi theo thời gian thực [5, 9]. Trong môi trường này, các thuật toán tối ưu hóa kinh điển (như Dijkstra hay quy hoạch tuyến tính) vốn được thiết kế cho mạng tĩnh trở nên quá tải về chi phí tính toán và thiếu linh hoạt trong việc thích ứng. Hơn nữa, các thuật toán này thường thất bại trong việc giải quyết bài toán tối ưu hóa đa mục tiêu phức tạp — nơi cần cân bằng đồng thời giữa các chỉ số QoS xung đột nhau như độ trễ, băng thông và hiệu quả năng lượng [10, 19]. Sự bế tắc này tạo ra nhu cầu cấp thiết phải tìm kiếm một cơ chế điều khiển mới có khả năng thích ứng và tự chủ cao hơn.

1.2. Xu hướng phát triển và nhu cầu đổi mới

Xu hướng phát triển của mạng viễn thông thế hệ mới đang chuyển dịch mạnh mẽ từ các mô hình "tự động hóa dựa trên quy tắc" (rule-based automation) sang "mạng tự trị" (autonomous networks) [11]. Trong kỷ nguyên 6G, mạng lưới không chỉ cần băng thông rộng mà còn phải sở hữu khả năng tự cấu hình, tự tối ưu và tự phục hồi. Trí tuệ nhân tạo (AI), đặc biệt là các kỹ thuật Học tăng cường sâu (Deep Reinforcement Learning - DRL), được cộng đồng nghiên cứu xác định là công nghệ then chốt để hiện thực hóa tầm nhìn này [12, 17].

Nhu cầu đổi mới đặt ra là phải thay thế các bảng định tuyến cứng nhắc bằng các tác nhân thông minh (AI Agents) có khả năng "tự học". Các tác nhân này cần tương tác trực tiếp với môi trường, phân tích trạng thái mạng tức thời để đưa ra quyết định định tuyến tối ưu trong thời gian thực, đáp ứng yêu cầu khắt khe của các ứng dụng tương lai.

1.3. Giải pháp tiếp cận mới

Trong khi việc ứng dụng AI là một hướng đi triển vọng, các thuật toán DRL tiêu chuẩn hiện nay chủ yếu tập trung vào tối ưu hóa **giá trị kỳ vọng trung bình (expected value)** của phần thưởng tích lũy [16, 27]. Cách tiếp cận này bộc lộ hạn chế lớn khi vận hành trong môi trường có tính ngẫu nhiên cao như SAGSINs: nó không phản ánh được các kịch bản biến động cực đoan hay các rủi ro tiềm ẩn (như tắc nghẽn đột ngột hoặc mất liên kết vệ tinh). Điều này dẫn đến các quyết định thiếu ổn định ("risk-neutral"), không phù hợp với các dịch vụ yêu cầu độ tin cậy cao.

Để giải quyết vấn đề trên, đề tài đề xuất ứng dụng **Học tăng cường phân phối (Distributional Reinforcement Learning)**, cụ thể là thuật toán **QR-DQN** [20]. Khác với mô hình truyền thống, kiến trúc này cho phép mô hình hóa toàn bộ phân phối xác suất của phần thưởng thay vì chỉ giá trị trung bình. Cách tiếp cận này giúp tác nhân AI có khả năng "nhận thức rủi ro" (risk-sensitive), từ đó đưa ra các chiến lược định tuyến an toàn và ổn định hơn ngay cả trong các điều kiện bất định [23, 24]. Đây là bước đột phá quan trọng để nâng cao độ tin cậy vận hành cho mạng SAGSINs.

1.4. Ý nghĩa thực tiễn và cấp thiết của đề tài

Việc nghiên cứu và ứng dụng thành công mô hình định tuyến nhận thức rủi ro trong SAGSINs mang lại ý nghĩa to lớn trên nhiều phương diện:

- **Về mặt khoa học:** Đề tài góp phần bổ sung vào cơ sở lý thuyết của mạng 6G, chứng minh tính khả thi của việc áp dụng Học tăng cường phân phối vào bài toán tối ưu hóa mạng phức hợp, một hướng đi mới so với các phương pháp DRL truyền thống.
- **Về mặt thực tiễn:**
 - *Đảm bảo dịch vụ trọng yếu:* Cơ chế định tuyến thông minh giúp duy trì kết nối liên mạch cho các ứng dụng đòi hỏi độ tin cậy tuyệt đối như cứu hộ cứu nạn, giám sát thiên tai và an ninh quốc phòng [8, 29].
 - *Tối ưu hóa vận hành:* Giải pháp giúp tự động hóa quá trình ra quyết định, giảm thiểu sự phụ thuộc vào can thiệp thủ công, từ đó cắt giảm chi phí vận hành (OPEX) và nâng cao hiệu quả khai thác tài nguyên vệ tinh [30].

Do đó, đề tài không chỉ giải quyết một bài toán kỹ thuật cụ thể mà còn đóng góp vào chiến lược phát triển hạ tầng số thông minh và bền vững trong tương lai.

2. MỤC ĐÍCH VÀ Ý NGHĨA CỦA ĐỀ TÀI

2.1. Mục đích của đề tài

2.1.1. Mục tiêu tổng quát

Mục tiêu cốt lõi của đề án là nghiên cứu, thiết kế và mô phỏng cơ chế ra quyết định tự chủ ứng dụng **Học tăng cường phân phối (Distributional Reinforcement Learning)** nhằm giải quyết bài toán định tuyến tài nguyên động, đa mục tiêu trong Mạng Tích hợp Không gian – Hàng không – Mặt đất – Biển (SAGSINs) [1, 2]. Hệ thống hướng đến việc tự động học một chính sách định tuyến tối ưu, có khả năng thích ứng linh hoạt với tính động cao và sự không đồng nhất của môi trường mạng, đồng thời đảm bảo các yêu cầu khắt khe về Chất lượng Dịch vụ (QoS) cho các ứng dụng thể hệ mới [3, 4].

Cơ sở khoa học và Lý do lựa chọn giải pháp QR-DQN:

Việc lựa chọn thuật toán **QR-DQN (Quantile Regression DQN)** được đề xuất dựa trên phân tích sâu sắc về những hạn chế mang tính bản chất của môi trường SAGSINs và các phương pháp định tuyến hiện có:

- **Thách thức từ tính ngẫu nhiên của SAGSINs:** Với đặc thù các nút mạng (Vệ tinh LEO, UAV) di chuyển liên tục, topo mạng thay đổi theo thời gian thực và các liên kết truyền dẫn chịu ảnh hưởng mạnh bởi môi trường vật lý [9, 29]. Do đó, các tham số QoS (độ trễ, băng thông, tỷ lệ mất gói) không phải là các giá trị tĩnh định mà tuân theo các phân bố xác suất phức tạp. Điều này đòi hỏi một mô hình có khả năng mô hình hóa phân bố phần thưởng để phản ánh chính xác độ bất định của môi trường [10, 30].
- **Hạn chế của định tuyến truyền thống (Dijkstra):** Các thuật toán kinh điển như Dijkstra vốn được thiết kế cho môi trường tĩnh và tối ưu hóa đơn mục tiêu. Chúng thiếu khả năng thích ứng (non-adaptive) với sự thay đổi topo liên tục và thường thất bại trong việc cân bằng đồng thời nhiều chỉ số QoS mâu thuẫn nhau (trade-off), ví dụ như yêu cầu độ trễ thấp nhưng lại bị giới hạn về băng thông [13, 19].

- **Hạn chế của DRL tiêu chuẩn (Standard DQN):** Các thuật toán DRL phổ biến (như DQN, DDPG) chỉ tập trung tối ưu hóa **giá trị kỳ vọng (mean value)** của phần thưởng tích lũy [16, 27]. Trong các bài toán định tuyến có rủi ro cao, việc chỉ dựa vào giá trị trung bình là không đủ, vì nó bỏ qua thông tin về phương sai và các sự kiện cực đoan (tail events) – ví dụ: các thời điểm độ trễ tăng vọt gây sập kết nối [11].
- **Tính ưu việt của QR-DQN:** Đề tài lựa chọn QR-DQN, một thuật toán tiên tiến thuộc nhóm Học tăng cường phân phối [20]. Bằng cách sử dụng kỹ thuật Hồi quy Phân vị (Quantile Regression), QR-DQN học toàn bộ phân phối xác suất của giá trị Q thay vì chỉ giá trị trung bình.
 - *Cơ chế:* Cho phép tác nhân nhận thức được bức tranh toàn cảnh về rủi ro, bao gồm cả kịch bản "tốt nhất" và "xấu nhất" của môi trường [24, 32].
 - *Kết quả:* Tác nhân học được chính sách "nhận thức rủi ro" (risk-sensitive policy), ưu tiên sự ổn định và đảm bảo các ngưỡng QoS nghiêm ngặt, phù hợp tối ưu cho bài toán định tuyến trong SAGSINs [23].

2.1.2. Các nhiệm vụ nghiên cứu chính

Để hiện thực hóa mục tiêu tổng quát, đề tài tập trung thực hiện ba nhóm nhiệm vụ cụ thể sau:

- **Nghiên cứu nền tảng lý thuyết:**
 - Phân tích kiến trúc SAGSINs, đặc tính vận hành và các thách thức trong việc đảm bảo QoS trong môi trường tích hợp 6G [5, 8].
 - Đánh giá giới hạn của thuật toán Dijkstra và DQN truyền thống trong môi trường động.
 - Nghiên cứu chuyên sâu về toán học của Học tăng cường phân phối và cơ chế Hồi quy Phân vị trong thuật toán QR-DQN [20, 31].
- **Đề xuất và thiết kế giải pháp:**
 - Xây dựng môi trường mô phỏng **SagsEnv** chuẩn hóa theo tiêu chuẩn *Gymnasium*, mô phỏng chính xác động học vệ tinh và các ràng buộc tài nguyên mạng thực tế.
 - Thiết kế kiến trúc mạng nơ-ron cho tác nhân QR-DQN với khả năng nhận thức rủi ro.
 - Xây dựng hàm phần thưởng (Reward Function) đa mục tiêu, tích hợp trọng số hóa các chỉ số QoS (độ trễ, băng thông, độ tin cậy) và hiệu quả tiêu thụ năng lượng [12, 18].
- **Thực nghiệm và Đánh giá:**
 - Huấn luyện tác nhân trong môi trường mô phỏng và phân tích sự hội tụ của thuật toán.
 - Đánh giá hiệu năng thông qua các kịch bản so sánh (benchmarking) với thuật toán Dijkstra và DQN dựa trên các chỉ số định lượng: tỷ lệ thành công (Success Rate), độ trễ trung bình, độ ổn định và khả năng phục hồi lỗi.

2.2. Ý nghĩa của đề tài

Đề tài mang lại những ý nghĩa quan trọng cả về mặt khoa học lẫn thực tiễn, góp phần thúc đẩy sự phát triển của thể hệ mạng tương lai và định hình xu hướng “*mạng tự trị*”.

2.2.1. Ý nghĩa khoa học

Đóng góp về phương pháp luận: Đề tài là một trong những nghiên cứu tiên phong áp dụng cách tiếp cận **Học tăng cường phân phối** vào bài toán định tuyến đa mục tiêu trong mạng SAGSINs. Phương pháp này đánh dấu sự chuyển dịch từ tối ưu hóa "hiệu suất trung bình" sang "quản lý rủi ro" và đảm bảo ràng buộc QoS nghiêm ngặt, khắc phục điểm yếu của DRL truyền thống [23, 27].

Đóng góp vào lý thuyết mạng tự trị: Kết quả nghiên cứu cung cấp bằng chứng thực nghiệm về tính khả thi của việc sử dụng một tác nhân AI (QR-DQN) làm "bộ não" điều khiển trung tâm, có khả năng tự học và vận hành ổn định trong một môi trường quy mô lớn và đầy rẫy tính ngẫu nhiên [11, 17].

Đóng góp về nền tảng mô phỏng: Việc xây dựng thành công môi trường *SagsEnv* mô phỏng chi tiết các ràng buộc QoS và động học vệ tinh cung cấp một nền tảng tham chiếu giá trị cho cộng đồng nghiên cứu DRL trong lĩnh vực viễn thông.

2.2.2. Ý nghĩa thực tiễn

Đặt nền móng cho mạng 6G: SAGSINs là kiến trúc trọng tâm của 6G. Việc phát triển thành công cơ chế định tuyến tự trị, nhận thức rủi ro là bước đi quan trọng để hiện thực hóa tầm nhìn về mạng 6G thông minh và tự chủ hoàn toàn [3, 4].

Đảm bảo độ tin cậy cho các dịch vụ trọng yếu: Hệ thống định tuyến đề xuất có khả năng lường trước rủi ro (như độ trễ tăng vọt, mất kết nối), từ đó đảm bảo độ tin cậy cực cao (Ultra-Reliability) cho các ứng dụng quan trọng như giám sát thiên tai, cứu hộ cứu nạn trên biển và an ninh quốc phòng [8, 29].

Tối ưu hóa chi phí vận hành: Bằng cách tự động hóa quá trình ra quyết định định tuyến – vốn đòi hỏi cấu hình thủ công phức tạp – giải pháp giúp giảm thiểu sự phụ thuộc vào con người, từ đó cắt giảm đáng kể chi phí vận hành (OPEX) và tối ưu hóa hiệu quả khai thác tài nguyên vệ tinh [30].

3. ĐỐI TƯỢNG VÀ PHẠM VI ĐỀ TÀI

3.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu trọng tâm của đề tài là **cơ chế định tuyến và phân bổ tài nguyên động** trong Mạng Tích hợp Không gian – Hàng không – Mặt đất – Biển (SAGSINs). Hệ thống này được điều khiển bởi một tác nhân thông minh (Intelligent Agent) ứng dụng kỹ thuật Học tăng cường sâu (Deep Reinforcement Learning - DRL), cụ thể là mô hình **QR-DQN (Quantile Regression DQN)**.

Mục tiêu là phát triển một cơ chế ra quyết định tự chủ, cho phép tác nhân học được chính sách định tuyến tối ưu π^* , có khả năng cân bằng các tiêu chí Chất lượng Dịch vụ (QoS) đa mục tiêu trong một môi trường đa tầng, biến động cao và phi tất định (non-deterministic) [1, 5].

Hình 1: Mô tả sơ bộ mô hình Học tăng cường sâu DRL

Reinforcement Learning



3.1.1. Cơ sở mô hình QR-DQN và Thuật toán

Nghiên cứu đi sâu vào cấu trúc mạng nơ-ron của QR-DQN, một biến thể tiên tiến thuộc nhóm **Học tăng cường Phân bố (Distributional RL)**. Khác với DQN truyền thống – vốn chỉ tối ưu hóa giá trị kỳ vọng (mean value) của phần thưởng – QR-DQN học toàn bộ phân phối xác suất của giá trị Q thông qua kỹ thuật Hồi quy Phân vị [20].

Chính sách tối ưu $\pi^*(S)$ được xác định bởi công thức:

$$\pi^*(S) = \arg \max_A E [Z(S, A)]$$

Trong đó:

- S : Trạng thái mạng hiện tại (State).
- A : Hành động lựa chọn nút kế tiếp (Action).
- $Z(S, A)$: Phân phối xác suất của phần thưởng ngẫu nhiên (thay vì một giá trị đơn lẻ) khi chọn hành động A tại trạng thái S .
- $E[Z(S, A)]$: Giá trị kỳ vọng của phân phối phần thưởng.

3.1.2. Kiến trúc hệ thống và Môi trường mô phỏng

Hệ thống được thiết kế dưới dạng mô phỏng khép kín với hai thành phần tương tác chính theo giao thức chuẩn *Gymnasium*:

- **Môi trường mạng (SagsEnv)**: Mô phỏng cấu trúc topo 3D động của các thực thể gồm Vệ tinh (LEO/GEO), Trạm mặt đất (Ground Station - GS) và Trạm biển (Sea Station - SS). Môi trường hoạt động theo chu trình khép kín: *Quan sát (Observation)* → *Hành động (Action)* → *Phần thưởng (Reward)*.

- **Tác nhân QR-DQN:** Đóng vai trò “bộ não trung tâm”, liên tục quan sát vector trạng thái, đưa ra quyết định định tuyến và cập nhật chính sách dựa trên tín hiệu phần thưởng phản hồi từ môi trường [31].

3.1.3. Không gian trạng thái và hàm phần thưởng

Để tác nhân hoạt động hiệu quả, hai thành phần then chốt được thiết kế gồm:

- **Không gian trạng thái (State Space):** Là vector đa chiều tại thời điểm t , phản ánh tài nguyên còn lại, trạng thái liên kết và các yêu cầu dịch vụ hiện tại.
- **Hàm phần thưởng (Reward Function):** Được thiết kế dạng đa mục tiêu (Multi-objective), định hướng tác nhân tối ưu hóa đồng thời các chỉ số QoS và hiệu quả sử dụng tài nguyên [12, 18].

Chi tiết các chỉ số cấu thành được mô tả trong các bảng sau:

Bảng 1: Chỉ số về yêu cầu dịch vụ

Chỉ số	Vai trò trong hệ thống	Ảnh hưởng đến phần thưởng
Độ trễ yêu cầu (Required Latency)	Giới hạn thời gian tối đa mà gói tin được phép truyền.	Phần thưởng giảm mạnh nếu độ trễ thực tế vượt ngưỡng.
Thông lượng yêu cầu (Required Uplink/Downlink)	Mức dữ liệu tối thiểu cần đạt được cho mỗi luồng.	Hành động dẫn đến băng thông thấp sẽ bị phạt.
Độ tin cậy yêu cầu (Required Reliability)	Xác suất thành công của đường truyền.	Tăng phần thưởng nếu độ tin cậy đạt hoặc vượt ngưỡng.
Độ ưu tiên (Priority)	Phân biệt các dịch vụ (Emergency > Video > Data).	Thêm trọng số ưu tiên trong hàm phần thưởng.

Bảng 2: Chỉ số về trạng thái đường đi

Chỉ số	Ý nghĩa và nhiệm vụ	Ảnh hưởng đến học tập
Độ trễ tích lũy (Actual Latency)	Tổng thời gian truyền qua các liên kết đã chọn.	Được dùng để tính cost tổng thể (negative reward).
Độ tin cậy tích lũy (Actual Reliability)	Tích xác suất thành công của từng liên kết.	Khuyến khích đường đi ổn định, hạn chế rủi ro.
Băng thông hẹp nhất (Bottleneck Bandwidth)	Giới hạn băng thông tối đa của đường đi.	Giảm reward nếu bottleneck quá nhỏ.
Thời gian chờ còn lại (Timeout)	Khoảng thời gian còn cho phép để hoàn tất truyền.	Nếu timeout < 0 $\rightarrow$ episode kết thúc sớm, penalty lớn.

Bảng 3: Chỉ số về hiệu quả sử dụng tài nguyên

Chỉ số	Vai trò	Cách phản ánh trong reward
Tải xử lý (CPU Load)	Mức sử dụng năng lực xử lý của nút.	Thưởng nếu phân bố tải đều, phạt nếu nút quá tải.
Năng lượng tiêu thụ (Power Consumption)	Đo chi phí năng lượng khi duy trì kết nối.	Giảm reward nếu tiêu thụ vượt ngưỡng tối ưu.
Hiệu quả tài nguyên (Resource Efficiency)	Tỷ lệ giữa dữ liệu truyền được và tài nguyên dùng.	Thưởng cao nếu đạt hiệu suất sử dụng tối đa.

Thông qua việc phân tích liên tục vector trạng thái này, tác nhân học được mối quan hệ đánh đổi (trade-off) giữa các chỉ tiêu QoS và tài nguyên. Ví dụ: tác nhân sẽ học cách hy sinh băng thông để chọn đường có độ tin cậy cao hơn cho các tác vụ khẩn cấp, hoặc tránh các nút đang quá tải để giảm độ trễ. Sự nhận thức rủi ro này giúp QR-DQN hình thành chính sách ổn định hơn các phương pháp truyền thống.

3.2. Phạm vi nghiên cứu

3.2.1. Phạm vi lý thuyết

Tập trung nghiên cứu chuyên sâu về lý thuyết **Học tăng cường sâu (DRL)** và cơ sở toán học của **Hồi quy Phân vị (Quantile Regression)** áp dụng trong mô hình QR-DQN [20, 24].

Nghiên cứu các giao thức và cơ chế vận hành chủ yếu tại **tầng Mạng (Network Layer)** và **tầng Liên kết dữ liệu (Data Link Layer)**.

Giới hạn: Đề tài không đi sâu phân tích các đặc tính vật lý chi tiết của tầng Truyền dẫn (Physical Layer) như kỹ thuật điều chế tín hiệu, mã hóa kênh, nhiễu vật lý chi tiết hay thiết kế phần cứng ăng-ten [7].

3.2.2. Phạm vi kỹ thuật và giải pháp

Giải pháp phần mềm: Tập trung xây dựng kiến trúc thuật toán để tác nhân QR-DQN có thể học và áp dụng trong môi trường mô phỏng máy tính (Computer Simulation). Phạm vi không bao gồm việc triển khai trên phần cứng vệt tinh thực tế hay các thiết bị mạng vật lý (FPGA/DSP).

Công cụ phát triển:

- *Backend:* Ngôn ngữ **Python** kết hợp thư viện **Stable-Baselines3** để triển khai QR-DQN và framework **Gymnasium** để xây dựng môi trường *SagsEnv*.
- *Frontend & Quản lý:* Sử dụng **NodeJS** để xây dựng giao diện trực quan hóa (Visualization) và quản lý tiến trình mô phỏng.
- *Cơ sở dữ liệu:* Sử dụng **MongoDB** (NoSQL) để lưu trữ cấu hình mạng và log kết quả huấn luyện.

3.2.3. Phạm vi thực nghiệm

Môi trường: Triển khai trên môi trường mô phỏng *SagsEnv* quy mô nhỏ và vừa, bao gồm các nút mạng đại diện cho Vệ tinh (Satellite), Trạm mặt đất (Ground Station) và Trạm biển (Sea Station).

Dữ liệu mô phỏng:

- Dữ liệu cấu trúc mạng và hồ sơ QoS được lưu trữ và truy xuất từ MongoDB.
- Vị trí các thực thể (LEO, GS, SS) được khởi tạo dựa trên dữ liệu thực tế và phân bố đều trên bề mặt Trái Đất.
- **Động học vệ tinh:** Quỹ đạo vệ tinh LEO được mô hình hóa thông qua các hàm lượng giác theo thời gian thực (t), đảm bảo tái hiện chính xác sự thay đổi vị trí và tính động của các liên kết trong *SagsEnv* [9].

Kịch bản đánh giá:

- So sánh hiệu năng trực tiếp (Benchmarking) giữa mô hình QR-DQN và thuật toán định tuyến kinh điển **Dijkstra** (Baseline).
- Đánh giá dựa trên các chỉ số định lượng: Tỷ lệ thành công (Success Rate), Mức độ thỏa mãn QoS, Độ trễ trung bình, và Độ lệch chuẩn (Standard Deviation) của các thông số.
- Phân tích quá trình hội tụ (Convergence) và khả năng thích nghi (Adaptability) của tác nhân QR-DQN trước các thay đổi đột ngột của mạng.

4. CẤU TRÚC ĐỀ TÀI

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

Chương này trình bày hệ thống các cơ sở lý thuyết nền tảng cho đề tài. Nội dung được chia thành ba trụ cột chính: (1) Lý thuyết về Mạng máy tính và Hệ điều hành, làm cơ sở cho việc hiểu cơ chế truyền tin và thiết kế môi trường mô phỏng; (2) Tổng quan về Mạng SAGSINs và bài toán định tuyến; và (3) Lý thuyết chuyên sâu về Học tăng cường sâu (DRL), từ các khái niệm cơ bản đến việc lựa chọn mô hình tiên tiến QR-DQN.

1.1. Cơ sở lý thuyết về Hệ điều hành và Mạng máy tính

Mục này trình bày các nguyên lý căn bản về kiến trúc phân tầng trong truyền thông dữ liệu và cơ chế quản lý tài nguyên tính toán của hệ điều hành. Đây là những kiến thức nền tảng chi phối cách thức dữ liệu được đóng gói, truyền tải và xử lý trong các hệ thống tin học.

1.1.1. Các giao thức và kiến trúc tầng Giao vận

Mô hình tham chiếu OSI (Open Systems Interconnection) và bộ giao thức TCP/IP là khung chuẩn mực cho việc thiết kế mạng máy tính. Trong đó, Tầng Giao vận (Transport Layer) đóng vai trò then chốt trong việc đảm bảo chất lượng truyền thông đầu cuối (End-to-End communication). Hai giao thức phổ biến nhất đại diện cho hai triết lý thiết kế đối lập là TCP và UDP.

Giao thức điều khiển truyền vận (TCP - Transmission Control Protocol) TCP là giao thức hướng kết nối (Connection-oriented), được thiết kế với mục tiêu tối thượng là đảm bảo độ tin cậy của dữ liệu (Reliability).

- **Cơ chế hoạt động:** TCP thiết lập một kênh truyền ảo thông qua quy trình "bắt tay ba bước" (Three-way handshake) trước khi truyền dữ liệu.
- **Kiểm soát luồng và tắc nghẽn (Flow & Congestion Control):** Sử dụng cơ chế cửa sổ trượt (Sliding Window) để điều chỉnh tốc độ gửi dữ liệu dựa trên khả năng tiếp nhận của bên nhận và trạng thái của mạng.
- **Đảm bảo toàn vẹn:** TCP sử dụng số thứ tự (Sequence Number) và mã xác nhận (ACK). Nếu gói tin bị mất hoặc lỗi, cơ chế truyền lại (Retransmission) sẽ được kích hoạt tự động.
- **Đặc điểm:** Độ tin cậy cao, đảm bảo thứ tự gói tin, nhưng phát sinh độ trễ (Latency) và chi phí quản lý (Overhead) lớn do các gói tin tiêu đề (Header) và quá trình chờ xác nhận.

Giao thức gói dữ liệu người dùng (UDP - User Datagram Protocol) UDP là giao thức phi kết nối (Connectionless), hoạt động theo mô hình "nỗ lực tối đa" (Best-effort delivery).

- **Cơ chế hoạt động:** UDP gửi các gói dữ liệu (Datagram) đi mà không cần thiết lập kết nối trước và không đảm bảo bên nhận đã nhận được hay chưa.
- **Đặc điểm:** Loại bỏ hoàn toàn các cơ chế kiểm soát luồng, truyền lại và sắp xếp thứ tự. Điều này giúp giảm thiểu tối đa độ trễ và kích thước tiêu đề gói tin.
- **Ứng dụng:** UDP tối ưu cho các ứng dụng yêu cầu tốc độ cao và chấp nhận tỷ lệ lỗi nhất định, ví dụ như truyền tải giọng nói (VoIP) hoặc phát trực tuyến (Streaming).

1.1.2. Lý thuyết về Hệ điều hành

Hệ điều hành (Operating System - OS) chịu trách nhiệm quản lý tài nguyên phần cứng và điều phối các tiến trình phần mềm. Trong các hệ thống tính toán hiện đại, khả năng xử lý song song là yếu tố cốt lõi để tối ưu hóa hiệu năng.

Khái niệm Tiến trình (Process) và Luồng (Thread)

- **Tiến trình:** Là một chương trình đang trong trạng thái thực thi, sở hữu không gian địa chỉ bộ nhớ và tài nguyên hệ thống riêng biệt.
- **Luồng:** Là đơn vị xử lý nhỏ nhất bên trong một tiến trình. Một tiến trình có thể chứa nhiều luồng (Multithreading). Các luồng trong cùng một tiến trình chia sẻ chung không gian bộ nhớ (Code, Data, Files) nhưng có ngăn xếp (Stack) và thanh ghi (Registers) riêng.

Cơ chế Đồng quy (Concurrency) và Đồng bộ hóa (Synchronization) Đa luồng cho phép hệ thống thực hiện nhiều tác vụ có vẻ như đồng thời (Concurrency) trên một lõi CPU hoặc thực sự song song (Parallelism) trên các kiến trúc đa lõi.

- **Tranh chấp dữ liệu (Race Condition):** Khi nhiều luồng cùng truy cập và thay đổi một vùng nhớ chia sẻ mà không có cơ chế kiểm soát, kết quả cuối cùng sẽ phụ thuộc vào thứ tự thực thi ngẫu nhiên, dẫn đến sai lệch dữ liệu.
- **Cơ chế khóa (Locking Mechanisms):** Để giải quyết vấn đề tranh chấp, khoa học máy tính sử dụng các nguyên lý đồng bộ hóa như:
 - *Mutex (Mutual Exclusion):* Đảm bảo tại một thời điểm chỉ có một luồng được quyền truy cập vào vùng găng (Critical Section).
 - *Semaphore:* Biến tín hiệu dùng để kiểm soát số lượng luồng được phép truy cập vào một tài nguyên giới hạn.

Quản lý bộ đệm và Lập lịch (Buffering & Scheduling) Hệ điều hành sử dụng các hàng đợi (Queues) và bộ đệm (Buffers) để quản lý luồng dữ liệu vào/ra (I/O). Bộ lập lịch CPU (CPU Scheduler) sử dụng các thuật toán như Round-Robin hoặc Priority Scheduling để phân chia thời gian xử lý cho các luồng, đảm bảo tính công bằng và hiệu quả sử dụng tài nguyên hệ thống.

1.2. Tổng quan về Mạng Tích hợp Không gian - Hàng không - Mặt đất - Biển (SAGSINs)

1.2.1. Khái niệm và Kiến trúc

Mạng Tích hợp Không gian - Mặt đất - Biển (SAGSINs) được định nghĩa là một kiến trúc mạng hội tụ, đa tầng và đa không gian, được xem là nền tảng cốt lõi cho mạng di động thế hệ thứ sáu (6G) [1, 2]. Mục tiêu của SAGSINs là cung cấp khả năng kết nối toàn cầu, liền mạch, chất lượng cao (QoS) và đáng tin cậy bằng cách tích hợp các phân đoạn mạng riêng lẻ thành một hệ thống đồng nhất [3, 4].

Kiến trúc được mô phỏng trong nghiên cứu này bao gồm ba phân đoạn chính hoạt động hiệp đồng:

- **Phân đoạn Không gian (Space Segment):** Bao gồm các hệ thống vệ tinh ở nhiều quỹ đạo khác nhau (GEO và LEO), đóng vai trò là lớp xương sống (Backbone) cung cấp vùng phủ sóng toàn cầu và kết nối đường dài [5].
- **Phân đoạn Mặt đất (Ground Segment):** Là hạ tầng mạng viễn thông truyền thống, bao gồm các Trạm mặt đất (Ground Stations - GS), đóng vai trò là các cổng Gateway kết nối với mạng lõi (Core Network).
- **Phân đoạn Hàng hải (Sea Segment):** Đảm bảo kết nối vươn ra các vùng biển và đại dương, tích hợp mạng lưới các nút mạng là các Trạm trên biển (Sea Stations - SS) [8, 29].

Lưu ý về phạm vi mô hình hóa: Mặc dù kiến trúc SAGSINs đầy đủ bao gồm Phân đoạn Hàng không (UAVs/HAPs) [6, 9], đề tài này thực hiện **trừu tượng hóa** vai trò của chúng để tập trung vào bài toán định tuyến đường trục (Backbone Routing). Hệ thống giả định rằng các kết nối "chặng cuối" (Last-mile connectivity) đã được xử lý bởi năng lực phủ sóng của các trạm mặt đất và trạm biển. Sự đơn giản hóa này cho phép nghiên cứu tập trung giải quyết thách thức lớn nhất: tối ưu hóa đường đi thông qua hệ thống vệ tinh động LEO/GEO.

1.2.2. Đặc điểm và Thách thức

SAGSINs sở hữu những đặc điểm cơ bản tạo nên sự phức tạp vượt trội so với mạng mặt đất truyền thống:

- **Tính không đồng nhất (Heterogeneity):** Sự đa dạng lớn về đặc tính vật lý (bán kính phủ sóng, quỹ đạo) và tài nguyên (băng thông Uplink/Downlink, năng lực CPU, năng lượng) giữa các nút mạng [13].
- **Tính động cao độ (High Dynamics):** Topo mạng biến đổi liên tục theo thời gian thực do sự di chuyển tốc độ cao của các chùm vệ tinh LEO. Các liên kết (Inter-Satellite Links - ISL) thường xuyên bị ngắt/kết nối lại [9].
- **Quy mô không gian trạng thái:** Số lượng lớn các nút mạng tạo ra một không gian trạng thái khổng lồ, gây khó khăn cho các thuật toán tìm kiếm truyền thống.
- **Môi trường truyền dẫn khắc nghiệt:** Tín hiệu đối mặt với độ trễ truyền dẫn lớn (phụ thuộc khoảng cách) và các suy hao do môi trường vật lý.

1.2.3. Bài toán định tuyến đa mục tiêu và QoS

1.2.3.1. Định nghĩa bài toán

Bài toán đặt ra là tìm một đường đi $P = n_1, n_2, n_3, \dots, n_k$ từ nút nguồn đến nút đích sao cho tối ưu hóa được một hàm mục tiêu tổng hợp, đồng thời thỏa mãn các ràng buộc về Chất lượng Dịch vụ QoS.

1.2.3.2. Các chỉ số QoS

Hệ thống định tuyến cần cân bằng các chỉ số sau [10, 30]:

- Độ trễ (Latency - L): Tổng thời gian truyền gói tin trên đường đi, bao gồm trễ truyền dẫn (L_{proc}) và trễ xử lý tại nút (L_{link}).

$$L_{total} = \sum_{i=1}^{k-1} L_{link(i,i+1)} + \sum_{i=1}^k L_{proc(i)}$$

- Độ tin cậy (Reliability - R): Xác suất thành công của việc truyền tin, tính bằng tích độ tin cậy của các liên kết thành phần (R_{link}).

$$R_{total} = \prod_{i=1}^{k-1} R_{link(i,i+1)}$$

- Băng thông (Throughput): Bị giới hạn bởi liên kết có băng thông nhỏ nhất trên đường đi (nút cổ chai). Băng thông gồm:
 - UpLink Bandwidth:

$$UL_{path} = \min(UL_{link(1,2)}, \dots, UL_{link(k-1,k)})$$

- DownLink Bandwidth:

$$DL_{\text{path}} = \min(DL_{\text{link}(1,2)}, \dots, DL_{\text{link}(k-1,k)})$$

- Hiệu quả năng lượng và cân bằng tải: Ưu tiên các nút có tài nguyên CPU và năng lượng dồi dào để tránh điểm nóng (hotspot) gây tắc nghẽn cục bộ.

1.3. Các công trình liên quan và Khoảng trống nghiên cứu

1.3.1. Tổng quan các phương pháp hiện có

Các phương pháp truyền thống (Traditional/Deterministic):

- Sử dụng mô hình đồ thị chụp nhanh (Snapshot) để chia mạng động thành các lát cắt tĩnh.
- Áp dụng thuật toán **Dijkstra** hoặc **Bellman-Ford** để tìm đường ngắn nhất [8].
- *Nhược điểm*: Chi phí tính toán lại quá lớn khi topo thay đổi; chỉ tối ưu đơn mục tiêu; không thích ứng được với lỗi mạng bất ngờ.

Các thuật toán Meta-heuristic:

- Sử dụng Giải thuật Di truyền (GA), Tối ưu hóa bầy đàn (PSO), hoặc đàn kiến (ACO) để giải quyết bài toán đa mục tiêu NP-khó.
- *Nhược điểm*: Tốc độ hội tụ chậm, không đáp ứng được yêu cầu xử lý thời gian thực (real-time) của mạng 6G.

Học tăng cường sâu (Standard DRL):

- Sử dụng **DQN**, **DDPG**, **PPO** để học chính sách định tuyến trực tiếp từ dữ liệu [12, 16, 17].
- *Ưu điểm*: Ra quyết định nhanh, không cần mô hình toán học cứng nhắc của môi trường.

1.3.2. Khoảng trống nghiên cứu (Research Gap)

Mặc dù các phương pháp DRL tiêu chuẩn (như DQN) đã đạt được kết quả khả quan [12, 15, 18], chúng tồn tại một hạn chế cốt lõi về mặt lý thuyết khi áp dụng vào môi trường rủi ro cao như SAGSINs: **Tính trung tính với rủi ro (Risk-Neutrality)**.

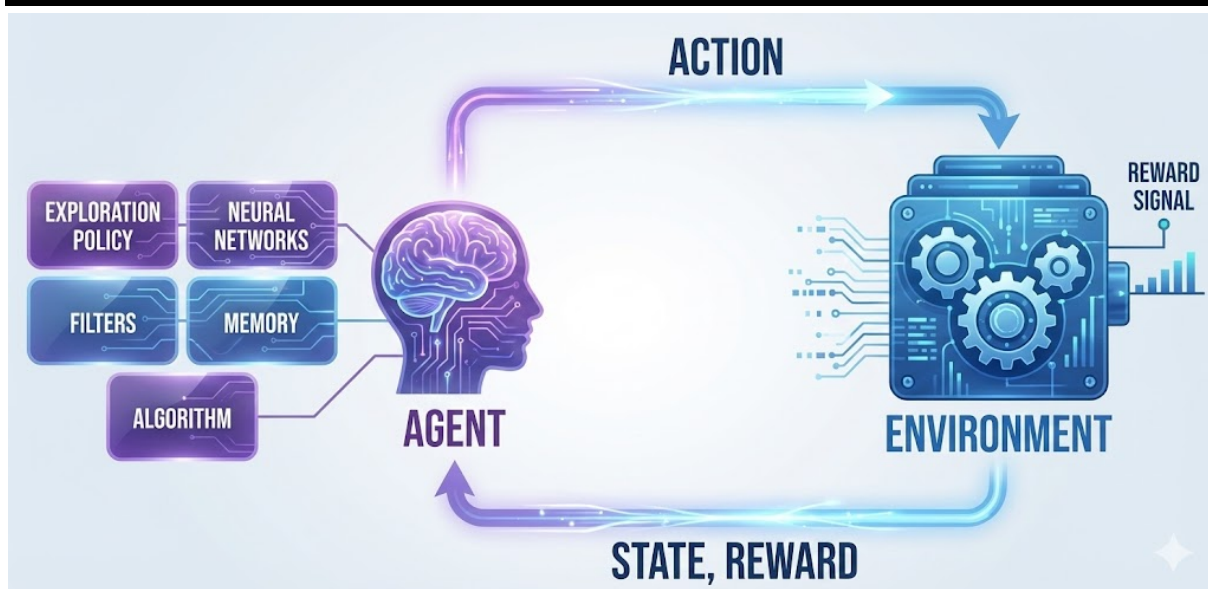
Standard DQN chỉ tối ưu hóa giá trị kỳ vọng (Expected Value) của phần thưởng:

$$Q(S, A) = E[R]$$

Điều này đồng nghĩa với việc DQN không thể phân biệt giữa một tuyến đường "ổn định" và một tuyến đường "rủi ro" (có phương sai lớn) nếu chúng có cùng giá trị trung bình [20, 24]. Đây là động lực để đề tài lựa chọn QR-DQN, một phương pháp tiếp cận tiên tiến giúp quản lý rủi ro thông qua việc học phân phối xác suất [20, 23, 31].

1.4. Cơ sở Lý thuyết về Học tăng cường sâu (DRL)

Hình 2: Mô tả tương tác giữa Agent – Environment

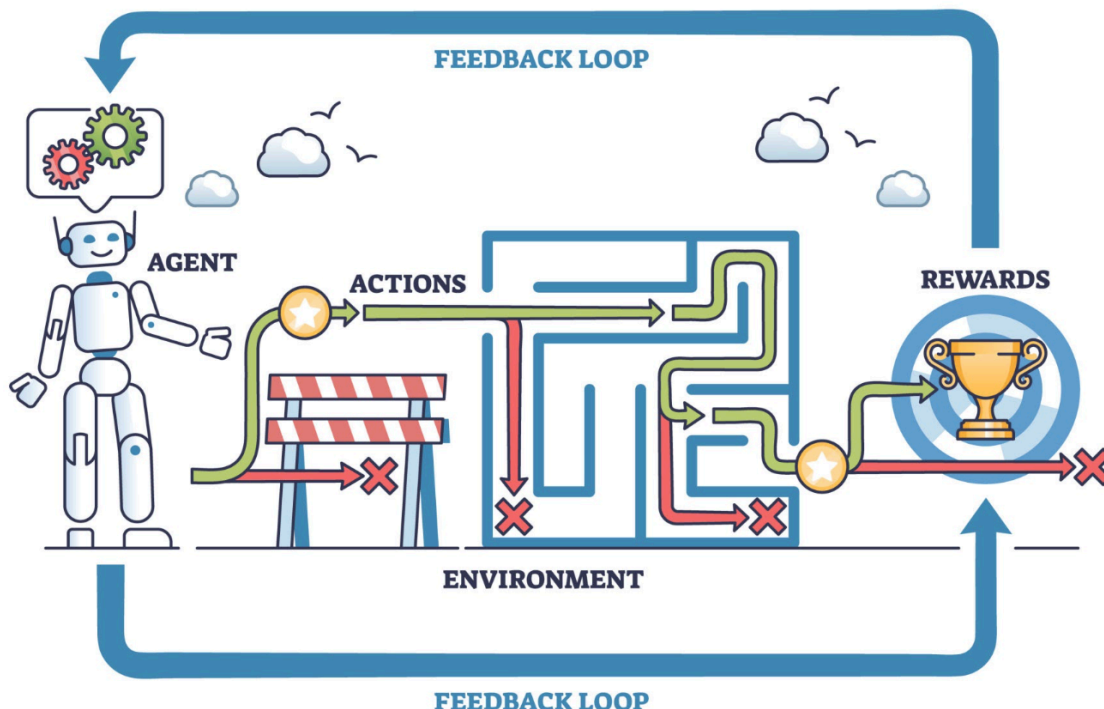


Mục này trình bày nền tảng lý thuyết từ Học tăng cường cơ bản đến kiến trúc mạng nơ-ron sâu (Deep Q-Network) và đi sâu phân tích thuật toán QR-DQN (Quantile Regression DQN) – giải pháp trọng tâm được đề xuất để giải quyết bài toán định tuyến nhận thức rủi ro trong mạng SAGSINs.

1.4.1. Giới thiệu chung về DRL

Hình 3: Mô tả tổng quan về Học tăng cường sâu DRL

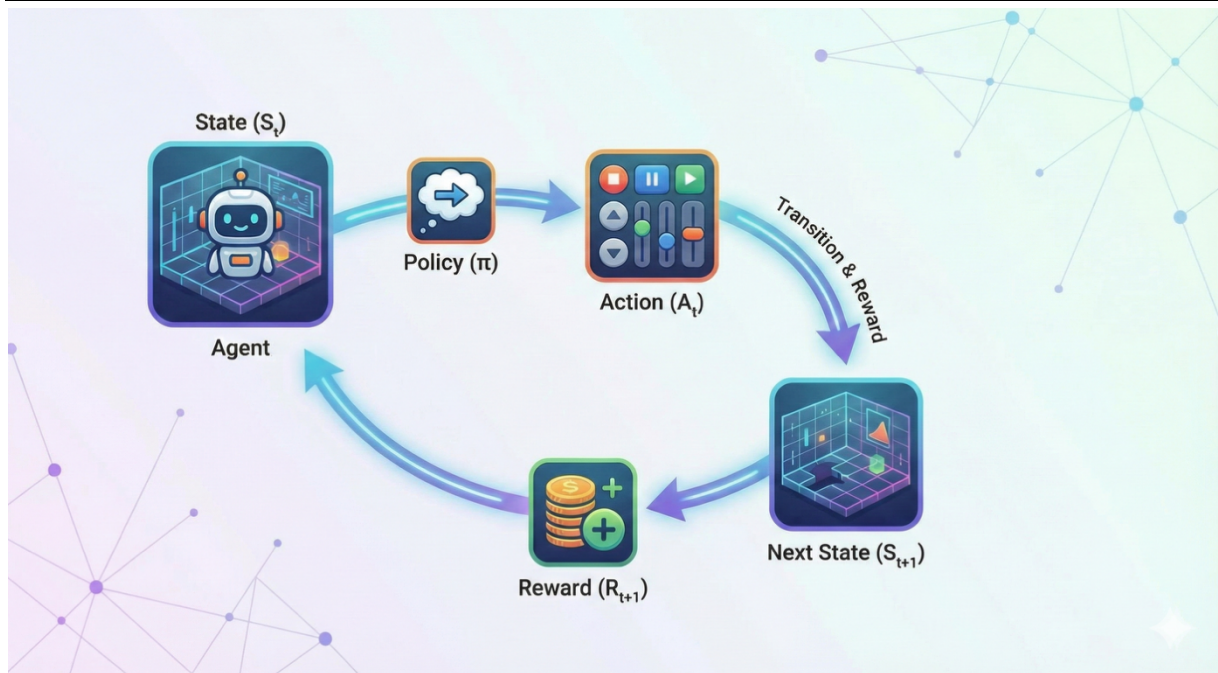
REINFORCEMENT LEARNING



Học tăng cường (RL) là một nhánh của Học máy, trong đó một **Tác nhân (Agent)** học cách ra quyết định thông qua quá trình thử-và-sai tương tác với **Môi trường (Environment)** để tối đa hóa phần thưởng tích lũy [20].

1.4.2. Quy trình Quyết định Markov (MDP)

Hình 4: Mô tả tổng quan quy trình Quyết định Markov (Markov Decision Process - MDP)



Bài toán RL được mô hình hóa bằng Quy trình Quyết định Markov (MDP) (S, A, P, R, γ) [11]. Khi không gian trạng thái S trở nên quá lớn (như trong mạng SAGSINs), các phương pháp bảng (Table-based) trở nên bất khả thi [2,10]. **Deep Reinforcement Learning (DRL)** ra đời bằng cách kết hợp Mạng nơ-ron sâu (DNN) để xấp xỉ các hàm giá trị hoặc chính sách, cho phép giải quyết các bài toán có chiều dữ liệu cao [12, 15, 17]. Trong quy trình này, các tham số mang ý nghĩa sau [17, 18]:

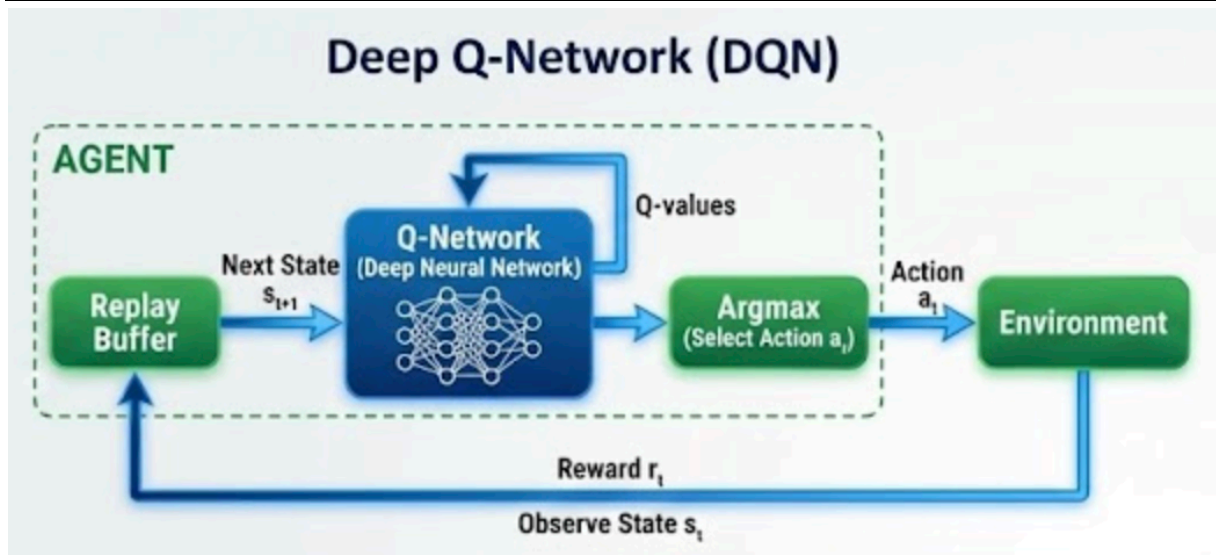
- S (State Space): Tập các trạng thái mô tả đầy đủ thông tin của môi trường tại thời điểm t , giúp tác nhân xác định tình huống hiện tại.
- A (Action Space): Không gian hành động, tập hợp các quyết định định tuyến khả thi.
- P (Transition Probability): Xác suất chuyển từ trạng thái s sang s' khi thực hiện hành động a .
- R (Reward Function): Hàm phần thưởng $R(s, a)$, tín hiệu phản hồi đánh giá chất lượng của hành động.
- γ (Discount Factor): Hệ số chiết khấu ($0 \leq \gamma \leq 1$), cân bằng giữa lợi ích tức thời và mục tiêu dài hạn.

1.4.3. Các mô hình DRL nổi bật và sự chuyển dịch công nghệ

Trước khi đi đến giải pháp đề xuất, ta xem xét các mô hình DRL tiêu biểu đã được ứng dụng trong định tuyến:

- **Deep Q-Network (DQN)** [15, 17, 18] :

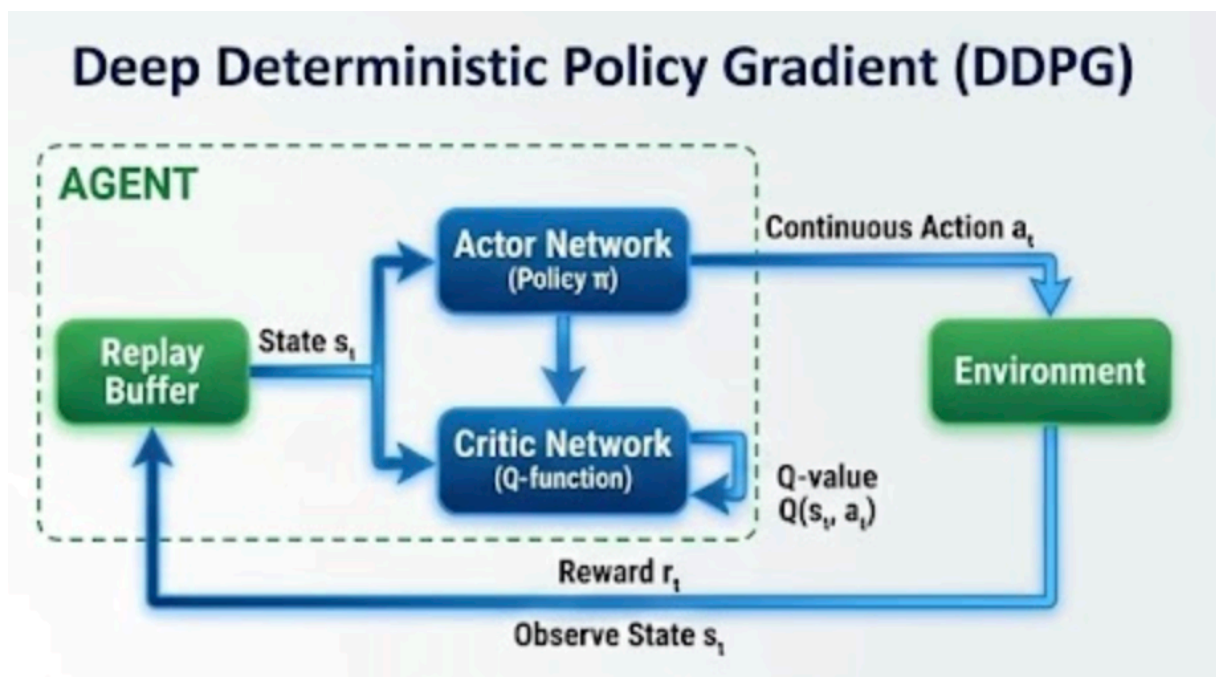
Hình 5: Mô tả mô hình Deep Q-Network



- *Nguyên lý*: Là thuật toán tiên phong kết hợp Q-Learning với Deep Neural Networks. DQN xấp xỉ hàm giá trị $Q(s,a)$ bằng một mạng nơ-ron.
- *Hạn chế*: DQN chỉ học giá trị trung bình (Mean Value) của phần thưởng. Nó thường gặp vấn đề "đánh giá quá cao" (Overestimation) và kém ổn định trong môi trường ngẫu nhiên cao.

• **Deep Deterministic Policy Gradient (DDPG) & PPO [10, 12, 16, 27]:**

Hình 6: Mô tả Mô hình Deep Deterministic Policy Gradient

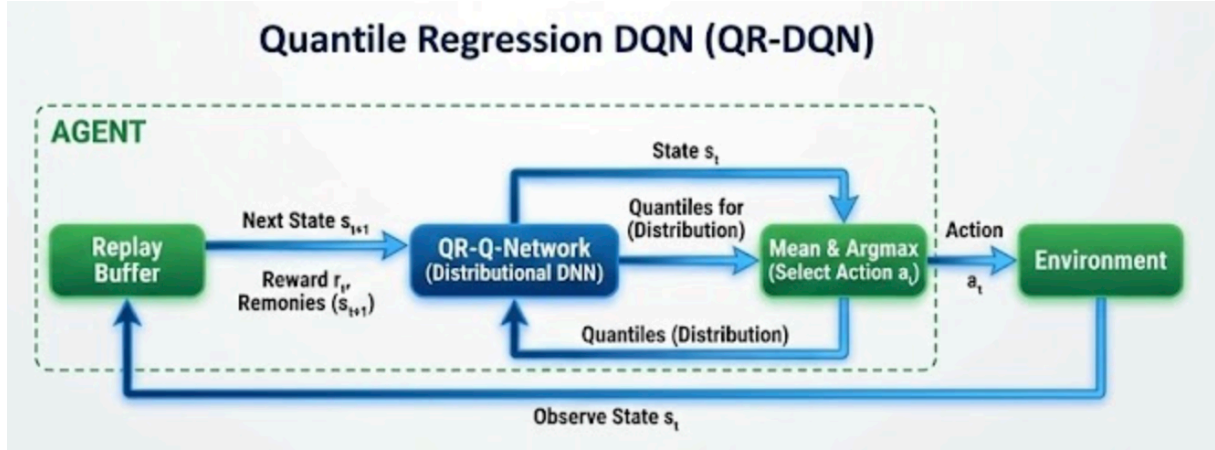


- *Nguyên lý*: Thuộc nhóm Actor-Critic, phù hợp với không gian hành động liên tục.
- *Hạn chế*: Vẫn dựa trên tối ưu hóa kỳ vọng, chưa giải quyết triệt để bài toán quản lý rủi ro.

Khoảng trống nghiên cứu: Các mô hình trên đều là "**Risk-Neutral**" (Trung tính với rủi ro). Trong mạng SAGSINs, nơi một liên kết vệ tinh có thể bị ngắt đột ngột, việc chỉ nhìn vào "giá trị trung bình" là không đủ. Ta cần một mô hình có thể nhìn thấy "phân phối" của rủi ro.

1.4.4. Giải pháp đề xuất: Học tăng cường phân bố (Distributional RL)

Hình 7: Mô tả Mô hình Quantile Regression DQN



Để khắc phục hạn chế của việc tối ưu hóa giá trị kỳ vọng trong DQN, đề tài áp dụng mô hình **QR-DQN (Quantile Regression Deep Q-Network)** [20]. Đây là một bước tiến trong lĩnh vực Học tăng cường Phân bố (Distributional RL), cho phép tác nhân xấp xỉ toàn bộ phân phối xác suất của phần thưởng thay vì chỉ ước lượng giá trị trung bình [20, 24].

Hình thức hóa bài toán Phân bố (Distributional Formalism)

Trong môi trường SAGSINs, phần thưởng tích lũy không phải là một số vô hướng mà là một biến ngẫu nhiên $Z(S, A)$. Phương trình Bellman phân bố (Distributional Bellman Equation) được định nghĩa như sau [20, 31]:

$$Z(X, A) = {}^D R(X, A) + \gamma Z(X', A^*)$$

Trong đó:

- X, A : Không gian trạng thái và hành động.
- $=^D$: Biểu thị sự đồng nhất về mặt phân phối xác suất.
- A^* : Hành động tối ưu tại trạng thái kế tiếp, $A^* = \arg \max_{A'} E [Z(X', A')]$.

Xấp xỉ Phân vị (Quantile Approximation)

QR-DQN xấp xỉ phân phối của $Z(S, A)$ bằng một phân phối rời rạc đồng nhất, được tham số hóa bởi tập hợp N giá trị hỗ trợ (supports) $\{\theta_1, \theta_2, \dots, \theta_N\}$ [20, 26].

Hàm phân phối xác suất tích lũy (CDF) được xấp xỉ như sau:

$$Z_\theta(S, A) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(S, A)}$$

Trong đó δ_z là phân phối Dirac delta tại giá trị z . Mỗi giá trị hỗ trợ $\theta_i(S, A)$ tương ứng với một mức phân vị cố định τ_i , được xác định bởi công thức trung điểm (midpoint quantile):

$$\tau_i = \frac{i - 0.5}{N}, \quad \forall i \in \{1, \dots, N\}$$

Như vậy, đầu ra của mạng nơ-ron không phải là một giá trị Q duy nhất, mà là vector $\theta(S, A) \in \mathbb{R}^N$ mô tả hình dạng của phân phối phân thưởng.

Cơ chế Học qua Hồi quy Phân vị (Quantile Regression Learning)

Mục tiêu huấn luyện của QR-DQN là cực tiểu hóa khoảng cách 1-Wasserstein (Earth Mover's Distance) giữa phân phối dự đoán và phân phối mục tiêu [20, 24]. Quá trình này được chuyển đổi thành bài toán Hồi quy Phân vị.

Xác định Phân phối Mục tiêu (Target Distribution):

Với một mẫu chuyển đổi (S, A, R, S') , phân phối mục tiêu $\mathcal{TZ}$ được xây dựng từ mạng mục tiêu (Target Network) với tham số θ^- :

$$y_j = r + \gamma \theta_j^-(S', A^*) \quad \forall j \in \{1, \dots, N\}$$

Hàm mất mát Quantile Huber Loss:

Để đảm bảo tính trơn của đạo hàm tại điểm 0 và sự ổn định khi sai số lớn, QR-DQN sử dụng hàm mất mát Huber áp dụng cho từng phân vị [20, 31].

Xét sai số giữa giá trị mục tiêu thứ j và giá trị dự đoán thứ i :

$$u_{ij} = y_j - \theta_i(s, a)$$

Hàm mất mát cho cặp phân vị (i, j) được định nghĩa:

$$\rho_{\tau_i}^\kappa(u_{ij}) = \mathcal{L}_\kappa(u_{ij}) \cdot \left| \tau_i - I_{\{u_{ij} < 0\}} \right|$$

Trong đó:

- $I_{\{u < 0\}}$: Hàm chỉ thị, bằng 1 nếu $u < 0$ (dự đoán lớn hơn thực tế) và 0 ngược lại. Thành phần $|\tau_i - I|$ tạo ra trọng số bất đối xứng đặc trưng của hồi quy phân vị [20].
- $\mathcal{L}_\kappa(u)$: Hàm Huber Loss tiêu chuẩn để hạn chế bùng nổ gradient:

$$\mathcal{L}_\kappa(u) = \begin{cases} 0.5u^2 & \text{nếu } |u| \leq \kappa \\ \kappa(|u| - 0.5\kappa) & \text{nếu } |u| > \kappa \end{cases}$$

Hàm mục tiêu tổng quát:

Hàm mất mát tổng thể $\mathcal{L}(\theta)$ được tính bằng cách lấy trung bình sai số giữa tất cả các cặp phân vị dự đoán và mục tiêu:

$$\mathcal{L}_{\mathcal{QR-DQN}}(\theta) = \sum_{i=1}^N E_j \left[\rho_{\tau_i}^\kappa(y_j - \theta_i(S, A)) \right]$$

Quá trình tối ưu hóa sử dụng thuật toán Gradient Descent ngẫu nhiên (SGD) hoặc Adam để cập nhật trọng số mạng nhằm cực tiểu hóa $\mathcal{L}(\theta)$.

Chính sách Ra quyết định (Decision Policy)

Mặc dù mô hình học toàn bộ phân phối, hành động tối ưu vẫn có thể được chọn dựa trên giá trị kỳ vọng (trung bình của các phân vị) để đảm bảo tối đa hóa phần thưởng tổng thể:

$$A^* = \arg \max_{A \in \mathcal{A}} Q(S, A) = \arg \max_{A \in \mathcal{A}} \left(\frac{1}{N} \sum_{i=1}^N \theta_i(S, A) \right)$$

Tuy nhiên, kiến trúc này mở ra khả năng áp dụng các chiến lược nhận thức rủi ro (Risk-sensitive Policies) [23, 24]. Ví dụ, trong các tình huống yêu cầu độ tin cậy cực cao, hệ thống có thể chọn hành động dựa trên giá trị rủi ro có điều kiện (CVaR) bằng cách chỉ tối ưu hóa trên tập con các phân vị thấp ($k < N$) [24, 27]:

$$A_{risk}^* = \arg \max_{A \in \mathcal{A}} \left(\frac{1}{k} \sum_{i=1}^k \theta_i(S, A) \right)$$

1.5. Khả năng ứng dụng của DRL vào bài toán Giao tiếp Cộng tác trong SAGSINs

Việc ứng dụng DRL vào bài toán định tuyến trong SAGSINs không chỉ là một sự thay thế kỹ thuật mà là một bước chuyển đổi mô hình (paradigm shift) từ "tĩnh" sang "động" [11]. Khả năng ứng dụng này được thể hiện qua ba khía cạnh cốt lõi:

- **Học Chính sách Định tuyến Động (Dynamic Policy Learning):** Thay vì tuân theo các quy tắc cứng nhắc như trong thuật toán Dijkstra hay OSPF (vốn yêu cầu tính toán lại toàn bộ khi topo thay đổi) [10, 18], tác nhân DRL học một chính sách $\pi_\theta(A|S)$ linh hoạt [17]. Chính sách này cho phép tự động cân bằng các mục tiêu QoS mâu thuẫn nhau (Trade-off) – ví dụ: chấp nhận một đường đi dài hơn (nhiều hop hơn) để đảm bảo băng thông cao, hoặc hy sinh băng thông để ưu tiên độ trễ thấp nhất cho các tác vụ khẩn cấp [14, 27].
- **Ra quyết định dựa trên Bối cảnh (Context-aware Decision Making):** Vector trạng thái S_t được thiết kế để cung cấp cho tác nhân một cái nhìn toàn diện:
 - *Thông tin cục bộ (Local Scope):* Tài nguyên (Buffer, CPU) của các nút lân cận.
 - *Thông tin toàn cục (Global Scope):* Khoảng cách ước lượng đến đích, tổng số hop đã đi.

Sự kết hợp này cho phép tác nhân thực hiện cơ chế "Giao tiếp cộng tác", trong đó mỗi quyết định chuyên tiếp gói tin (hop-by-hop) đều hướng tới lợi ích tối ưu của toàn mạng lưới thay vì tối ưu cục bộ tham lam [10].

- **Khả năng Tự phục hồi và Thích ứng (Self-healing & Adaptability):** Trong môi trường SAGSINs, khi một vệ tinh di chuyển ra khỏi vùng phủ hoặc một liên kết bị ngắt do vật cản, trạng thái môi trường S_t sẽ thay đổi tức thời [1, 5]. Do tác nhân DRL đã được huấn luyện trên hàng triệu kịch bản biến động, nó có khả năng đưa ra quyết định thay thế tối ưu ngay lập tức (Inference time cực thấp) mà không cần tái cấu hình hệ thống, đảm bảo tính liên tục của dịch vụ [9, 18].

1.5.1. Ưu thế của QR-DQN

Sự khác biệt căn bản giữa mô hình đề xuất (QR-DQN) và các mô hình DRL truyền thống (DQN/DDPG) nằm ở cách tiếp cận thông tin phản hồi từ môi trường [20].

Hạn chế của Ước lượng Kỳ vọng (Expectation Estimation)

Các thuật toán DRL tiêu chuẩn mô hình hóa giá trị hành động $Q(S, A)$ là giá trị kỳ vọng của phần thưởng tích lũy [15, 16]:

$$Q(S, A) = E[Z(S, A)]$$

Phương pháp này "nén" toàn bộ sự bất định của tương lai thành một con số trung bình. Hệ quả là tác nhân trở nên "trung tính với rủi ro" (Risk-neutral): nó không thể phân biệt giữa một hành động mang lại phần thưởng ổn định và một hành động có biến động lớn (rủi ro cao) nếu chúng có cùng giá trị trung bình [20, 24].

Sức mạnh của Phân phối (Distributional Perspective)

QR-DQN tiếp cận theo hướng Học tăng cường Phân bố, học toàn bộ phân phối xác suất của biến ngẫu nhiên $Z(S, A)$. Thay vì một giá trị vô hướng, hàm Q được biểu diễn bởi tập hợp N phân vị rời rạc [20, 26]:

$$Z_{\theta}(S, A) \approx \{\theta_1, \theta_2, \dots, \theta_N\}$$

Trong đó mỗi θ_i tương ứng với một mức xác suất τ_i . Điều này cung cấp cho tác nhân một "bản đồ rủi ro" chi tiết: các phân vị thấp ($\tau < 0.2$) đại diện cho các kịch bản xấu nhất (Worst-case scenarios), trong khi các phân vị cao đại diện cho các kịch bản tốt nhất [23, 31].

1.5.2. Ứng dụng định tuyến Nhận thức Rủi ro (Risk-Aware Routing)

Trong bối cảnh cụ thể của mạng SAGSINs, khả năng học phân phối của QR-DQN mang lại những ưu thế vượt trội cho bài toán định tuyến:

- **Phân biệt Rủi ro Tuyến đường:** Xét ví dụ hai tuyến đường A và B có cùng độ trễ trung bình là 50ms.
 - *Tuyến A:* Độ trễ ổn định dao động 45-55ms.
 - *Tuyến B:* Độ trễ thường là 20ms, nhưng thỉnh thoảng tắc nghẽn lên tới 200ms. Một tác nhân DQN truyền thống sẽ coi hai tuyến này tương đương (hoặc thậm chí chọn B). Ngược lại, QR-DQN sẽ phát hiện ra "đuôi dài" (heavy tail) trong phân phối của tuyến B ở các phân vị thấp [20, 24]. Tác nhân có thể được cấu hình để tránh tuyến B nhằm đảm bảo cam kết chất lượng dịch vụ (SLA).
- **Độ ổn định trong Môi trường Phi tĩnh:** Bằng cách tối ưu hóa khoảng cách Wasserstein giữa các phân phối, QR-DQN hội tụ ổn định hơn và ít bị ảnh hưởng bởi nhiễu (noise) trong quá trình huấn luyện so với việc tối ưu hóa sai số bình phương trung bình (MSE) của DQN [20, 21].
- **Tối ưu hóa Đa mục tiêu Linh hoạt:** Tác nhân có thể học cách đánh đổi thông minh: chấp nhận giảm băng thông (chọn phân vị trung bình) để đổi lấy độ tin cậy cực cao (tối ưu hóa phân vị thấp) cho các dịch vụ y tế/quân sự, hoặc ngược lại cho các dịch vụ giải trí [24, 27].

1.6. Tổng kết chương

Chương này đã hoàn thành việc hệ thống hóa toàn diện các cơ sở lý thuyết nền tảng, tạo dựng khung tham chiếu vững chắc cho toàn bộ đề tài nghiên cứu. Khởi đầu bằng việc phân tích sâu kiến trúc đa tầng của Mạng SAGSINs, nội dung chương đã làm nổi bật những thách thức vận hành cố hữu như tính không đồng nhất, quy mô lớn và đặc biệt là tính động cao độ của các

nút mạng vệ tinh. Để có cơ sở mô hình hóa chính xác các ràng buộc vật lý phức tạp này trong môi trường mô phỏng, các nguyên lý cốt lõi về mạng máy tính, bao gồm giao thức truyền tải và cơ chế quản lý tài nguyên hệ điều hành, đã được xác lập làm hệ quy chiếu kỹ thuật.

Quan trọng hơn hết, chương đã cung cấp những luận cứ khoa học thuyết phục cho việc lựa chọn giải pháp công nghệ thông qua việc phân tích sự chuyển dịch từ Học tăng cường sâu truyền thống sang Học tăng cường Phân bố. Trong đó, thuật toán QR-DQN với cơ chế Hồi quy Phân vị đã được chứng minh là phương pháp tối ưu nhờ khả năng "nhận thức rủi ro", cho phép giải quyết hiệu quả bài toán đảm bảo Chất lượng Dịch vụ (QoS) trong môi trường đầy biến động. Những nền tảng lý thuyết này chính là tiền đề quan trọng để **Chương 2** đi sâu vào chi tiết phương pháp đề xuất, bao gồm việc thiết kế môi trường mô phỏng SagsEnv theo chuẩn Gymnasium và hiện thực hóa thuật toán điều khiển thông minh.

CHƯƠNG 2: PHƯƠNG PHÁP ĐỀ XUẤT VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày phương pháp luận và kiến trúc hệ thống tổng thể nhằm giải quyết bài toán định tuyến động trong mạng SAGSINs. Trọng tâm của chương là việc chuyển đổi bài toán vật lý phức tạp của mạng vệ tinh thành một Quy trình Quyết định Markov (MDP) chuẩn hóa để huấn luyện tác nhân QR-DQN. Đồng thời, chương đi sâu vào thiết kế hàm phần thưởng đa mục tiêu nhằm cân bằng giữa các chỉ số QoS và hiệu quả tài nguyên.

Nội dung cụ thể bao gồm: (i) Thiết kế kiến trúc 3 tầng; (ii) Mô hình hóa không gian trạng thái/hành động; (iii) Phương pháp chuẩn hóa dữ liệu; (iv) Cấu trúc mạng nơ-ron QR-DQN và (v) Quy trình huấn luyện - đánh giá.

2.1. Nguyên tắc thiết kế

Để giải quyết hiệu quả các thách thức về tính động và quy mô lớn của mạng SAGSINs, hệ thống được thiết kế dựa trên ba nguyên tắc nền tảng xuyên suốt:

- Tự chủ trong chu trình OODA (Observe - Orient - Decide - Act):** Hệ thống vận hành hoàn toàn độc lập, khép kín chu trình điều khiển từ việc thu thập quan sát trạng thái mạng, phân tích tình huống, ra quyết định định tuyến đến thực thi và cập nhật trạng thái mà không cần sự can thiệp thủ công của con người.
- Tính thích ứng theo thời gian thực (Real-time Adaptability):** Trước sự biến đổi liên tục của môi trường (vị trí vệ tinh thay đổi theo quỹ đạo, sự biến động tài nguyên tại các nút), hệ thống phải có khả năng học hỏi (Learning) và phản ứng linh hoạt. Cơ chế suy luận (Inference) cần đảm bảo độ trễ thấp nhất để đáp ứng các yêu cầu định tuyến tức thời.
- Tối ưu hóa đa mục tiêu (Multi-objective Optimization):** Quyết định định tuyến không chỉ dựa trên một tiêu chí đơn lẻ (như khoảng cách ngắn nhất) mà phải là kết quả của sự cân bằng tối ưu giữa các yếu tố QoS mâu thuẫn nhau (Độ trễ - Băng thông - Độ tin cậy) và hiệu quả sử dụng năng lượng, hướng tới việc tối đa hóa hàm phần thưởng tổng thể.

2.2. Kiến trúc hệ thống đề xuất

Hệ thống được xây dựng theo kiến trúc phân tầng (Layered Architecture), trong đó mỗi tầng đảm nhận một chức năng chuyên biệt và tương tác chặt chẽ qua các giao diện chuẩn hóa, hình thành một vòng lặp điều khiển thông minh khép kín.

Tầng 1: Hạ tầng Mô phỏng (Infrastructure Layer)

Đây là tầng nền tảng, chịu trách nhiệm mô phỏng các thực thể vật lý của mạng SAGSINs. Các đối tượng được định nghĩa hướng đối tượng (OOP) và khởi tạo dựa trên dữ liệu cấu hình từ cơ sở dữ liệu MongoDB.

Các thành phần thực thể:

- Vệ tinh (Satellites):** Bao gồm các nút mạng trên quỹ đạo tầm thấp (LEO) và địa tĩnh (GEO). Đặc biệt, đối với vệ tinh LEO, vị trí không gian (x, y, z) được cập nhật liên tục theo thời gian thực t dựa trên các mô hình quỹ đạo sử dụng hàm lượng giác, đảm bảo tái hiện chính xác động học di chuyển quanh Trái Đất.

- **Trạm mặt đất (Ground Stations - GS):** Các nút cố định trên đất liền, đóng vai trò là cổng kết nối (Gateway) đến mạng lõi Internet.
- **Trạm biển (Sea Stations - SS):** Các nút cố định hoặc nổi trên đại dương, đảm bảo tính liên tục của kết nối tại các vùng biển xa.

Dữ liệu vị trí của các trạm GS và SS được lấy từ thực tế và phân bố đều trên bề mặt mô phỏng để phản ánh đặc trưng bao phủ toàn cầu.

***Lưu ý về phạm vi:** Đề tài thực hiện trừu tượng hóa phân đoạn UAVs và giả định rằng kết nối chặng cuối (Last-mile) đã được xử lý bởi năng lực phủ sóng của GS/SS. Sự đơn giản hóa này giúp giảm bớt sự bùng nổ không gian trạng thái, cho phép nghiên cứu tập trung sâu vào bài toán cốt lõi: định tuyến đường trục (Backbone Routing) qua hệ thống vệ tinh động.*

Trong đề tài này, hệ thống được thiết kế số lượng các nút như sau:

Bảng 4: Số lượng nút trong mạng SAGGINS

Thành phần	Số lượng	Mô tả
Vệ tinh (LEO/GEO)	60	Di chuyển theo quỹ đạo, cung cấp kết nối động.
Trạm mặt đất (GS)	16	Điểm đến cuối cùng của các yêu cầu, kết nối mạng lõi.
Trạm biển (SS)	10	Các nút cố định tại khu vực hải dương.
Người dùng (Requests)	Liên tục	Sinh ngẫu nhiên theo thời gian, với nhiều loại dịch vụ khác nhau.

Tầng 2: Môi trường Tương tác (Environment Interface Layer)

Tầng này đóng vai trò cầu nối, hiện thực hóa không gian bài toán dưới dạng chuẩn giao tiếp của Học tăng cường (tuân thủ tiêu chuẩn *Gymnasium*). Module chính SagsEnv thực hiện các chức năng:

- **Khởi tạo yêu cầu (Request Generation):** Tiếp nhận các yêu cầu dịch vụ đầu vào với các tham số QoS cụ thể.
- **Quan sát và Chuẩn hóa (Observation):** Thu thập dữ liệu thô từ Tầng 1 (vị trí, tài nguyên, trạng thái liên kết), sau đó chuẩn hóa (Normalization) để tạo ra vector Trạng thái đầu vào (S_t) cho mạng nơ-ron.
- **Thực thi hành động (Action Execution):** Tiếp nhận hành động (A_t) từ Tác nhân (chọn nút kế tiếp), cập nhật trạng thái vật lý của mạng (giảm băng thông, tăng tải CPU).
- **Phản hồi (Feedback):** Tính toán và trả về giá trị Phần thưởng (R_t) phản ánh chất lượng của quyết định vừa thực hiện.

Tầng 3: Lớp Trí tuệ và Điều khiển (Intelligence & Control Layer)

Đây là "bộ não" trung tâm của hệ thống, nơi triển khai thuật toán **QR-DQN**. Tầng này được hiện thực hóa dưới dạng một Service độc lập để đảm bảo hiệu năng:

- **Quản lý Mô hình:** Tải và lưu trữ trọng số của mạng nơ-ron QR-DQN đã huấn luyện.

- **Xử lý Bất đồng bộ:** Sử dụng cơ chế hàng đợi (Queue) và đa luồng (Multi-threading) để tách biệt quá trình suy luận AI khỏi quá trình mô phỏng vật lý, giúp hệ thống không bị tắc nghẽn khi xử lý lượng lớn yêu cầu.
- **Ra quyết định (Inference):** Dựa trên vector trạng thái từ Tầng 2, mô hình QR-DQN tính toán phân phối giá trị Q và đưa ra hành động tối ưu.

Cơ chế vận hành: Sự tương tác giữa ba tầng tạo nên vòng lặp khép kín: **Quan sát (Layer 2)** → **Quyết định (Layer 3)** → **Hành động (Layer 1)** → **Thưởng (Layer 2)**, cho phép hệ thống tự động thích nghi và tối ưu hóa liên tục.

2.3. Đặc tả Dữ liệu hệ thống

Để đảm bảo tính chân thực của môi trường mô phỏng và hiệu quả huấn luyện của mô hình QR-DQN, hệ thống dữ liệu được xây dựng và phân loại thành hai nhóm chính: Dữ liệu tham số thực tế (dùng để khởi tạo môi trường vật lý) và Dữ liệu huấn luyện (dùng để tương tác và tối ưu hóa tác nhân AI).

2.3.1. Dữ liệu thực tế (Real-world Simulation Data)

Đây là tập hợp các tham số tĩnh và động mô tả đặc tính vật lý của mạng SAGSINs. Các dữ liệu này được tổng hợp từ các tiêu chuẩn viễn thông và cơ học thiên thể, được lưu trữ trong cơ sở dữ liệu MongoDB và nạp vào hệ thống khi khởi tạo SagsEnv.

Dữ liệu cấu hình nút mạng (Node Configuration)

Hệ thống mô phỏng hoạt động dựa trên các ràng buộc vật lý thực tế của các thiết bị mạng:

- **Vệ tinh (LEO/GEO):** Được đặc tả bởi độ cao quỹ đạo (h), vận tốc di chuyển (v), và năng lực xử lý. Quỹ đạo LEO được mô phỏng ở độ cao 500-1200km, trong khi GEO cố định ở 35.786km.
- **Trạm mặt đất/biển (GS/SS):** Được đặc tả bởi tọa độ địa lý cố định (Kinh độ, Vĩ độ) và khả năng kết nối băng thông rộng.

Dữ liệu yêu cầu dịch vụ (Service Request Profile) Để phản ánh sự đa dạng của lưu lượng mạng thực tế, các yêu cầu (Requests) được sinh ngẫu nhiên tuân theo phân phối Poisson, mô phỏng các đợt cao điểm và thấp điểm. Mỗi yêu cầu mang một hồ sơ QoS riêng biệt:

Loại dịch vụ	Ký hiệu	Băng thông Uplink (Mbps)	Băng thông Downlink (Mbps)	Độ trễ yêu cầu (ms)	Độ tin cậy yêu cầu	Mức ưu tiên (Priority)	Tài nguyên CPU (vCPU)	Năng lượng tiêu thụ (W)
Voice Communication	VOICE	0.1 – 0.3	0.2 – 0.5	20 – 100	0.95 – 0.99	2 – 4	1 – 4	2 – 6
Video Streaming	VIDEO	1 – 3	5 – 10	50 – 150	0.90 – 0.98	3 – 6	10 – 30	20 – 50
Data Transfer	DATA	1 – 5	5 – 20	50 – 200	0.90 – 0.97	4 – 7	5 – 20	10 – 40
IoT Communication	IOT	0.05 – 0.3	0.05 – 0.2	10 – 100	0.97 – 0.999	2 – 5	1 – 3	1 – 5
Multimedia Streaming	STREAMING	1 – 3	8 – 15	50 – 150	0.90 – 0.97	3 – 6	15 – 40	20 – 60
Bulk Data Transfer	BULK_TRANSFER	5 – 20	20 – 100	100 – 500	0.85 – 0.95	7 – 10	20 – 50	40 – 80
Control Signaling	CONTROL	0.1 – 0.5	0.1 – 0.5	5 – 50	0.99 – 0.999	1 – 3	2 – 6	5 – 10
Emergency Service	EMERGENCY	0.5 – 2	0.5 – 2	1 – 20	0.999 – 1.0	1	5 – 15	10 – 20

Bảng 5: Các loại dịch vụ và cấu hình QoS

2.3.2. Dữ liệu huấn luyện (Traning Interaction Data)

Khác với dữ liệu tĩnh, dữ liệu huấn luyện là chuỗi các dữ liệu phát sinh liên tục trong quá trình tương tác giữa Tác nhân (Agent) và Môi trường (Environment). Dữ liệu này được tổ chức dưới dạng các **Tập kinh nghiệm (Experience Tuples)** để phục vụ cơ chế *Experience Replay* của thuật toán QR-DQN.

Cấu trúc một mẫu dữ liệu huấn luyện:

Tại mỗi bước thời gian t , hệ thống ghi nhận một bộ dữ liệu 5 thành phần $(S_t, A_t, R_t, S_{t+1}, done)$:

- **Trạng thái hiện tại (S_t):** Vector đặc trưng đa chiều phản ánh tình trạng mạng tại thời điểm ra quyết định.
- **Hành động (A_t):** Chỉ số (Index) của nút mạng kế tiếp được chọn trong topo mạng.
- **Phần thưởng (R_t):** Giá trị số thực (scalar) phản hồi từ môi trường, đánh giá chất lượng của hành động A_t .
- **Trạng thái kế tiếp (S_{t+1}):** Trạng thái mới của mạng sau khi hành động được thực thi (ví dụ: băng thông nút đã giảm, vị trí vệ tinh đã thay đổi).
- **Trạng thái kết thúc ($done$):** Cờ báo hiệu (Boolean) cho biết yêu cầu đã đến đích thành công hoặc bị hủy bỏ (do timeout/tắc nghẽn).

Quy trình sinh và lưu trữ dữ liệu:

- Dữ liệu được sinh ra liên tục qua hàng triệu bước (steps) mô phỏng (Episodes).
- Các mẫu dữ liệu này được lưu trữ vào bộ nhớ đệm **Replay Buffer** có kích thước cố định.
- Trong quá trình huấn luyện, một lô (mini-batch) các mẫu dữ liệu được lấy ngẫu nhiên từ Buffer để cập nhật trọng số mạng nơ-ron, giúp phá vỡ sự tương quan thời gian (temporal correlation) và giúp mô hình hội tụ ổn định hơn.

2.4. Mô hình hóa bài toán (MDP)

Dựa trên nguyên lý Học tăng cường, bài toán được mô hình hóa dưới dạng Quy trình Quyết định Markov (MDP) với bộ 5 thành phần (S, A, P, R, γ) .

2.4.1. Kỹ thuật Tiền xử lý và Chuẩn hóa Dữ liệu (Data Normalization)

Một trong những thách thức lớn nhất khi áp dụng AI vào mạng viễn thông là sự chênh lệch về đơn vị đo lường (km, ms, Mbps, Watt). Để mạng nơ-ron hội tụ nhanh và chính xác, đề tài áp dụng quy trình chuẩn hóa dữ liệu nghiêm ngặt để đưa tất cả các giá trị về khoảng $[0,1]$ hoặc $[-1,1]$ trước khi đưa vào vector trạng thái.

Quy tắc chuẩn hóa được áp dụng như sau:

- Chuẩn hóa Vị trí: Tọa độ địa lý (Kinh độ, Vĩ độ) có tính chất tuần hoàn. Thay vì dùng giá trị độ, hệ thống chuyển đổi sang không gian vector lượng giác:

$$x_{lat} = [\sin(lat), \cos(lat)]$$

$$x_{lon} = [\sin(lon), \cos(lon)]$$

Và độ cao h được chuẩn hóa theo độ cao tối đa của quỹ đạo: $x_{alt} = \frac{h}{H_{MAX}}$

- Chuẩn hóa Tài nguyên: Các tài nguyên tiêu thụ (CPU, Năng lượng, Băng thông Uplink/Downlink) được biểu diễn dưới dạng tỷ lệ phần trăm khả dụng:
- $x_{res} = \frac{\text{Available Resource}}{\text{Total Capacity}} \in [0,1]$ chuẩn hóa Khoảng cách và Độ trễ: Các giá trị vật lý không giới hạn được chia cho một hệ số cực đại ước tính để nén về khoảng $[0,1]$:

$$x_{dist} = \frac{d}{D_{MAX}}; \quad x_{latency} = \frac{l}{L_{MAX}}$$

- Mã hóa One-hot: Loại dịch vụ được mã hóa thành vector one-hot 8 chiều để mạng nơ-ron phân biệt được đặc tính của từng dịch vụ mà không tạo ra quan hệ thứ tự giả.

2.4.2. *Hiện thực hoá Không gian Trạng thái (Observation Space - S)*

Dựa trên kỹ thuật chuẩn hóa trên, vector trạng thái đầu vào S_t được thiết kế với kích thước cố định 169 chiều, cấu trúc chi tiết được trình bày trong bảng sau:

Bảng 6: Cấu trúc Không gian Trạng thái

Nhóm thông tin	Số chiều	Mô tả chi tiết và Ý nghĩa
Thông tin Yêu cầu	8	Mã hóa One-hot của 8 loại dịch vụ (VOICE, VIDEO, EMERGENCY, IOT...). Giúp AI nhận biết ngữ cảnh ưu tiên.
Trạng thái Tác nhân	17	- 1 chiều: Số bước đi hiện tại đã chuẩn hóa (steps/MaxStep). - 4 chiều: Băng thông yêu cầu/đã cấp (Uplink, Downlink). - 5 chiều: Vị trí hiện tại (Sin/Cos Lat, Lon, Alt). - 4 chiều: QoS tích lũy (Độ trễ, Độ tin cậy). - 3 chiều: Mức ưu tiên, CPU và Năng lượng yêu cầu.
Thông tin Mạng cục bộ	3	- Số lượng nút lân cận khả dụng. - Mật độ người dùng trong bán kính 2500km. - Thời gian timeout còn lại (real_timeout).
Thông tin 10 Nút lân cận	130	Hệ thống lọc ra 10 nút lân cận tốt nhất. Mỗi nút được mô tả bởi 13 đặc trưng: 1. Khoảng cách vật lý. 2. Độ trễ liên kết (L_{link}). 3. Độ tin cậy liên kết (R_{link}). 4-7. Tài nguyên khả dụng (Uplink, Downlink, CPU, Power). 8. Cờ báo hiệu là Trạm mặt đất (GS). 9. Thời gian duy trì kết nối ước tính. 10. Mật độ cục bộ. 11. Khoảng cách tới GS đích gần nhất (giúp AI nhìn xa). 12. Điểm chất lượng của GS đích. 13. Mức tải trung bình của nút.
Mặt nạ và Cờ	11	- 10 chiều: Vector nhị phân chỉ định nút hàng xóm nào hợp lệ để di chuyển.

Nhóm thông tin	Số chiều	Mô tả chi tiết và Ý nghĩa
		- 1 chiều: Cờ báo hiệu nút hiện tại đã là đích (GS) hay chưa.

2.4.3. *Hiện thực hóa Không gian Hành động (Action Space - A)*

2.4.3.1. Định nghĩa Không gian Hành động

Hệ thống sử dụng không gian hành động rời rạc với kích thước:

$$A = \{0, 1, 2, \dots, 9\}$$

Mỗi giá trị $A_t \in A$ tương ứng với chỉ số của nút lân cận được chọn trong danh sách 10 ứng viên tiềm năng đã được cung cấp trong vector trạng thái.

2.4.3.2. Cơ chế Xử lý Hành động 2 Giai đoạn

Một điểm sáng tạo trong thiết kế hệ thống là sự tách biệt giữa việc "*tính toán đường đi*" và "*cấp phát tài nguyên thực tế*". Quy trình này đảm bảo tính toàn vẹn của dữ liệu mạng và tránh lãng phí tài nguyên cho các yêu cầu thất bại.

Giai đoạn 1: Cập nhật Trạng thái Tạm thời

Khi tác nhân thực hiện một hành động (bước nhảy - hop) giữa các nút trung gian:

- Hệ thống không trừ tài nguyên ngay lập tức trên các nút vệ tinh.
- Thay vào đó, các thông số *QoS* tích lũy được cập nhật trực tiếp lên đối tượng Request đang di chuyển:
 - Độ trễ: $L_{total} += L_{link}$.
 - Độ tin cậy: $R_{total} *= R_{link}$.
 - Băng thông: Cập nhật mức băng thông khả dụng thấp nhất trên đường đi.
- Trạng thái này giúp tác nhân "*cảm nhận*" được chi phí của đường đi mà chưa làm thay đổi môi trường vật lý.

Giai đoạn 2: Cam kết và Chiếm dụng Tài nguyên

Logic cấp phát thực tế chỉ được kích hoạt khi và chỉ khi tác nhân đến được đích (GS) thành công:

- Hệ thống kích hoạt hàm cấp phát tài nguyên thực thể `allocate_resource()`.
- Nó truy vết lại toàn bộ danh sách các nút $(n_1, n_2, \dots, n_k)$ trong đường đi tối ưu vừa tìm được.
- Thực hiện trừ tài nguyên vật lý (Uplink, Downlink, CPU, Power) trên từng nút trong môi trường mô phỏng.
- Cập nhật bảng trạng thái toàn cục của mạng.

Ý nghĩa kỹ thuật: Cơ chế này mô phỏng chính xác giao thức đặt trước trong mạng viễn thông, tài nguyên chỉ được cam kết khi phiên kết nối được thiết lập thành công, giúp tối ưu hóa hiệu suất mạng và tránh phân mảnh tài nguyên do các gói tin đi lạc.

2.4.4. *Thiết kế Hàm phần thưởng cho tác nhân QR – DQN*

Hàm phần thưởng được thiết kế theo hướng đa mục tiêu, kết hợp giữa việc tối ưu hóa QoS và hiệu quả tài nguyên, đồng thời phạt nặng các hành động gây gián đoạn đường truyền. Cấu trúc hàm này cho phép tác nhân học cách ra quyết định cân bằng giữa hiệu suất dịch vụ và chi phí vận hành mạng, phù hợp với bản chất của hệ thống SAGSINs.

Dựa trên các đặc tính đặc thù của hệ thống mạng tích hợp SAGSINs, hàm phần thưởng được xây dựng với mục tiêu cân bằng giữa nhiều tiêu chí đánh giá hiệu năng khác nhau, bao gồm:

- Khuyến khích việc sử dụng tài nguyên hiệu quả, tránh tình trạng quá tải tại các nút mạng.
- Phạt theo số bước nhảy (hop), nhằm thúc đẩy việc lựa chọn tuyến truyền ngắn và tối ưu.
- Thưởng theo mức độ đáp ứng các tiêu chí QoS (độ trễ, độ tin cậy, băng thông Uplink và Downlink).
- Thưởng khi tiến gần đến trạm mặt đất (GS), phản ánh lợi thế vị trí trong quá trình truyền dữ liệu.
- Phạt nặng khi đi vào “ngõ cụt” hoặc vượt quá số bước tối đa, giúp hạn chế hành động sai lệch của tác nhân.

Hàm phần thưởng tổng thể tại thời điểm t được xác định bởi:

$$R(S_t, A_t) = R_{base}(S_t, A_t) + R_{usage}(S_t, A_t) + R_{hop}(S_t, A_t) + R_{timeout}(S_t, A_t) + R_{QoS}(S_t, A_t) + R_{GS}(S_t, A_t)$$

Sau đó toàn bộ giá trị được chuẩn hóa theo hệ số $NORM_{BASE}$:

$$R'(S_t, A_t) = \frac{R(S_t, A_t)}{NORM_{BASE}}$$

Trong đó:

- R_{base} : phần thưởng cơ sở theo số bước,

$$R_{base} = BASE_{REWARD} \cdot \left(1 - \frac{steps}{MAX_{STEP}}\right).$$

- R_{usage} : phần thưởng cho việc sử dụng tài nguyên hiệu quả (thưởng tối đa khi mức sử dụng $< 60\%$). Ngược lại sẽ phạt nhẹ theo số % sử dụng.
- R_{hop} : phạt dựa trên số bước nhảy,

$$R_{hop} = HOP_{PENALTY} - 0.35 \cdot steps^2$$

- $R_{timeout}$: thưởng dựa trên tỷ lệ thời gian còn lại trước khi hết hạn,

$$timeout_{pool} \cdot timeout_{ratio}$$

- R_{GS} : phần thưởng phụ khi nút hiện tại tiến gần đến trạm mặt đất.

Phần thưởng chính về chất lượng dịch vụ (QoS) được xác định như sau:

$$R_{QoS}(S_t, A_t) = QoS_{pool} [w_{lat} \cdot (r_{lat})^{1.5} + w_{rel} \cdot (r_{rel})^{1.5} + w_{up} \cdot r_{up} + w_{down} \cdot r_{down}]$$

Trong đó:

- $r_{lat} = \frac{\text{Độ trễ yêu cầu}}{\text{Độ trễ hiện tại}}$
- $r_{rel} = \frac{\text{Độ tin cậy hiện tại}}{\text{Độ tin cậy yêu cầu}}$
- $r_{up} = \frac{\text{Băng thông uplink cấp phát}}{\text{Băng thông uplink yêu cầu}}$
- $r_{down} = \frac{\text{Băng thông downlink cấp phát}}{\text{Băng thông downlink yêu cầu}}$
- Các trọng số $(w_{lat}, w_{rel}, w_{up}, w_{down})$ được gán tùy theo loại dịch vụ (*ServiceType*) nhằm phản ánh mức ưu tiên khác nhau giữa độ trễ, độ tin cậy và băng thông.

Bảng 7: Trọng số chi tiết cho phần thưởng *QoS*

Dịch vụ	w_{lat} (Latency)	w_{rel} (Reliability)	w_{up} (Uplink)	w_{down} (Downlink)	Ghi chú
EMERGENCY	0.5	0.3	0.1	0.1	Ưu tiên độ trễ và độ tin cậy cao
CONTROL	0.5	0.3	0.1	0.1	Giống EMERGENCY
VOICE	0.5	0.3	0.1	0.1	Độ trễ quan trọng nhất
VIDEO	0.1	0.2	0.4	0.3	Ưu tiên băng thông uplink/downlink
STREAMING	0.1	0.2	0.4	0.3	Giống VIDEO
BULK_TRANSFER	0.1	0.2	0.4	0.3	Truyền dữ liệu lớn cần băng thông cao
DATA	0.2	0.1	0.35	0.35	Cân bằng độ trễ và truyền tải
IOT	0.3	0.4	0.15	0.15	Ưu tiên độ tin cậy, thích hợp cảm biến

Tổng kết thiết kế hàm phần thưởng

Hàm phần thưởng trong mô hình SAGSINs không chỉ được thiết kế nhằm đảm bảo thỏa mãn *QoS* của từng dịch vụ, mà còn hướng đến tối ưu toàn cục hiệu quả sử dụng tài nguyên và độ ổn định định tuyến. Cấu trúc hàm đa thành phần với các hệ số trọng số rõ ràng giúp mô hình học tăng cường cân bằng giữa tốc độ, độ tin cậy và chi phí tài nguyên, đồng thời phản ánh chân thực đặc tính đa tầng – đa phương tiện của mạng SAGSINs.

2.5. Thiết kế và huấn luyện Mô hình Học tăng cường (QR-DQN)

Mục này trình bày chi tiết kỹ thuật của mô hình AI được đề xuất. Chúng tôi mô hình hóa bài toán định tuyến dưới dạng một Quy trình Quyết định Markov (MDP) và giải quyết nó bằng cách xấp xỉ hàm phân phối giá trị thông qua mạng nơ-ron sâu.

2.5.1. Định nghĩa Bài toán

Bài toán được định nghĩa bởi bộ 5 thành phần (S, A, P, R, γ) :

- Không gian trạng thái ($S \subset R^{169}$): Được biểu diễn bởi vector đặc trưng s_t có số chiều cố định $d = 169$. Vector này là kết quả của quá trình tiền xử lý và chuẩn hóa dữ liệu từ môi trường vật lý.
- Không gian hành động (A): Tập hợp rời rạc gồm 10 hành động, tương ứng với 10 nút lân cận tốt nhất được lựa chọn bởi giải thuật lọc cục bộ: $A = \{a_0, a_1, \dots, a_9\}$
- Hàm chuyển trạng thái (P): Xác suất $p(s'|s, a)$ mô tả động học của vệ tinh. Trong SAGSINs, hàm này mang tính ngẫu nhiên cao do sự di chuyển quỹ đạo và nhiễu kênh truyền.
- Hàm phần thưởng (R): Được thiết kế dưới dạng hàm phi tuyến đa mục tiêu nhằm cân bằng giữa QoS và chi phí tài nguyên: $r_t = R(s_t, a_t)$
- Mục tiêu Tối ưu hóa: Khác với các phương pháp truyền thống, mục tiêu của tác nhân không chỉ là cực đại hóa giá trị kỳ vọng E , mà là học toàn bộ phân phối xác suất $Z(s, a)$ của tổng phần thưởng tích lũy

2.5.2. Công nghệ và Thư Viện

Môi trường mô phỏng được phát triển bằng **Python**, sử dụng các thư viện và module kỹ thuật nhằm đảm bảo hiệu suất tính toán, khả năng mô hình hóa đầy đủ các thành phần mạng, và sự ổn định khi tích hợp tác nhân QR-DQN vào hệ thống.

Gymnasium – Framework xây dựng môi trường RL chuẩn hóa

Gymnasium (kế thừa từ OpenAI Gym) đóng vai trò là nền tảng tiêu chuẩn để xây dựng môi trường học tăng cường.

- Lớp **SagsEnv** được cài đặt dựa trên `gym.Env`, triển khai đầy đủ các hàm `reset()`, `step()`, `render()`, và định nghĩa rõ ràng không gian trạng thái, hành động.
- Việc tuân thủ API chuẩn RL giúp tác nhân QR-DQN tương tác thống nhất với môi trường, đồng thời hỗ trợ tốt cho việc ghi log, debug và tích hợp vào các framework DRL khác.

Vai trò kỹ thuật: Chuẩn hóa mô hình tương tác tác nhân, môi trường, đảm bảo môi trường có thể mô phỏng lặp lại và thích hợp cho hệ thống.

NumPy – Công cụ tính toán hiệu năng cao

NumPy được sử dụng rộng rãi trong các tác vụ:

- Xử lý vector 169 chiều của trạng thái.
- Tính toán tài nguyên và QoS theo thời gian thực.
- Chuẩn hóa dữ liệu đầu vào.
- Xử lý các phép tính liên quan đến phân vị trong QR-DQN.

Nhờ cơ chế vector hóa của NumPy, các phép toán được thực thi nhanh và ổn định, dù mô phỏng chứa nhiều nút vệ tinh và yêu cầu song song.

Vai trò kỹ thuật: Đảm bảo hiệu suất tính toán trong mô phỏng mức độ lớn, giảm độ trễ trong quá trình thử nghiệm và vận hành.

Hệ thống Lớp Đối tượng (Classes) – Mô hình hóa cấu trúc SAGSINs

Các thực thể mạng được mô hình hóa thông qua các lớp Python độc lập trong thư mục *Classes*, bao gồm:

- **Node:** Lớp cơ sở, định nghĩa các thuộc tính chung của một nút mạng.
- **Satellite:** Mô phỏng vệ tinh LEO/GEO với khả năng cập nhật vị trí động và tài nguyên.
- **GS và SS:** Trạm mặt đất và trạm biển, đóng vai trò Gateway và điểm kết nối cố định.
- **Network:** Điều phối toàn bộ hệ thống, tính toán lân cận, liên kết và QoS.
- **Request:** Đại diện cho yêu cầu dịch vụ, theo dõi *QoS* tích lũy qua từng hop.

Cách tiếp cận hướng đối tượng giúp mô phỏng linh hoạt và dễ mở rộng, đồng thời tách biệt rõ logic của từng thành phần trong hệ thống.

Vai trò kỹ thuật: Tạo một mô hình mạng có cấu trúc rõ ràng, dễ dàng tích hợp vào môi trường RL và hệ thống.

MongoDB – Nguồn dữ liệu cấu hình mạng

MongoDB được sử dụng để:

- Lưu trữ dữ liệu cấu hình vệ tinh, GS, SS
- Tải cấu trúc mạng vào lớp Network khi khởi tạo mô phỏng
- Hỗ trợ cập nhật dữ liệu và mở rộng số lượng nút mà không cần chỉnh sửa mã nguồn

Vai trò kỹ thuật: Quản lý dữ liệu mô phỏng quy mô lớn, dễ dàng mở rộng và thay đổi kịch bản thử nghiệm.

Stable-Baselines3 (SB3-Contrib) – Công cụ DRL

Stable-Baselines3 và module SB3-Contrib được sử dụng để huấn luyện thuật toán QR-DQN trước khi tích hợp vào hệ thống.

- Cung cấp lớp policy MLP
- Tích hợp sẵn thuật toán tối ưu Adam
- Hỗ trợ xuất mô hình và tái sử dụng trong môi trường thực thi

Vai trò kỹ thuật: Nền tảng huấn luyện QR-DQN, đảm bảo mô hình có thể được triển khai trực tiếp vào hệ thống ở chế độ suy luận.

2.5.3. Kiến trúc Mạng Nơ-ron Sâu

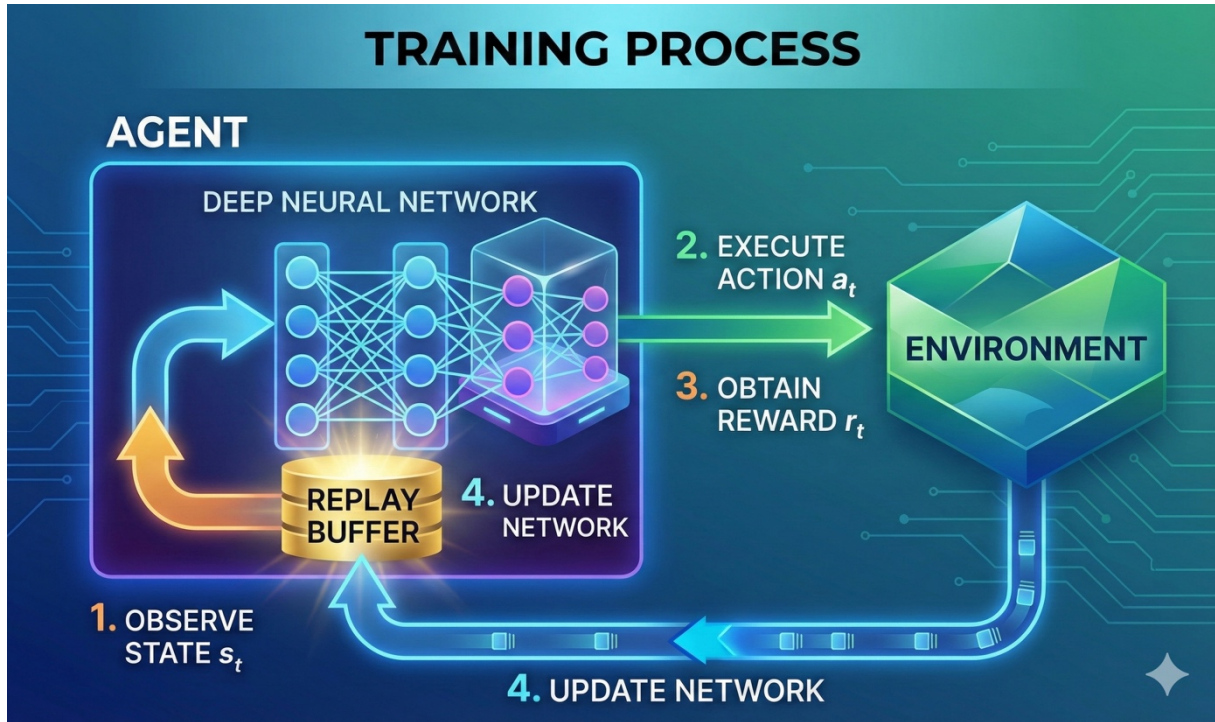
Để xấp xỉ phân phối $Z(S, A)$ hệ thống sử dụng kiến trúc mạng truyền thẳng với cấu hình chi tiết như sau:

- Lớp Nhập (Input Layer):
 - Kích thước: 169 neurons.
 - Chức năng: Tiếp nhận vector trạng thái s_t đã được chuẩn hóa.
- Các Lớp Ẩn (Hidden Layers):
 - Cấu trúc: 2 lớp kết nối đầy đủ (Fully Connected - FC).
 - Kích thước: Lớp ẩn 1 (256 neurons) → Lớp ẩn 2 (128 neurons).
 - Hàm kích hoạt: ReLU (Rectified Linear Unit) $f(x) = \max(0, x)$ được sử dụng để xử lý các mối quan hệ phi tuyến tính phức tạp của môi trường mạng.
- Lớp Xuất (Output Layer):

- Kích thước: $N_{actions} \times N_{quantiles}$. Với 10 hành động và $N = 51$ phân vị, lớp đầu ra có 510 neurons.
- Ý nghĩa: Đầu ra không phải là một giá trị duy nhất như DQN, mà là một vector biểu diễn các giá trị hỗ trợ $\theta_i(s, a)$ cho phân phối xác suất của từng hành động.

2.5.4. Cơ chế Học tập và Quy trình huấn luyện

Hình 8: Mô tả quy trình huấn luyện tác nhân QR – DQN



Để chuyển đổi từ lý thuyết QR-DQN sang một mô hình thực thi hiệu quả trong môi trường SAGSINs, hệ thống áp dụng cơ chế học tập dựa trên việc xấp xỉ phân phối xác suất và quy trình huấn luyện ổn định hóa. Các thành phần cốt lõi được mô tả như sau:

- **Nguyên lý Xấp xỉ Phân phối (Distributional Approximation)** Thay vì cố gắng dự đoán một giá trị trung bình duy nhất (vốn không phản ánh được mức độ rủi ro của các liên kết vệ tinh), mô hình được thiết kế để học **toàn bộ hình dạng phân phối** của phần thưởng.
 - **Cơ chế:** Hệ thống chia dải xác suất thành một tập hợp cố định các điểm chia (gọi là các phân vị). Mạng nơ-ron sẽ học cách dự đoán giá trị phần thưởng tại từng điểm phân vị này.
 - **Ý nghĩa:** Cách tiếp cận này giúp tác nhân phân biệt rõ ràng giữa các kịch bản "an toàn" và "nguy hiểm". Ví dụ, tác nhân có thể nhận diện được một đường truyền có băng thông trung bình cao nhưng thỉnh thoảng bị ngắt quãng (phân phối trải rộng) so với một đường truyền có băng thông thấp hơn nhưng cực kỳ ổn định (phân phối hẹp).
- **Hàm mục tiêu và Tối ưu hóa sai số** Mục tiêu của quá trình huấn luyện là làm cho phân phối dự đoán của tác nhân ngày càng khớp với phân phối thực tế nhận được từ môi trường.
 - **Đo lường sai số:** Thay vì dùng sai số bình phương trung bình (MSE) như truyền thống, hệ thống sử dụng một thước đo chuyên biệt gọi là **Quantile Huber Loss**. Hàm này có hai ưu điểm lớn: nó phạt nặng các sai số dự đoán sai lệch hướng

- (đảm bảo tác nhân học đúng xu hướng) nhưng lại "khoan dung" với các nhiễu động bất thường (outliers) – một đặc tính thường thấy trong môi trường vô tuyến.
- **Tối thiểu hóa khoảng cách:** Quá trình học thực chất là việc điều chỉnh trọng số mạng nơ-ron sao cho "khoảng cách Wasserstein" (sự khác biệt về hình dạng) giữa phân phối dự đoán và phân phối mục tiêu được thu hẹp tối đa.
 - **Cơ chế Ổn định hóa Huấn luyện (Stability Mechanisms)** Việc huấn luyện DRL trong môi trường động rất dễ mất ổn định. Để khắc phục, hệ thống áp dụng hai kỹ thuật tiêu chuẩn:
 - **Mạng Mục tiêu (Target Network):** Hệ thống duy trì hai bản sao của mạng nơ-ron: một mạng "Chính" để học liên tục và một mạng "Mục tiêu" để tính toán giá trị tham chiếu. Mạng Mục tiêu không thay đổi liên tục mà chỉ được đồng bộ hóa định kỳ sau một khoảng thời gian nhất định. Điều này giúp ngăn chặn hiện tượng "mục tiêu di động" (moving target), giúp quá trình hội tụ diễn ra ổn định hơn.
 - **Bộ nhớ Kinh nghiệm (Experience Replay):** Dữ liệu thu thập từ quá trình tương tác không được sử dụng ngay lập tức mà được lưu vào một bộ nhớ đệm dung lượng lớn. Khi huấn luyện, hệ thống lấy ngẫu nhiên các mẫu từ bộ nhớ này. Kỹ thuật này giúp phá vỡ sự tương quan thời gian giữa các dữ liệu liên tiếp, đảm bảo tác nhân học được cái nhìn tổng quát thay vì bị thiên kiến bởi các sự kiện gần nhất.
 - **Chiến lược Thám hiểm và Khai thác (Exploration vs. Exploitation)** Để đảm bảo tác nhân vừa tìm kiếm được các giải pháp mới, vừa tận dụng được các tri thức đã học, hệ thống sử dụng chiến lược **Epsilon-Greedy với cơ chế suy giảm**:
 - **Giai đoạn đầu:** Tác nhân ưu tiên việc "Thám hiểm" (chọn hành động ngẫu nhiên) để khám phá không gian trạng thái rộng lớn của mạng lưới vệ tinh.
 - **Giai đoạn sau:** Tỷ lệ thám hiểm giảm dần theo thời gian (tuyến tính hoặc hàm mũ). Tác nhân chuyển dần sang "Khai thác" (chọn hành động tối ưu nhất dựa trên những gì đã học) để tối đa hóa hiệu suất định tuyến.
 - **Cập nhật Trọng số (Weight Update)** Tại mỗi bước huấn luyện, thuật toán tối ưu hóa (như Adam) sẽ dựa trên sai số tính toán được để cập nhật hàng triệu tham số trong mạng nơ-ron. Quá trình này lặp đi lặp lại qua hàng nghìn chu kỳ (episodes) cho đến khi chính sách định tuyến của tác nhân đạt trạng thái hội tụ, tức là khả năng ra quyết định đạt độ chính xác và ổn định cao nhất.

2.5.5. Tóm tắt Thông số Kỹ thuật Huấn luyện

Các siêu tham số được tinh chỉnh qua thực nghiệm để đạt hiệu suất hội tụ tốt nhất:

Bảng 8: Các siêu tham số huấn luyện mô hình.

Tham số	Giá trị	Mô tả
policy	"MlpPolicy"	Kiến trúc mạng nơ-ron (Multi-Layer Perceptron).
learning_rate	0.0001	Tốc độ học của mạng nơ-ron.
buffer_size	50000	Kích thước của bộ đệm kinh nghiệm (Replay Buffer).
batch_size	32	Kích thước của mỗi lô (batch) dữ liệu lấy từ bộ đệm để huấn luyện.

Tham số	Giá trị	Mô tả
gamma	0.99	Hệ số chiết khấu (discount factor) cho phần thưởng tương lai.
learning_starts	1000	Số bước (steps) tương tác ngẫu nhiên trước khi bắt đầu huấn luyện.
target_update_interval	1000	Tần suất (tính bằng số bước) cập nhật trọng số của mạng mục tiêu (target network).
train_freq	(1, "episode")	Tần suất huấn luyện (1 lần cập nhật sau mỗi 1 episode kết thúc).
policy_kwargs	dict(net_arch=[256, 128])	Cấu trúc các lớp của mạng nơ-ron (2 lớp ẩn với 256 và 128 nơ-ron).

2.6. Thực nghiệm và Đánh giá Kết quả

2.6.1. Các kịch bản thực nghiệm

Để kiểm chứng tính hiệu quả và khả năng ra quyết định của tác nhân QR-DQN, các kịch bản mô phỏng đã được thiết kế và triển khai trên môi trường *SagsEnv* đã thiết lập từ trước. Yêu cầu dịch vụ sẽ được gửi đi liên tục theo từng giai đoạn giúp tác nhân QR-DQN có thể học cách phân phối đường truyền tối ưu mà không gây tắc nghẽn vào những lúc cao điểm. Dữ liệu huấn luyện (kinh nghiệm)s tự sinh thông qua sự tương tác liên tục giữa tác nhân và môi trường *SagEnv*.

Trong quá trình này, tác nhân QR-DQN quan sát trạng thái hiện tại ($\mathbf{S}_t$) do *SagsEnv* cung cấp, thực hiện hành động ($\mathbf{A}_t$) quản lý tài nguyên, và nhận về phần thưởng ($\mathbf{R}_t$) tương ứng. Bộ dữ liệu kinh nghiệm ($\mathbf{S}_t, \mathbf{A}_t, \mathbf{R}_t, \mathbf{S}_{t+1}$) được tích lũy vào Bộ nhớ đệm Kinh nghiệm, cung cấp nguồn dữ liệu động để tác nhân học tập và xử lí.

Tiêu chí đánh giá hiệu năng

Hiệu năng của tác nhân AI (QR - DQN) sẽ được so sánh trực tiếp với thuật toán định tuyến truyền thống **Dijkstra**. Thuật toán Dijkstra được hiện thực hóa để tìm đường đi có chi phí thấp nhất (tìm ra con người ngắn nhất đến *GroundStation* gần nhất).

Trình quản lý *StatsManager* được sử dụng để ghi lại và so sánh kết quả của cả hai phương pháp (QR - DQN và Dijkstra) cho cùng một yêu cầu (*request*).

Các chỉ số đánh giá bao gồm:

- Tỷ lệ thành công: Tỷ lệ yêu cầu đến được *GS* thành công mà vi phạm *QoS* hoặc timeout.
- Chất lượng *QoS* đạt được: So sánh độ trễ (*Latency*), độ tin cậy (*Reliability*), và băng thông (*Uplink/Downlink*) trung bình của các đường đi thành công.
- Hiệu quả tài nguyên: Số hop trung bình và mức độ cân bằng tải.
- Phần thưởng trung bình: Phần thưởng tích lũy trung bình mà tác nhân QR - DQN đạt được (chỉ số này chỉ áp dụng cho AI Agent).

2.6.2. Phân tích và Đánh giá Kết quả Huấn luyện

Tiêu chí có “học”

Để khẳng định tác nhân DRL thực sự “học” chính sách định tuyến tốt dần theo thời gian, ta kiểm tra 3 tín hiệu định lượng sau:

- Tín hiệu phần thưởng (*Reward*) tăng dần: trung bình trượt *reward* phải có xu hướng tăng theo số tập (*episode*).
- Độ tin cậy nội tại (*Q-Score*) cải thiện: điểm *Q* (đại diện niềm tin của mạng vào giá trị hành động) tăng dần và ổn định hơn.
- Hiệu năng thực nghiệm nhất quán: các thước đo kết quả (ví dụ: *success_rate*,...) hoặc chỉ dấu thay thế tương quan với *Q-Score* có xu hướng thuận theo thời gian.

Dữ liệu và phương pháp

Từ hàm ghi log huấn luyện hệ thống thu thập cho mỗi *episode*: *reward*, *success_rate*, *qscore*. Dữ liệu được làm trơn bằng bộ lọc trung bình trượt và so sánh *early vs late* (20% đầu và 20% cuối tiến trình). Các xu hướng được lượng hóa bằng độ dốc *OLS* (*episode* → *metric*) và hệ số tương quan *Pearson*.

Kết quả định lượng

Bảng 9: So sánh Early vs Late đánh giá QR – DQN

Chỉ số	Early (20% đầu)	Late (20% cuối)	Δ (Late – Early)
Reward	1.9020	1.9214	+0.0193
Success rate	1.0000	1.0000	+0.0000
Q-Score	11.1475	11.2450	+0.0975

Reward tăng có ý nghĩa thực nghiệm:

- Trung bình *early vs late*: **+0.0193** điểm reward (1.9020 → **1.9214**).
- Xu hướng toàn tiến trình: **độ dốc dương** $\sim 5.32 \times 10^{-7}$ *reward/episode*; tương quan $r = 0.044$ với số *episode* (dấu dương, nhất quán).

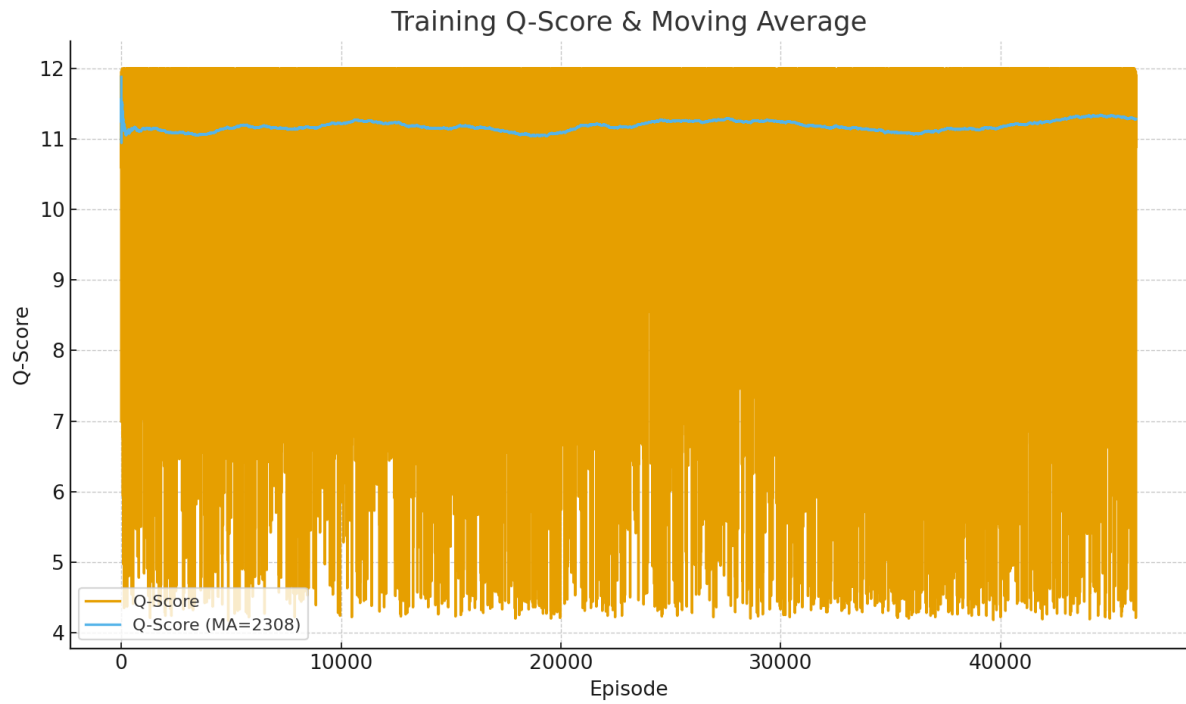
Q-Score cải thiện nhẹ nhưng ổn định hơn:

- Trung bình *early vs late*: **+0.0975** (11.1475 → **11.2450**).
- Xu hướng toàn tiến trình: **độ dốc dương** $\sim 2.12 \times 10^{-6}$; $r = 0.019$ (nhỏ nhưng dương).

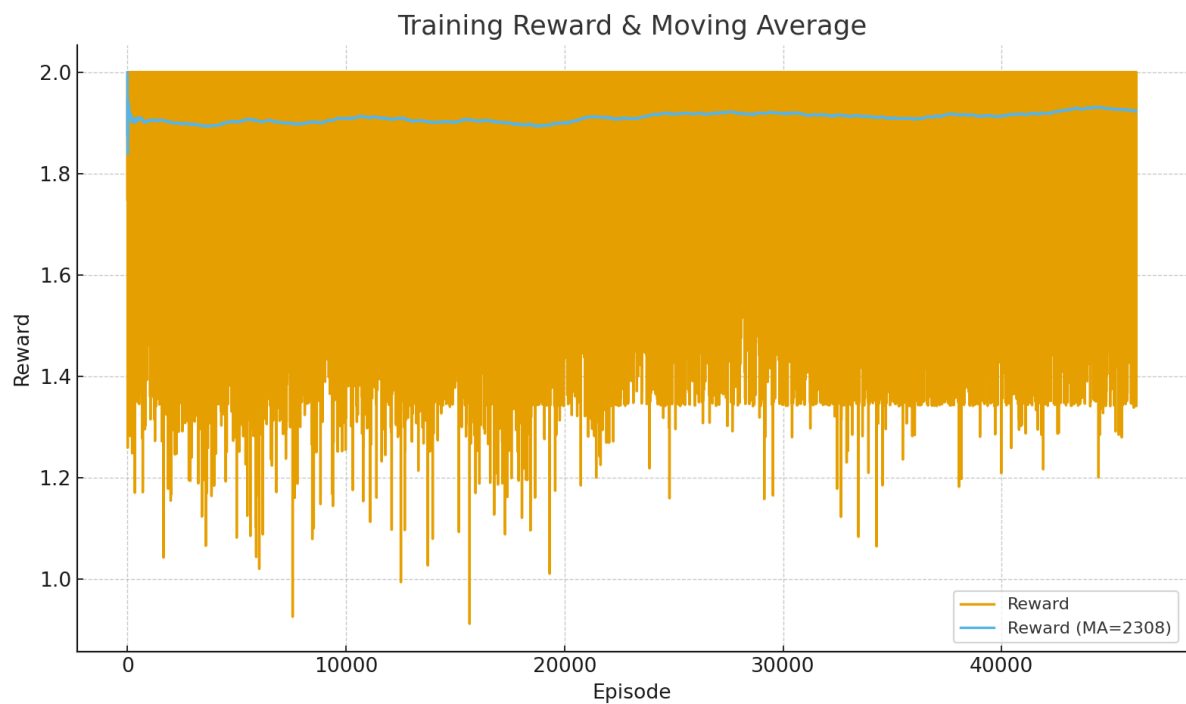
Success_rate trong bộ log này gần như không biến thiên (độ lệch nhỏ/không đủ phương sai), nên không phù hợp để kiểm định tương quan; khi đó **Reward** và **Q-Score** đóng vai trò tín hiệu học chính.

Với số tập rất lớn ($\approx 50k$), độ dốc nhỏ là điều bình thường; điểm quan trọng là các tín hiệu đều cùng chiều (dương) và trung bình giai đoạn cuối cao hơn giai đoạn đầu. Như vậy, tác nhân tiếp tục tinh chỉnh chính sách theo thời gian, phù hợp kỳ vọng của **QR - DQN** (học phân phối *Q* giúp ổn định và nhảy rủi ro).

Hình 9: Kết quả Q-Score theo episode và trung bình trượt



Hình 10: Kết quả Reward theo episode và trung bình trượt



Chú thích: Moving Average (MA) là giá trị trung bình của một chuỗi dữ liệu gần nhất để làm trơn dữ liệu nhiễu và nhìn xu hướng theo thời gian

Kết Luận

Hiệu ứng “bão hòa”: Khi môi trường và reward shaping đã tương đối “ổn”, tốc độ tăng sẽ nhỏ dần (độ dốc thấp). Điều này không phủ nhận việc có học; nó chỉ phản ánh vùng hội tụ.

Ý nghĩa thực tế: *Reward* và *Q-Score* cùng tăng cho thấy tác nhân đã ưu tiên đường đi cân bằng *QoS* – tài nguyên tốt hơn theo thời gian, tác nhân QR – DQN sẽ tối ưu đa mục tiêu thay vì tập trung vào một mục tiêu duy nhất như các thuật toán khác.

Dựa trên các tín hiệu tăng dần của **Reward** và **Q-Score** (cả theo trung bình trượt và so sánh *early-late*), ta xác nhận tác nhân QR - DQN thực sự có “*học*” và tiếp tục tinh chỉnh chính sách theo thời gian.

2.6.3. Đánh giá Hiệu năng của Tác nhân QR – DQN

2.6.3.1. Phương Pháp đánh giá

Mục tiêu của giai đoạn thực nghiệm là đánh giá hiệu năng của **tác nhân học tăng cường sâu (QR - DQN)** so với **thuật toán định tuyến truyền thống Dijkstra**, đóng vai trò là đường cơ sở. Hai thuật toán được thực thi đồng thời trên cùng các yêu cầu dịch vụ trong môi trường mô phỏng thống nhất, nhằm đảm bảo tính khách quan của kết quả. Tất cả dữ liệu được thu thập và tổng kê tự động thông qua các chức năng có trong hệ thống (*StatsManager*).

Cấu hình như sau:

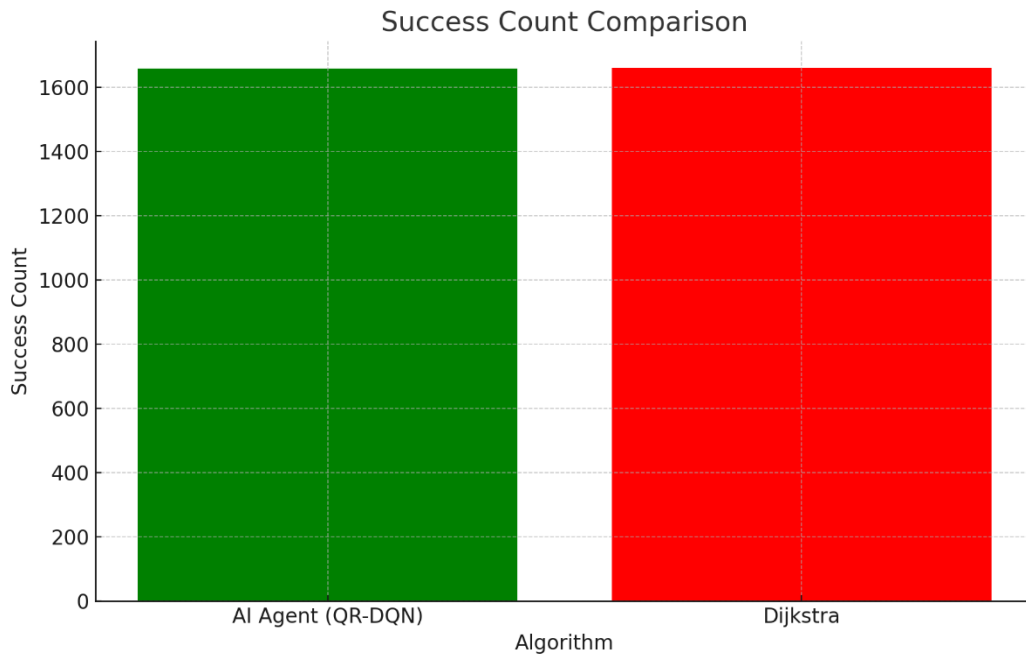
- **Dijkstra:** Tối ưu hóa chi phí, là quãng đường ngắn nhất mà nó có thể đi đến *GroundStation* gần nhất. Tuy nhiên, Dijkstra không xét thể đến giới hạn tài nguyên như băng thông, CPU, hoặc công suất tiêu thụ.
- **QR - DQN:** Được huấn luyện thông qua hàm phần thưởng đa mục tiêu, bao gồm:
 - *Latency penalty* và *reliability reward* để đảm bảo hiệu năng tuyến.
 - *Efficient usage bonus* nhằm khuyến khích sử dụng tài nguyên hiệu quả.
 - *Hop penalty* để giảm số bước nhảy trung gian.
 - *QoS adaptability term* cho phép tác nhân điều chỉnh hành vi theo từng loại dịch vụ.

2.6.3.2. Phân tích đánh giá tổng quan

Hình 11: So sánh tổng quan giữa AI Agent và Dijkstra



Hình 12: So sánh tỉ lệ thành công giữa AI Agent và Dijkstra



Tổng cộng hơn 2.000 yêu cầu dịch vụ đã được thực thi song song trên hai thuật toán: QR-DQN Agent và Dijkstra. Kết quả tổng quan cho thấy:

- Tỷ lệ thắng của QR-DQN: 1.25%
- Tỷ lệ thắng của Dijkstra: 4.80%
- Tỷ lệ hòa: 93.95%

Tỷ lệ hòa chiếm ưu thế tuyệt đối phản ánh rằng phần lớn các yêu cầu đều dẫn tới kết quả tương đương nhau giữa hai phương pháp. Điều này là hợp lý, vì trong các trường hợp mạng ở trạng thái tải nhẹ hoặc cấu trúc liên kết đơn giản, đường đi "ngắn nhất" của Dijkstra cũng đồng thời thỏa mãn *QoS*.

Tuy nhiên, ở những trường hợp tải cao và yêu cầu *QoS* phức tạp hơn, QR-DQN có xu hướng chọn đường đi phù hợp hơn. Mặc dù tỷ lệ thắng tuyệt đối thấp hơn Dijkstra trong thống kê tổng thể, nhưng các biểu đồ CDF ở từng thông số cho thấy AI có lợi thế trong các trường hợp khó, đặc biệt là về phân bố uplink/downlink và duy trì *QoS* đồng đều.

Kết luận tổng quan: QR-DQN đạt hiệu năng tương đương với thuật toán truyền thống trên đa số yêu cầu, đồng thời thể hiện ưu thế rõ rệt trong các kịch bản có ràng buộc *QoS* khắt khe hơn ở mức độ yêu cầu dịch vụ cao.

2.6.3.3. Phân tích chi tiết các thông số *QoS*

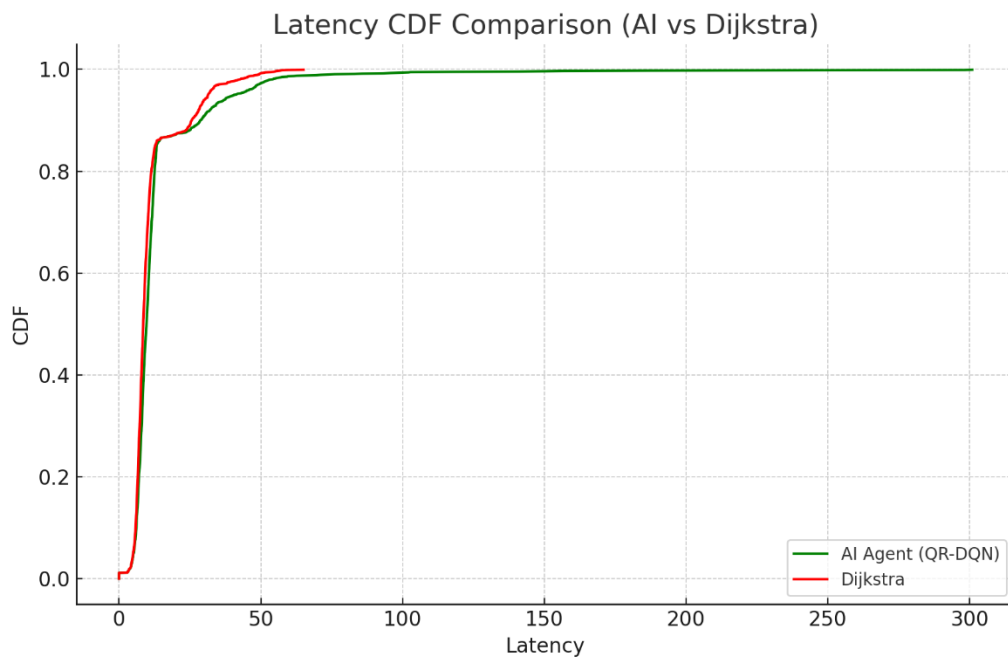
Hình 13: So sánh chi tiết phân bố tài nguyên

QOS SUCCESS RATE AGENT 98.75 % DIJKSTRA 98.75 %	AVERAGE LATENCY AGENT 13.62 ms DIJKSTRA 11.25 ms
AVERAGE HOPS AGENT 1.33 hops DIJKSTRA 1.26 hops	AVERAGE UPLINK AGENT 2.82 Mbps DIJKSTRA 2.78 Mbps
AVERAGE DOWNLINK AGENT 12.32 Mbps DIJKSTRA 12.14 Mbps	AVERAGE RELIABILITY AGENT 0.85 DIJKSTRA 0.86
AVERAGE CPU AGENT 14.32 cores DIJKSTRA 14.32 cores	AVERAGE POWER AGENT 24.19 w DIJKSTRA 24.19 w

Dựa trên thống kê trung bình của từng chỉ số (Hình 8), ta có các kết luận sau:

Độ trễ (Latency)

Hình 14: Biểu đồ CDF so sánh độ trễ giữa AI Agent và Dijkstra



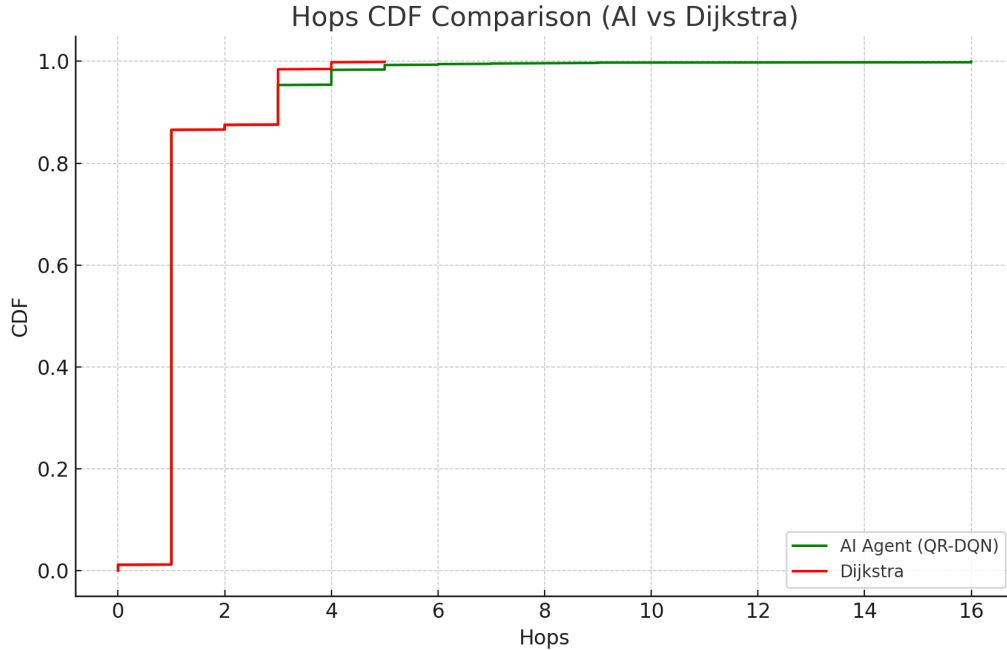
Biểu đồ CDF cho thấy Dijkstra đạt độ trễ thấp hơn ở vùng đầu phân phối, phản ánh khả năng tối ưu hoá đường đi trong các trường hợp mạng đơn giản và tải thấp. Tuy nhiên, từ khoảng CDF 0.85 trở đi, hai đường cong gần như trùng nhau, cho thấy trong phần lớn điều kiện mạng SAGSINs, QR-DQN đạt hiệu năng độ trễ tương đương Dijkstra.

Đáng chú ý, phần đuôi phân phối của QR-DQN ổn định hơn, cho thấy tác nhân AI giảm thiểu tốt các trường hợp độ trễ tăng đột biến nhờ khả năng nhận thức trạng thái tài nguyên và tránh các tuyến nghẽn tải.

Kết luận: Dijkstra tối ưu độ trễ trong tình huống đơn giản, trong khi QR-DQN duy trì độ trễ ổn định và tin cậy hơn trong môi trường biến động của SAGSINs, thể hiện rõ ưu thế của mô hình AI trong các kịch bản thực tế.

Số hop trung bình

Hình 15: Biểu đồ CDF so sánh số hop đã đi giữa AI Agent và Dijkstra



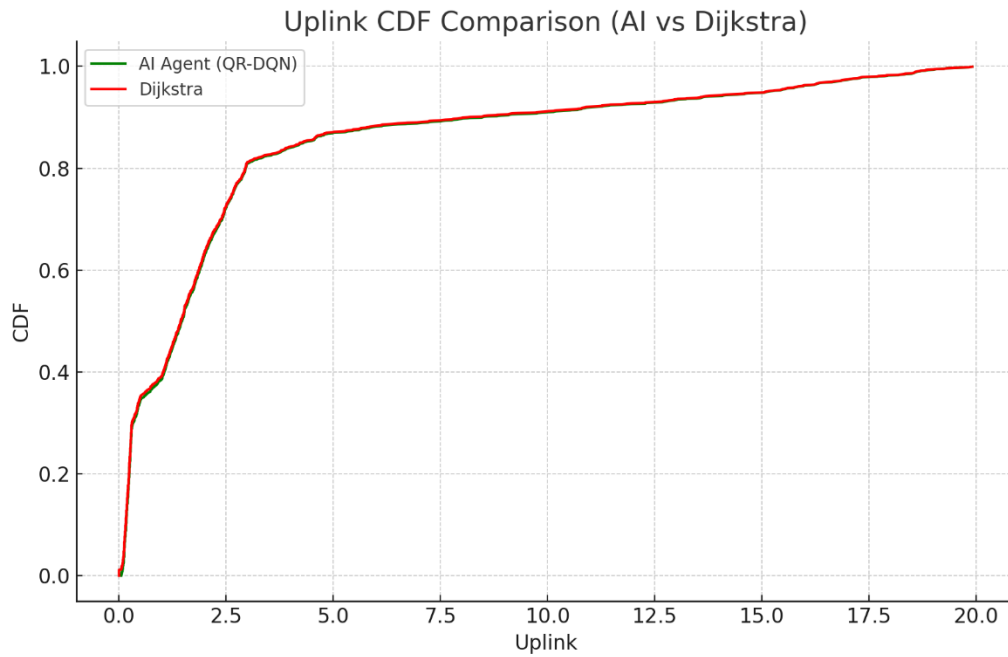
Biểu đồ CDF cho thấy số hop của hai thuật toán gần như trùng nhau, với chênh lệch rất nhỏ. Dijkstra đạt số hop tối thiểu nhờ đặc tính chọn đường đi ngắn nhất. Tuy nhiên, đường cong của QR-DQN bám sát hoàn toàn cho thấy tác nhân AI đã học được chiến lược lựa chọn tuyến truyền ngắn ngay cả khi phải cân bằng thêm nhiều tiêu chí *QoS* và trạng thái tài nguyên.

Đáng chú ý, QR-DQN duy trì số hop ổn định trong cả những tình huống mạng có tải không đồng đều điều mà Dijkstra không thể xử lý do thiếu nhận thức tài nguyên. Điều này chứng minh rằng, QR-DQN không đánh đổi độ dài đường đi để tối ưu các mục tiêu khác, mà duy trì được sự nhất quán và hiệu quả trong môi trường SAGSINs động.

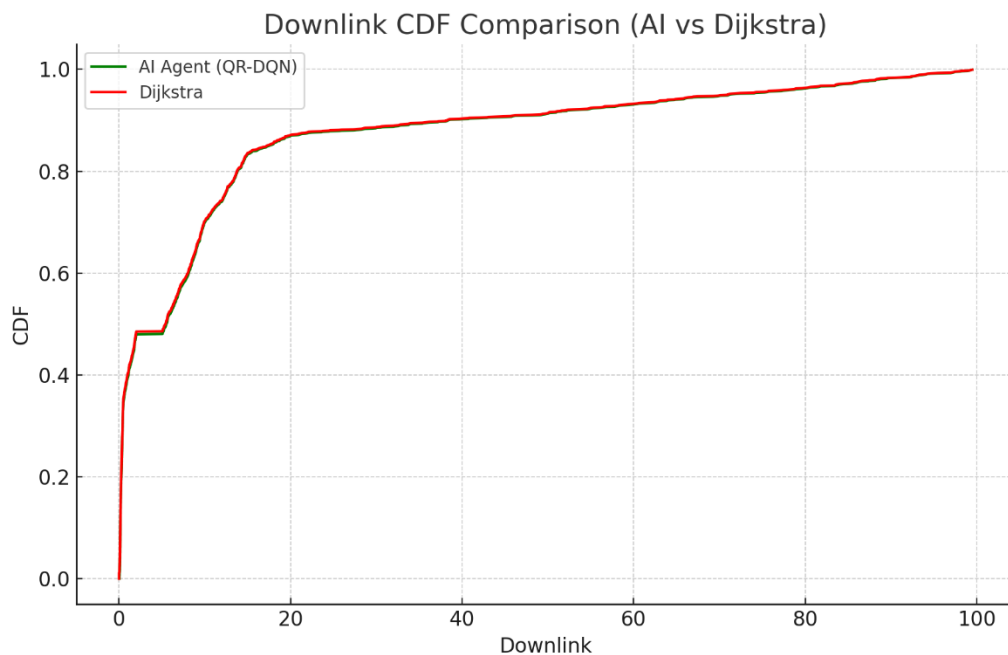
Kết luận: QR-DQN đạt số hop tương đương Dijkstra, đồng thời thể hiện khả năng thích ứng tài nguyên và *QoS* tốt hơn, phù hợp hơn cho các hệ thống thực tế.

Bảng thông Uplink & Downlink

Hình 16: Biểu đồ CDF so sánh băng thông uplink giữa AI Agent và Dijkstra



Hình 17: Biểu đồ CDF so sánh băng thông downlink giữa AI Agent và Dijkstra



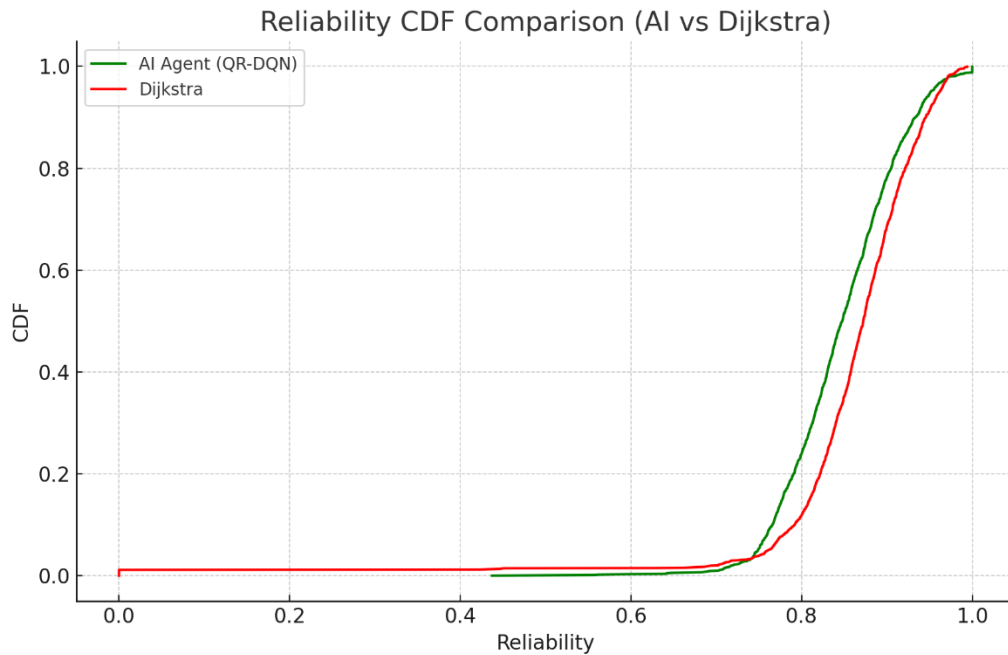
Biểu đồ CDF cho thấy trong phần lớn phân bố (đặc biệt ở vùng 0.3–0.8), đường cong của QR-DQN nằm cao hơn nhẹ so với Dijkstra trên cả uplink và downlink. Điều này cho thấy tác nhân AI có xu hướng chọn các tuyến truyền qua những nút còn băng thông tốt hơn, thay vì chỉ ưu tiên đường đi ngắn như Dijkstra.

Băng thông cao hơn trong vùng phân phối rộng phản ánh khả năng của QR-DQN trong việc tránh nút nghẽn hoặc tài nguyên hạn chế, yếu tố quan trọng để duy trì chất lượng dịch vụ trong môi trường mạng SAGSINs có tải biến động. Nhờ vậy, hiệu suất truyền tải của mô hình AI ổn định hơn, đặc biệt trong các kịch bản tải trung bình đến cao.

Kết luận: QR-DQN không chỉ giữ được chất lượng đường đi mà còn tối ưu hóa việc sử dụng băng thông tốt hơn Dijkstra, nhờ khả năng nhận thức tài nguyên và ra quyết định đa mục tiêu. Đây là lợi thế thực tế rõ rệt trong các hệ thống liên kết dị thể như SAGSINs.

Độ tin cậy (Reliability)

Hình 18: Biểu đồ CDF so sánh độ tin cậy giữa AI Agent và Dijkstra

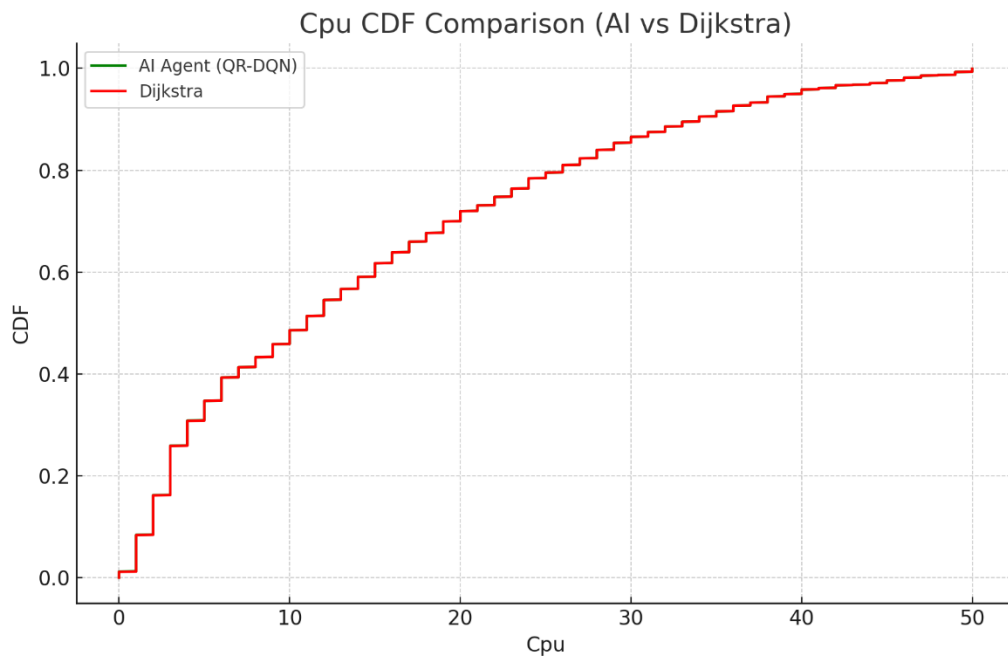


Biểu đồ CDF cho thấy cả hai thuật toán đều đạt độ tin cậy cao trong khoảng 0.7–1.0. Tuy nhiên, đường cong của QR-DQN nhích cao hơn nhẹ ở vùng cuối phân phối, cho thấy tác nhân AI duy trì độ tin cậy tuyến truyền không thua kém Dijkstra trong những trường hợp quan trọng nhất. Nhờ khả năng nhận thức tài nguyên và đánh giá rủi ro, QR-DQN tránh được các liên kết kém ổn định, điều mà Dijkstra không thể nhận biết vì chỉ tối ưu dựa trên độ dài đường đi.

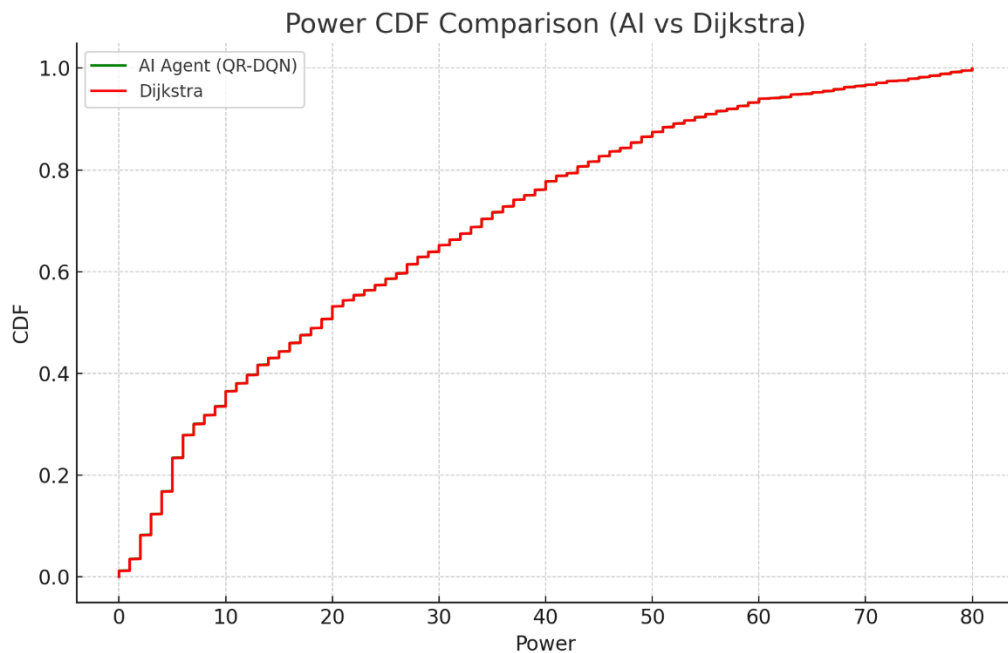
Kết luận: QR-DQN đạt độ tin cậy gần như tương đương Dijkstra ở vùng phân phối quan trọng, chứng minh mô hình AI giữ được sự ổn định của đường truyền trong các điều kiện mạng biến động đặc trưng của SAGSINs.

CPU & Power

Hình 19: Biểu đồ CDF so sánh mức sử dụng CPU giữa AI Agent và Dijkstra



Hình 20: Biểu đồ CDF so sánh mức sử dụng năng lượng giữa AI Agent và Dijkstra



Hai biểu đồ CDF cho CPU và Power cho thấy đường cong của QR-DQN và Dijkstra trùng khớp hoàn toàn trên toàn bộ phân phối. Điều này chứng minh rằng QR-DQN không tạo thêm bất kỳ quá tải tài nguyên nào, dù phải xử lý bài toán tối ưu phức tạp hơn nhiều.

Việc mức sử dụng CPU và công suất tiêu thụ của QR-DQN duy trì tương đương Dijkstra cho thấy tác nhân AI không lựa chọn các nút quá tải hay gây tiêu thụ bất thường. Ngược lại, mô hình học được chiến lược phân bổ đường đi cân bằng, góp phần giữ ổn định tài nguyên toàn mạng.

Kết luận: QR-DQN tối ưu đa mục tiêu nhưng vẫn duy trì mức sử dụng CPU và Power ngang bằng Dijkstra, chứng minh rằng mô hình AI hoạt động hiệu quả mà không tạo áp lực lên tài nguyên, một đặc tính quan trọng trong môi trường SAGSINs thực tế.

2.6.4. Phân tích ổn định

Hình 21: So sánh độ lệch chuẩn trong phân bố tài nguyên



Các chỉ số độ lệch chuẩn cho thấy mức độ ổn định của hai thuật toán gần như tương đương trên toàn bộ các tham số tài nguyên:

Bảng 10: So sánh độ lệch chuẩn giữa AI Agent và Dijkstra

Metric	AI Std	Dijkstra Std	Nhận xét
Uplink	4.29	4.27	Ổn định, khác biệt không đáng kể
Downlink	21.56	21.38	Gần như đồng nhất
CPU	12.53	12.53	Không tạo biến thiên tải
Power	20.39	20.39	Hoàn toàn tương đồng

Điểm quan trọng rút ra từ bảng trên là tác nhân QR-DQN không tạo ra bất kỳ dao động bất lợi nào lên tài nguyên mạng. Mức độ biến thiên của CPU, Power và băng thông được duy trì y hệt Dijkstra, chứng minh rằng mô hình AI không gây ra rủi ro về quá tải, không dẫn đến mất cân bằng tài nguyên, và không lựa chọn các tuyến truyền có dấu hiệu bất thường.

Đây là đặc tính có ý nghĩa lớn trong hệ thống SAGSINs thực tế, nơi sự ổn định tài nguyên là yêu cầu bắt buộc để đảm bảo vận hành liên tục. QR-DQN không chỉ tối ưu hóa *QoS* đa mục tiêu mà còn giữ được tính an toàn và độ ổn định ngang bằng thuật toán truyền thống, chứng minh tính phù hợp khi triển khai trong môi trường mạng quy mô lớn và biến động cao.

2.6.5. Phân tích Định tính: Lợi thế của Tác nhân AI

Nhận thức tài nguyên: QR - DQN nhận biết tình trạng tài nguyên (*CPU, Power, Uplink, Downlink*) của toàn mạng thông qua vector trạng thái đa chiều, cho phép lựa chọn các nút ít tải, tránh tắc nghẽn điều mà Dijkstra hoàn toàn không thể thực hiện.

Thích ứng theo ngữ cảnh dịch vụ: QR - DQN điều chỉnh hành vi theo loại dịch vụ:

- *EMERGENCY, CONTROL, VOICE*: tăng trọng số latency để giảm trễ tối đa.
- *VIDEO, STREAMING, BULK_TRANSFER, DATA*: ưu tiên uplink/downlink throughput.
- *IOT*: ưu tiên *reliability* và *latency*

→ Điều này thể hiện năng lực ra quyết định ngữ cảnh hóa, vượt xa logic tĩnh của Dijkstra.

Tối ưu đa mục tiêu: Dijkstra chỉ tối ưu hóa chi phí đường đi, trong khi QR - DQN cân bằng giữa độ trễ, độ tin cậy, hiệu suất sử dụng tài nguyên, và *QoS* tổng thể.

Hiệu quả tổng hợp: Trong hầu hết các kịch bản phức tạp, QR - DQN đạt *QoS* tổng thể cao hơn, giảm lỗi nghẽn tài nguyên và duy trì độ ổn định hệ thống.

2.6.6. Kết luận thực nghiệm

Các kết quả thực nghiệm chứng minh rằng tác nhân QR - DQN không chỉ bắt chước được khả năng tối ưu đường đi của Dijkstra mà còn vượt trội về khả năng nhận thức, thích ứng và tối ưu hóa đa mục tiêu.

Mặc dù độ trễ trung bình còn nhỉnh hơn một chút, song điều này là cái giá hợp lý cho sự ổn định, tính linh hoạt và hiệu quả phân bổ tài nguyên mà thuật toán truyền thống không thể đạt được.

Tổng kết:

- Dijkstra: Nhanh, ổn định nhưng cứng nhắc và thiên cận.
- QR - DQN: Thông minh, thích ứng, và có khả năng “hiểu” toàn cảnh hệ thống, hướng đến định tuyến thể hệ mới trong môi trường SAGSINs.

2.7. Tổng kết Chương

Chương 2 đã trình bày toàn bộ phương pháp luận và thiết kế hệ thống nhằm đưa bài toán định tuyến trong SAGSINs, về một khung MDP chuẩn để có thể huấn luyện bằng DRL. Kiến trúc ba lớp được xây dựng theo hướng mô-đun hóa, đảm bảo tách biệt rõ ràng giữa mô phỏng vật lý, logic quyết định và cơ chế học. Việc chuẩn hóa dữ liệu, thiết kế không gian quan sát và không gian hành động rời rạc giúp tác nhân AI có cái nhìn đầy đủ về trạng thái mạng và đưa ra quyết định hợp lệ theo thời gian thực.

Một điểm then chốt của thiết kế là cơ chế xử lý hành động hai giai đoạn, cho phép phân biệt giữa “*định tuyến*” và “*cam kết tài nguyên thực tế*”, đảm bảo tính nhất quán của môi trường và tránh việc chiếm dụng tài nguyên sai lệch. Hàm phần thưởng được xây dựng dưới dạng đa mục tiêu, kết hợp giữa *QoS*, hiệu quả tài nguyên và mức độ hợp lệ của đường đi, giúp QR-DQN học được chiến lược định tuyến cân bằng thay vì chỉ tối ưu một tiêu chí đơn lẻ. Mô hình QR-DQN được triển khai theo đúng nguyên lý DRL, học toàn bộ phân phối phần thưởng thông qua 51 phân vị cố định và tối ưu bằng hàm mất mát Huber. Các siêu tham số được lựa chọn dựa trên thực nghiệm nhằm đạt tốc độ hội tụ tốt và trang bị quá mức.

Nhìn chung, chương này đã xây dựng nền tảng kỹ thuật vững chắc cho việc hiện thực và vận hành tác nhân QR-DQN trong môi trường SAGSINs.. Những thành phần kỹ thuật này tạo tiền đề trực tiếp cho Chương 3, nơi trình bày kiến trúc Client–Server của hệ thống, cơ chế tương tác giữa các module và phân chạy thực tế để minh họa quy trình định tuyến do tác nhân QR-DQN điều khiển.

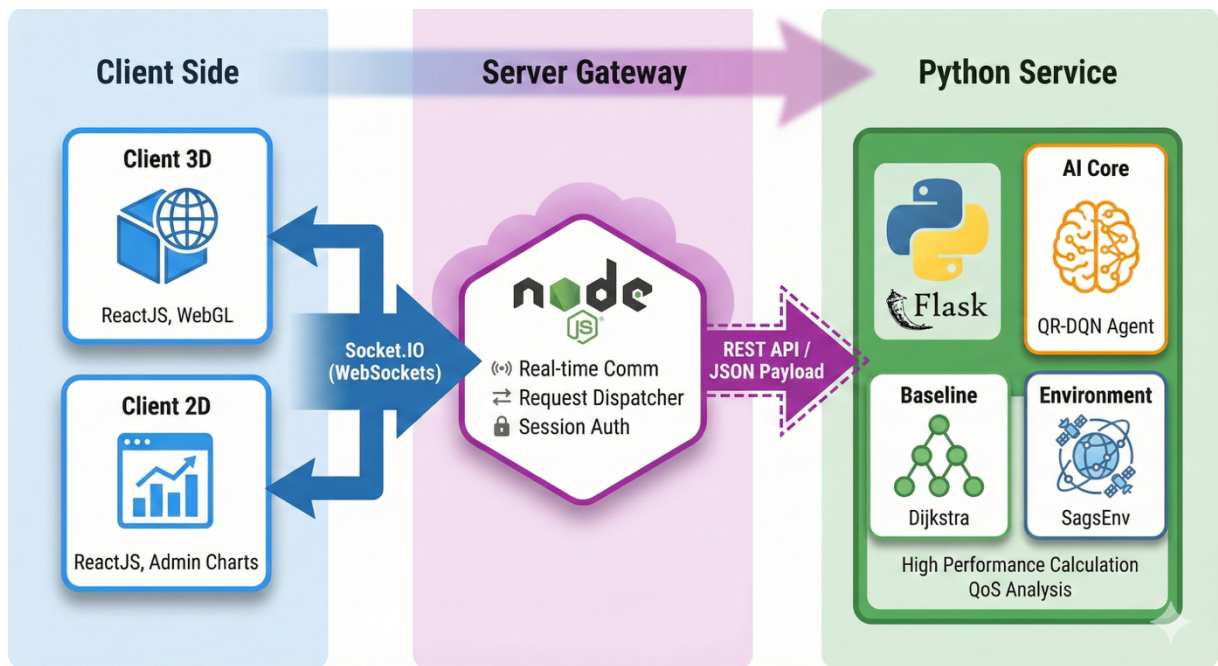
CHƯƠNG 3: TỐI ƯU HỢP TÁC BẰNG TRÍ TUỆ NHÂN TẠO TRONG MẠNG TÍCH HỢP KHÔNG GIAN – MẶT ĐẤT – BIỂN (SAGSINS)

Chương này trình bày quá trình hiện thực hóa mô hình định tuyến QR-DQN vào hệ thống mô phỏng hoàn chỉnh, được xây dựng theo kiến trúc Client–Server đa tầng. Mục tiêu của chương là mô tả rõ cách mô hình AI được tích hợp vào hệ thống vận hành thực tế, bao gồm quá trình giao tiếp giữa các thành phần, cơ chế truyền request, xử lý song song Dijkstra – QR-DQN, cũng như cách hiển thị kết quả mô phỏng và giám sát tài nguyên.

Nội dung chương giới thiệu chi tiết kiến trúc phần mềm gồm ba lớp: (i) Client tương tác với người dùng và quản trị viên, (ii) Server Gateway đảm nhiệm điều phối và giao tiếp thời gian thực, và (iii) Python Service đóng vai trò bộ xử lý thuật toán trung tâm. Đồng thời, chương mô tả luồng hoạt động của hệ thống từ lúc người dùng gửi yêu cầu mô phỏng, Python Service xử lý định tuyến, cho đến khi kết quả được phản hồi và hiển thị trên giao diện 2D/3D.

3.1. Mô hình kiến trúc Client – Server

Hình 22: Mô tả tổng quan hệ thống



Hệ thống được triển khai theo kiến trúc Client–Server đa tầng, cho phép tách biệt rõ ràng giữa giao diện người dùng, tầng giao tiếp trung gian và tầng xử lý thuật toán. Kiến trúc này đảm bảo khả năng mở rộng, dễ bảo trì và hỗ trợ hoạt động thời gian thực trong quá trình mô phỏng định tuyến. Ba thành phần chính của hệ thống bao gồm:

3.1.1. Client (Giao diện người dùng)

Hệ thống sử dụng hai ứng dụng React độc lập, mỗi ứng dụng phục vụ một nhóm người dùng khác nhau:

- Client Mô phỏng (3D Simulation Client): Giao diện mô phỏng 3D hỗ trợ người dùng gửi yêu cầu dịch vụ (request) và quan sát trực quan quá trình tìm đường trong không

gian SAGSINs. Môi trường 3D hiển thị vị trí vệ tinh, tuyến truyền và trạng thái mạng theo thời gian thực.

- Client Giám sát (2D Monitoring Client): Giao diện 2D dành cho quản trị viên hệ thống, cung cấp chức năng giám sát tài nguyên mạng, theo dõi số lượng yêu cầu, tải mạng và các thông số vận hành khác.

3.1.2. Server Gateway

Server Gateway đóng vai trò là tầng trung gian giữa các client và bộ xử lý thuật toán:

- Giao tiếp thời gian thực với client thông qua Socket.IO
- Điều phối yêu cầu, phân loại request và chuyển tiếp các tác vụ tính toán nặng sang Python Service
- Đảm bảo quản lý phiên, đồng bộ trạng thái và duy trì độ tin cậy của kết nối

Server này giúp giảm tải cho tầng thuật toán và duy trì khả năng phục vụ nhiều client song song, mở rộng của toàn bộ hệ thống.

3.1.3. Python Service (Flask Backend)

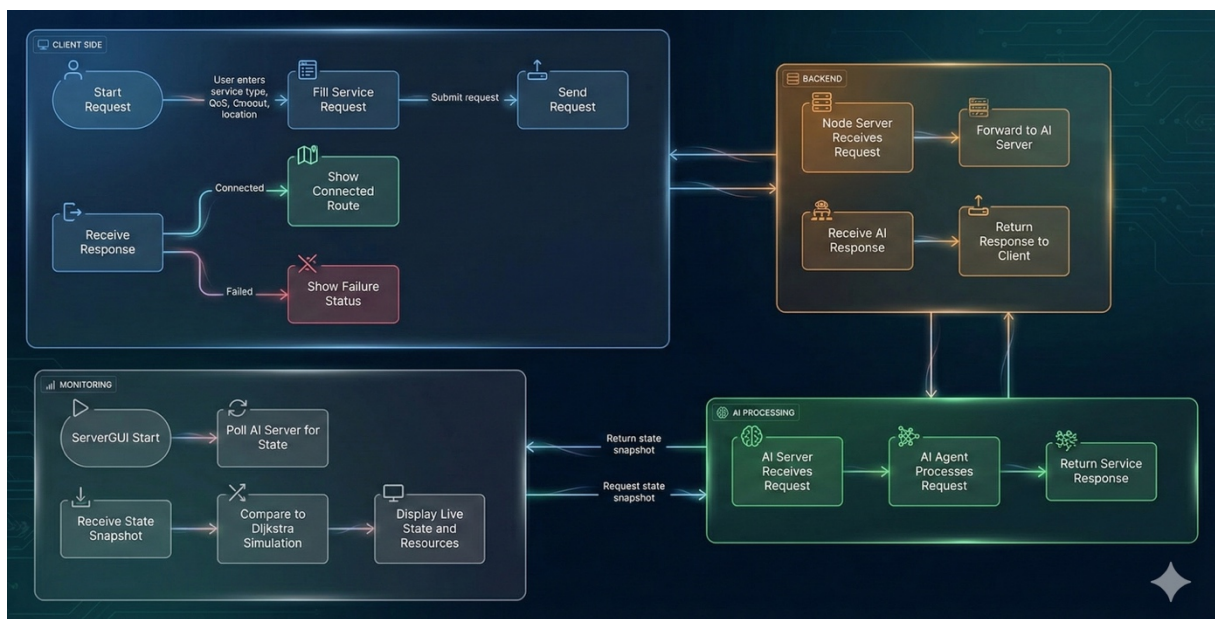
Python Service là thành phần đóng vai trò bộ xử lý trung tâm của hệ thống:

- Được xây dựng bằng *Flask*, cung cấp API để nhận yêu cầu từ Server Gateway
- Thực thi thuật toán định tuyến **Dijkstra** và tác nhân **QR-DQN** song song
- Tương tác trực tiếp với *SagsEnv* để mô phỏng trạng thái mạng, cập nhật tài nguyên và trả về đường đi tối ưu
- Xử lý các tác vụ tính toán nặng như mô phỏng hop-to-hop, tính *QoS*, và quản lý trạng thái mạng

Thành phần này chính là nơi hiện thực hóa toàn bộ logic trí tuệ nhân tạo của hệ thống.

3.2. Luồng hoạt động và mô tả chương trình

Hình 23: Sơ đồ khối mô tả kiến trúc Client-Server và luồng dữ liệu của ứng dụng



Chúng ta có thể chia hoạt động của hệ thống thành hai luồng chính: Luồng mô phỏng định tuyến (người dùng chính) và luồng giám sát tài nguyên (quản trị viên).

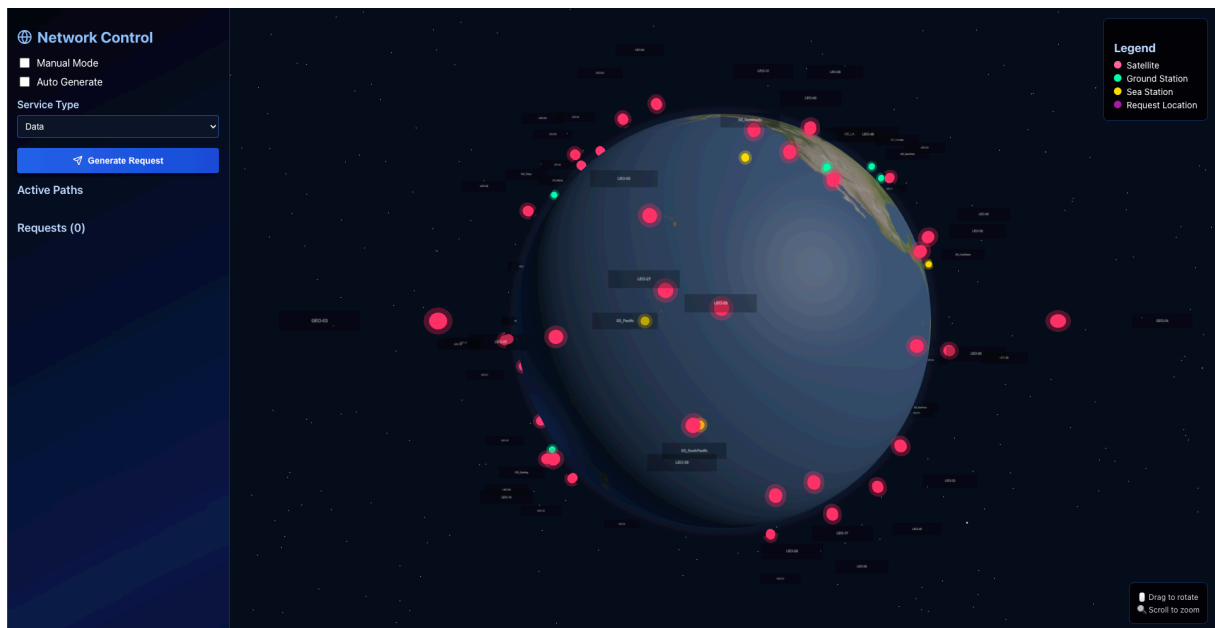
Luồng 1: Mô phỏng định tuyến (User Client)

Luồng này mô tả quy trình xử lý một yêu cầu mô phỏng từ phía người dùng, từ lúc tạo request đến khi nhận kết quả định tuyến.

Giai đoạn 1: Khởi tạo và Gửi yêu cầu (Client Mô phỏng)

- Người dùng khởi chạy ứng dụng **Client_GUI**.
- Giao diện cung cấp mô hình trực quan 3D của mạng vệ tinh và một biểu mẫu nhập liệu cho phép lựa chọn:
 - Loại yêu cầu (*Request Type*)
 - Chế độ tạo yêu cầu thủ công (*Manual Mode*) hay tự động (*Auto Generate*)
- Ứng dụng thiết lập kết nối Socket.IO tới Server Gateway (cổng 3000).
- Khi người dùng gửi yêu cầu, client đóng gói dữ liệu và truyền đến Server Gateway qua kênh Socket.IO

Hình 24: Giao diện Client chính với quả địa cầu 3D và nhập liệu mô phỏng



Mô tả: Giao diện chính cho phép người dùng tương tác trực quan. Bên phải là mô hình 3D, bên trái là các trường nhập liệu để định nghĩa một yêu cầu mô phỏng.

Giai đoạn 2: Tiếp nhận và Điều phối (Server Gateway)

- Server Gateway (Node.js) lắng nghe kết nối Socket.IO và chờ các sự kiện yêu cầu mới.
- Khi nhận được yêu cầu, máy chủ phân tích nội dung request.
- Dựa trên lựa chọn thuật toán, Node Server gửi yêu cầu tương ứng đến Python Service qua HTTP API, trong đó:
 - Dijkstra được dùng làm đường cơ sở
 - QR-DQN được dùng để thực thi định tuyến và phân tích, so sánh, đánh giá so với các thuật toán truyền thống.

Giai đoạn 3: Xử lý nghiệp vụ (Python Service)

- Python Service (Flask) tiếp nhận yêu cầu tại API endpoint tương ứng.
- Dịch vụ nạp cấu trúc mạng, trạng thái tài nguyên và khởi tạo các đối tượng mô phỏng.
- Hệ thống thực thi song song hai phương pháp định tuyến:
 - Dijkstra: tìm đường đi ngắn nhất dựa trên trọng số liên kết.
 - QR - DQN: tác nhân đã huấn luyện dự đoán hành động tối ưu dựa trên phân phối giá trị.
- Cả hai kết quả được ghi vào file log (service_comparison_log) để phục vụ phân tích.
- Python Service trả về đường đi và các chỉ số QoS cho Server Gateway.

Giai đoạn 4: Trả kết quả và Hiển thị (Client Mô phỏng)

- Node Server nhận phản hồi từ Python Service qua HTTP.
- Kết quả được gửi trở lại User Client qua Socket.IO.
- Ứng dụng hiện mô-đun (modal) hiển thị:
 - Đường đi được định tuyến.
 - Thông số QoS đạt được.
 - Trạng thái thành công hoặc thất bại.

Hình 25: Giao diện Modal hiển thị kết quả đường đi



Mô tả: Sau khi xử lý hoàn tất, client sẽ hiển thị kết quả mô phỏng, bao gồm đường đi chi tiết (danh sách các nút) để đi được tới *GateWay*.

Luồng 2: Giám sát tài nguyên (Admin Client)

Luồng này chp phép quản trị viên giám toàn bộ mạng theo thời gian thực.

Giai đoạn 1: Yêu cầu dữ liệu (Client Giám sát)

- Quản trị viên mở ứng dụng Server_GUI.
- Ứng dụng thiết lập kết nối Socket.IO đến Server Gateway (cổng 3005).

- Client gửi sự kiện *get-stats* yêu cầu dữ liệu giám sát hệ thống.

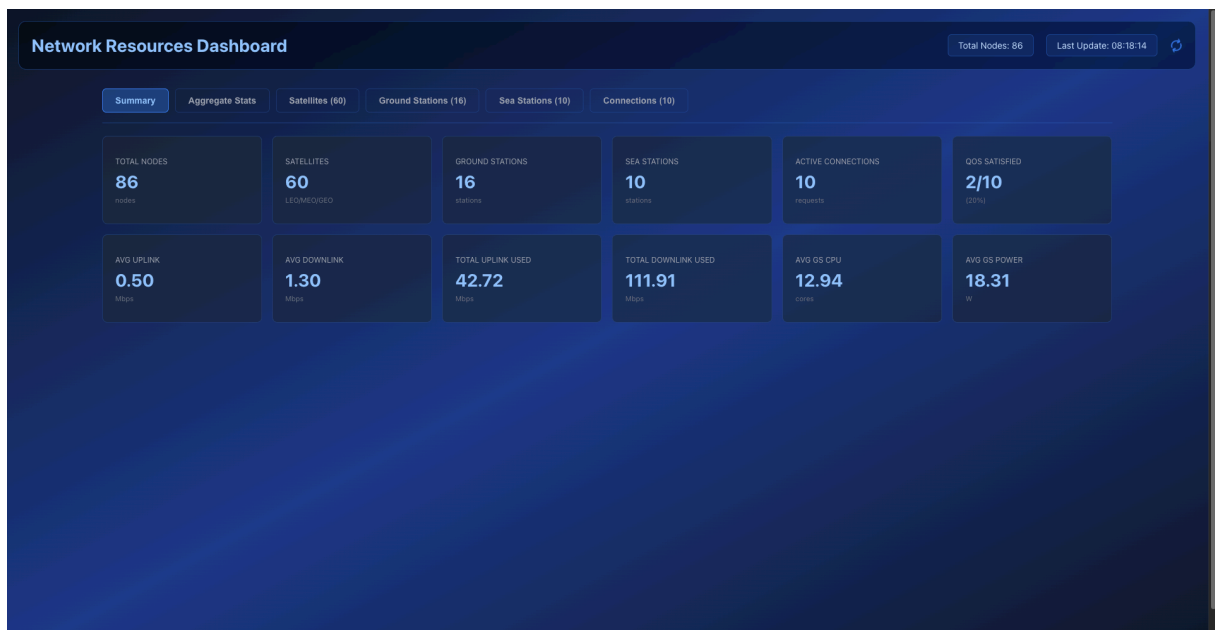
Giai đoạn 2: Truy xuất dữ liệu (Server Gateway)

- Node Server nhận yêu cầu lấy thông tin dữ liệu *get-stats*.
- Máy chủ truy cập các file thống kê do Python Service sinh ra (state snapshots, log tài nguyên, thông số QoS,...).
- Node Server tổng hợp dữ liệu và gửi lại qua sự kiện *stats-data* đến Admin Client.

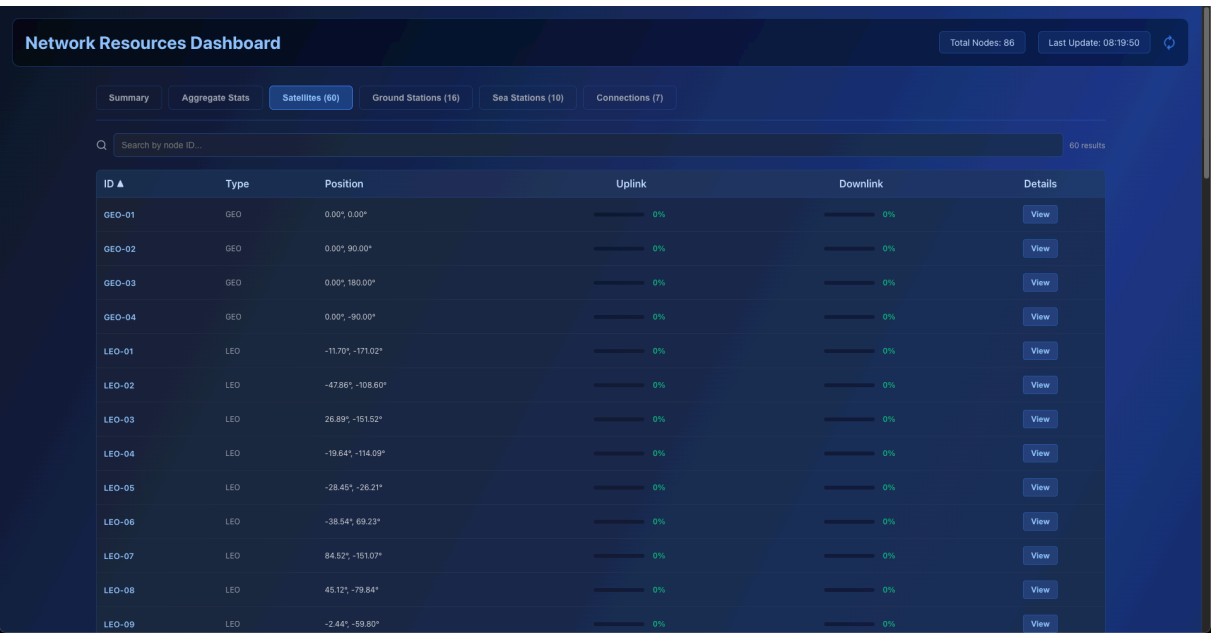
Giai đoạn 3: Hiện thị Dashboard (Client Giám sát)

- Admin Client nhận dữ liệu **stats-data**.
- Thông tin được cập nhật vào dashboard giám sát dưới dạng biểu đồ, bảng biểu và sơ đồ mạng trực quan, bao gồm:
 - Trạng thái toàn bộ vệ tinh (LEO/GEO)
 - Trạm mặt đất (GS)
 - Trạm biển (SS)
 - Trạng thái toàn bộ kết nối, mức độ sử dụng của từng nút.
 - So sánh chi tiết giữa QR-DQN và Dijkstra
 - Phân bố QoS và độ lệch chuẩn

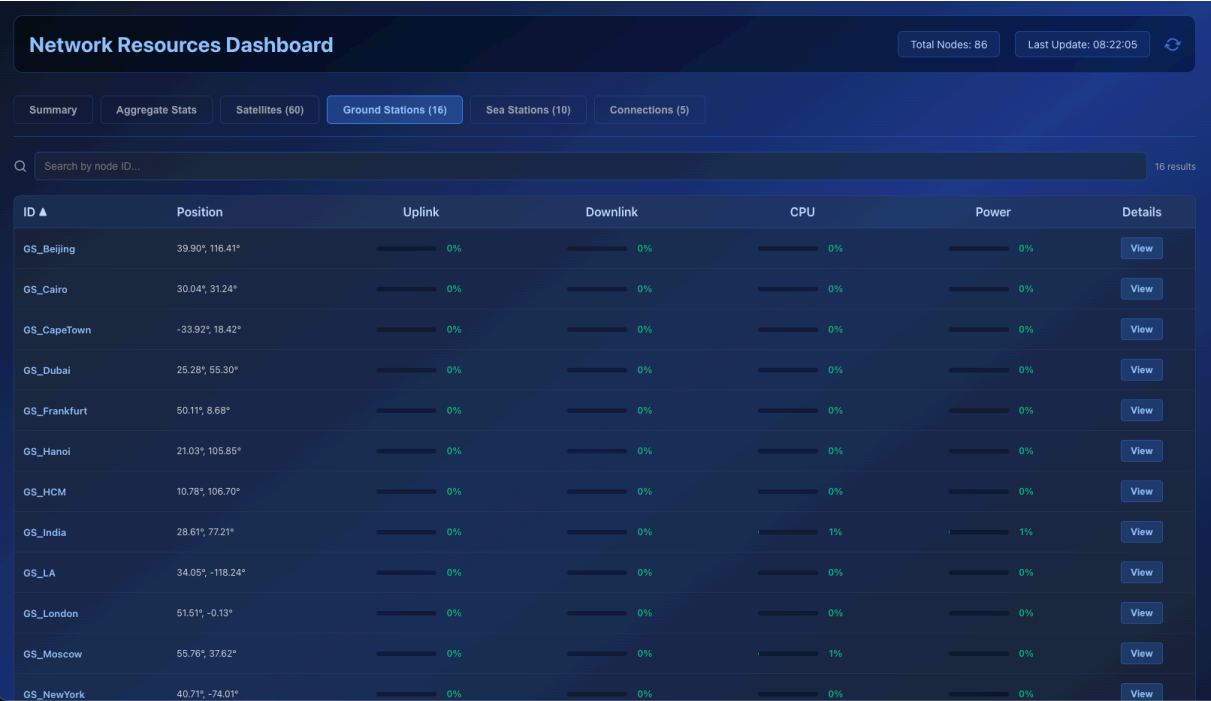
Hình 26: Giao diện Dashboard giám sát



Hình 27: Giao diện giám sát toàn bộ vệ tinh Satellites

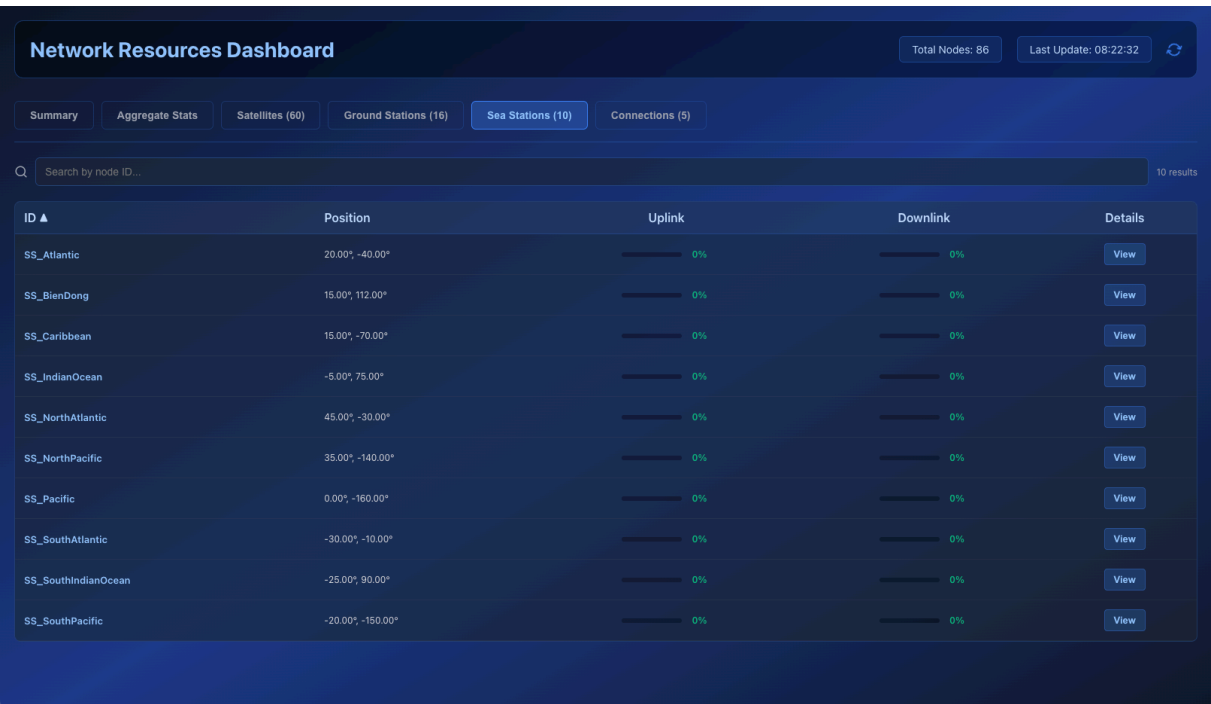


Hình 28: Giao diện giám sát toàn bộ trạm mặt đất GroundStation

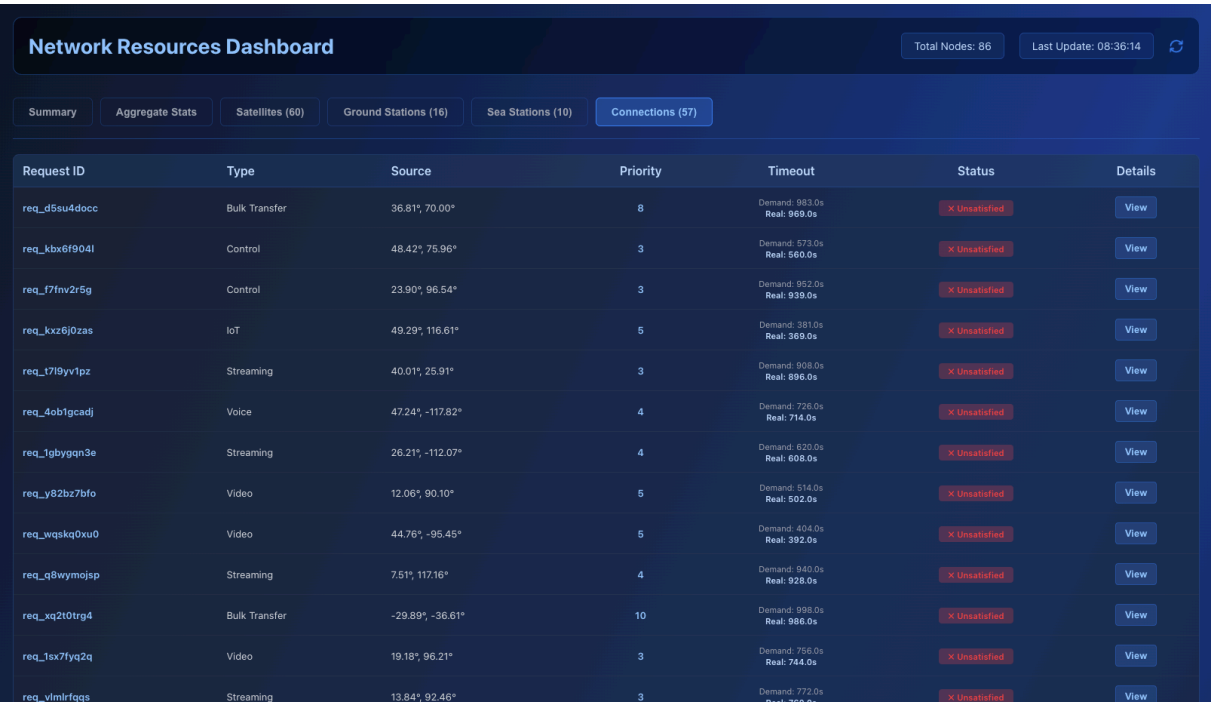


Hình 29: Giao diện giám sát toàn bộ trạm biển SeaStation

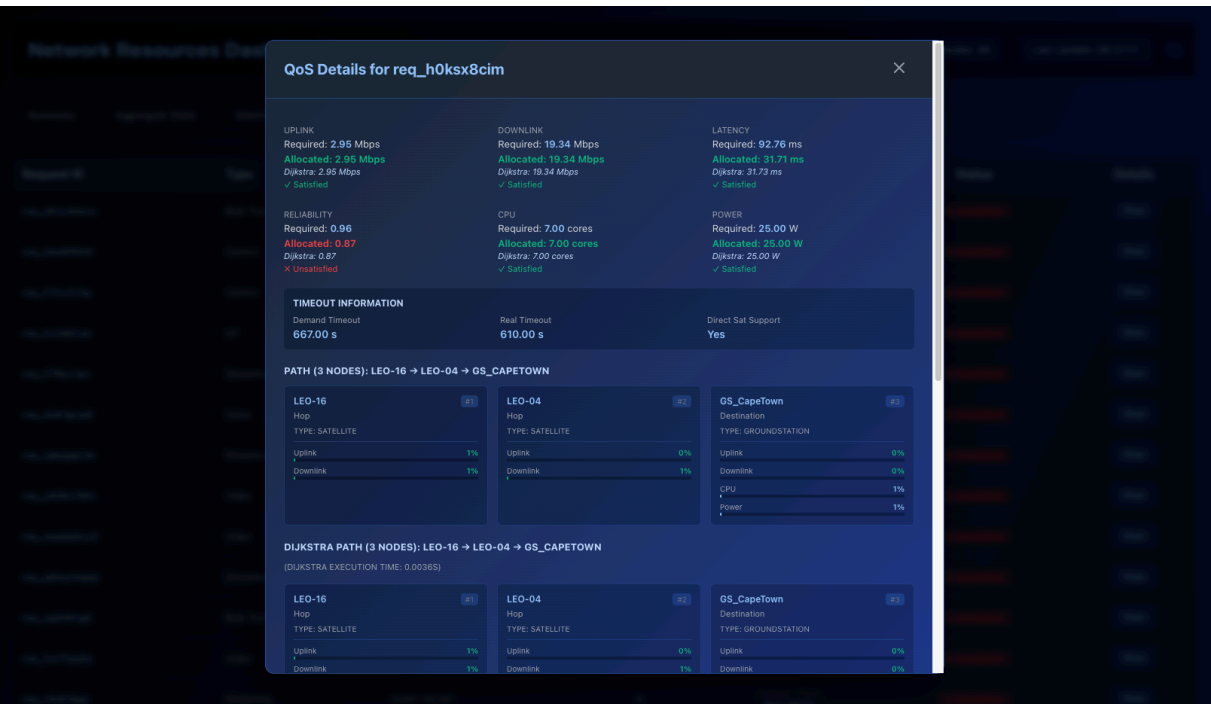
PBL4: DỰ ÁN HỆ ĐIỀU HÀNH & MẠNG MÁY TÍNH



Hình 30: Giao diện giám sát toàn bộ kết nối Connection tại thời điểm thực



Hình 31: Giao diện chi tiết các thông số AI Agent đạt được ở 1 request ngẫu nhiên



Hình 32: Biểu đồ so sánh chi tiết các thông số AI Agent và Dijkstra đạt được ở 1 request ngẫu nhiên



Hình 33: Biểu đồ so sánh tổng quan AI Agent và Dijkstra đạt được trong toàn bộ quá trình



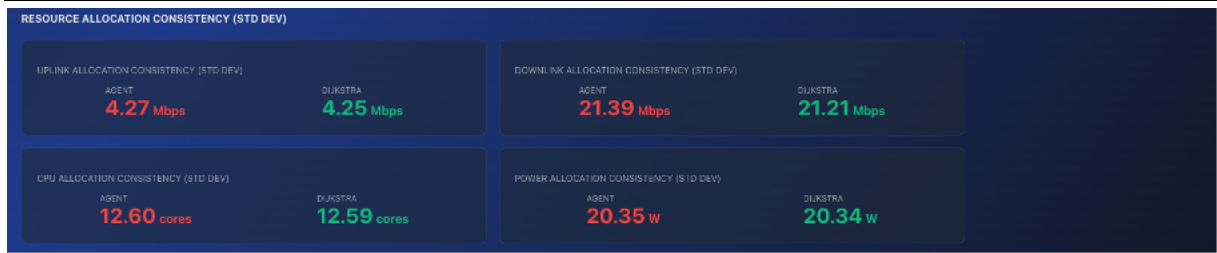
Hình 34: Biểu đồ so sánh hiệu suất AI Agent và Dijkstra đạt được trong toàn bộ quá trình



Hình 35: Giao diện so sánh chi tiết QoS của AI Agent và Dijkstra



Hình 36: Giao diện so sánh độ lệch chuẩn QoS của AI Agent và Dijkstra



Mô tả: Giao diện quản trị hiển thị các thông số thống kê về tổng số vệ tinh (*LEO, GEO*), trạm mặt đất (*GS*), trạm biển (*SS*), và các biểu đồ thể hiện tình trạng tài nguyên mạng.

3.3. Tổng kết Chương

Chương này đã trình bày chi tiết việc triển khai mô hình định tuyến QR-DQN trong một hệ thống mô phỏng hoàn chỉnh dựa trên kiến trúc Client–Server. Các thành phần Client, Server Gateway và Python Service được kết hợp thành một quy trình vận hành nhất quán, trong đó các yêu cầu mô phỏng được xử lý thời gian thực và kết quả định tuyến được phản hồi trực quan cho cả người dùng và quản trị viên.

Thông qua hai luồng hoạt động chính là mô phỏng định tuyến và giám sát tài nguyên, hệ thống cho phép đánh giá toàn diện hành vi của tác nhân AI trong môi trường SAGSINs động, đồng thời đảm bảo khả năng quan sát trạng thái mạng, tải tài nguyên và hiệu năng của từng thuật toán. Việc tích hợp QR-DQN vào tầng xử lý Python Service cho phép nghiên cứu hành vi của mô hình trong điều kiện gần với thực tế, cung cấp dữ liệu đáng tin cậy cho bước phân tích kết quả.

Chương 3 đã chứng minh tính khả thi của việc tích hợp và vận hành mô hình QR-DQN trong hệ thống mô phỏng SAGSINs. Việc triển khai thành công kiến trúc Client-Server, cơ chế xử lý song song và khả năng phản hồi thời gian thực đảm bảo tác nhân AI hoạt động ổn định và ra quyết định định tuyến hiệu quả trong môi trường phức tạp. Các kết quả vận hành này là cơ sở thực nghiệm để đánh giá hiệu năng, giới hạn của mô hình, đồng thời là tiền đề cho phân kết luận và định hướng phát triển ở chương tiếp theo.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Dựa trên cơ sở lý thuyết đã được trình bày ở Chương 1, phương pháp luận đề xuất và thiết kế hệ thống ở Chương 2, cùng với chạy thực nghiệm ở Chương 3, chương cuối cùng này sẽ tổng kết lại những kết quả cốt lõi mà đề tài đã đạt được. Đồng thời, chương cũng sẽ chỉ ra những hạn chế còn tồn tại trong khuôn khổ nghiên cứu và đề xuất các hướng phát triển tiềm năng trong tương lai, nhằm hoàn thiện và mở rộng giải pháp cho đề tài Mạng Tích hợp Không gian – Mặt đất – Biên (SAGSINs).

Chương này tổng kết lại toàn bộ kết quả nghiên cứu, đánh giá hiệu quả của hệ thống định tuyến tự trị dựa trên QR - DQN, đồng thời đề xuất những hướng mở rộng tiềm năng trong tương lai.

1. Kết quả đạt được

Đề tài đã giải quyết thành công mục tiêu tổng quát đã đề ra: nghiên cứu, đề xuất và kiểm chứng một kiến trúc ứng dụng Học tăng cường sâu (DRL) để xây dựng một hệ thống định tuyến tài nguyên động tự chủ cho Mạng Tích hợp Không gian – Mặt đất – Biên (SAGSINs).

Quá trình nghiên cứu và thực nghiệm đã đạt được những kết quả cụ thể như sau:

- Xây dựng thành công môi trường mô phỏng (SagsEnv) có độ trung thực cao: Đã hiện thực hóa thành công môi trường mô phỏng SagsEnv, tuân thủ chuẩn *gymnasium*. Môi trường này mô phỏng chi tiết và động các thành phần cốt lõi của mạng SAGSINs, bao gồm các thực thể mạng đa tầng (Vệ tinh *LEO/GEO*, Trạm mặt đất *GS*, Trạm biển *SS*), các yêu cầu dịch vụ đa dạng với nhiều loại *QoS* phức tạp, và các đặc tính của kênh truyền.
- Hiện thực hóa và huấn luyện thành công tác nhân DRL: Đã thiết kế và triển khai thành công một tác nhân AI sử dụng thuật toán QR – DQN, dựa trên kiến trúc mạng nơ-ron “*MlpPolicy*”. Tác nhân này có khả năng xử lý không gian trạng thái phức tạp đa chiều và học được các chiến lược định tuyến phức tạp từ hàm phần thưởng đa mục tiêu.
- Kết quả thực nghiệm cho thấy tác nhân QR - DQN đạt hiệu năng vượt trội so với thuật toán Dijkstra truyền thống: Thông qua *StatsManager*, kết quả thực nghiệm đã chứng minh Tác nhân QR - DQN khá vượt trội hơn Dijkstra ở các khía cạnh:
 - Khả năng thích ứng *QoS*: Tác nhân QR - DQN có khả năng tìm đường đi linh hoạt, ưu tiên các trọng số *QoS* khác nhau (*latency*, *uplink*, *downlink*, *cpu*, *power*) tùy theo loại dịch vụ, điều mà Dijkstra với các thuật toán cổ điển “*cứng nhắc*” không thể làm được.
 - Khả năng cân bằng tải: Tác nhân QR - DQN, được huấn luyện liên tục khắc nghiệt, chủ động tránh các nút mạng bị quá tải hoặc sắp bị quá tải, giúp phân bổ lưu lượng đồng đều hơn và tăng tỷ lệ thành công chung và ổn định của toàn bộ hệ thống.
- Khẳng định tính tự trị và khả năng tự phục hồi của hệ thống: Kết quả nổi bật nhất là khả năng thích ứng thời gian thực của tác nhân QR - DQN. Khi đối mặt với sự thay đổi trạng thái mạng, tác nhân đã tự động nhận diện sự thay đổi này trong vector quan sát *obs* và chủ động điều hướng sang các tuyến đường thay thế. Điều này chứng minh được nguyên tắc thiết kế cốt lõi là “*Tính tự trị*” và “*Tính thích ứng*” của QR - DQN.

2. Hạn chế của đề tài

Mặc dù đã đạt được các mục tiêu chính, đề tài vẫn còn một số hạn chế mang tính khoa học cần được nhìn nhận một cách khách quan:

- Độ trung thực của môi trường mô phỏng: Môi trường SagsEnv, dù chi tiết, vẫn là một sự đơn giản hóa so với thực tế. Đề tài chưa mô phỏng Phân đoạn Hàng không (UAVs) và chưa đi sâu vào các vấn đề phức tạp của tầng vật lý như mô hình kênh truyền chi tiết.
- Vấn đề về quy mô: Các thực nghiệm được tiến hành trên một mạng SAGSINs quy mô nhỏ (60 vệ tinh, 16 trạm mặt đất, 10 trạm biển). Khả năng mở rộng của mô hình tác nhân đơn (*Single-Agent QR - DQN*) để quản lý một mạng lưới toàn cầu với hàng ngàn nút mạng di động vẫn là một thách thức lớn.
- Không gian hành động rời rạc hóa: Để đơn giản hóa quá trình huấn luyện, không gian hành động đã được rời rạc hóa thành việc lựa chọn một trong 10 nút lân cận. Trong thực tế, các quyết định điều hành mạng (ví dụ: phân bổ bao nhiêu % băng thông) thuộc không gian liên tục.
- Yêu cầu tài nguyên tính toán: Việc huấn luyện một mô hình DRL sâu như QR - DQN trên một môi trường phức tạp bằng vector đa chiều đòi hỏi tài nguyên tính toán (GPU) rất lớn và thời gian đáng kể để hội tụ và khó khăn trong việc đạt được mục tiêu của đề tài trong thực tế.

3. Hướng phát triển trong tương lai

Từ những kết quả đã đạt được và các hạn chế đã chỉ ra, đề tài mở ra nhiều hướng nghiên cứu và phát triển tiềm năng trong tương lai:

- Nâng cao độ trung thực cho môi trường mô phỏng: Tích hợp Phân đoạn Hàng không (UAVs) vào lớp *Network* và tích hợp các mô hình kênh truyền thực tế hơn vào SagsEnv để buộc tác nhân học các chiến lược mạnh mẽ hơn.
- Nghiên cứu các kiến trúc AI có khả năng mở rộng: Thay vì một tác nhân trung tâm (*Single-Agent*), có thể nghiên cứu các kiến trúc Học tăng cường đa tác nhân (*Multi-Agent Reinforcement Learning - MARL*). Trong đó, mỗi nút (ví dụ: mỗi vệ tinh) là một tác nhân riêng, và chúng học cách giao tiếp, phối hợp với nhau để tối ưu hóa mục tiêu toàn cục.
- Chuyển đổi sang không gian hành động liên tục: Nghiên cứu và áp dụng các thuật toán DRL tiên tiến có khả năng hoạt động trong không gian hành động liên tục để tác nhân có thể ra các quyết định tinh vi hơn (ví dụ: quyết định phân bổ *chính xác* 5.5 Mbps băng thông, thay vì chỉ “chọn” một nút).
- Tối ưu hóa huấn luyện: Triển khai đầy đủ chiến lược huấn luyện theo chương trình. Thay vì huấn luyện trên môi trường mô phỏng ngay từ đầu, cho tác nhân học tuần tự từ các môi trường đơn giản để tăng tốc độ hội tụ.
- Ứng dụng Trí tuệ Nhân tạo có thể giải thích: Một hạn chế của các mạng nơ-ron sâu là tính “*hộp đen*”. Hướng nghiên cứu tương lai cần tập trung vào việc áp dụng các kỹ thuật XAI (như *LIME*, *SHAP*) để diễn giải và hiểu được lý do tại sao tác nhân QR - DQN lại đưa ra một quyết định định tuyến cụ thể.

4. Tổng kết

Tổng kết lại, đề tài “*Tối ưu hợp tác bằng Trí tuệ nhân tạo trong mạng Tích hợp Không Gian – Mặt đất – Biển (SAGSINs)*” đã đặt những viên gạch nền móng quan trọng cho hướng nghiên cứu và phát triển mạng viễn thông tự chủ thế hệ mới.

Về mặt học thuật, đề tài đã:

- Xây dựng và chuẩn hóa một môi trường mô phỏng có độ trung thực cao SagsEnv theo chuẩn *Gymnasium*, tạo tiền đề cho các nghiên cứu tiếp theo trong lĩnh vực Học tăng cường cho mạng hợp nhất (*Integrated Networks*).
- Đề xuất và huấn luyện thành công mô hình QR - DQN có khả năng học chính sách định tuyến tối ưu trong không gian trạng thái phức tạp đa chiều, vượt qua các giới hạn của phương pháp truyền thống như Dijkstra.
- Thiết kế hàm phần thưởng đa mục tiêu phản ánh đúng bản chất cân bằng giữa *QoS*, hiệu quả tài nguyên và tính ổn định mạng — đóng vai trò quan trọng trong quá trình huấn luyện và giúp mô hình đạt hiệu năng cao.

Về mặt thực tiễn, hệ thống được đề xuất thể hiện:

- Khả năng tự trị và thích ứng động cao: tác nhân QR - DQN có thể tự điều chỉnh chiến lược định tuyến khi topo mạng hoặc tài nguyên thay đổi, mà không cần can thiệp thủ công.
- Tính khả thi ứng dụng: kết quả mô phỏng và đánh giá định lượng chứng minh DRL hoàn toàn có thể triển khai để hỗ trợ các cơ chế định tuyến tự động trong các hệ thống mạng 6G hoặc mạng vệ tinh lai (Hybrid Satellite–Terrestrial).

Nhìn chung, đề tài đã góp phần quan trọng trong việc khẳng định tiềm năng to lớn của Học tăng cường sâu (DRL) trong việc tự động hóa, tối ưu hóa và thông minh hóa các hệ thống mạng hợp nhất thế hệ tương lai. Các kết quả đạt được không chỉ có giá trị khoa học mà còn mang ý nghĩa thực tiễn, mở ra hướng đi khả thi cho việc phát triển các mạng viễn thông tự trị trong kỷ nguyên mạng viễn thông 6G.

TÀI LIỆU THAM KHẢO

- [1] Yang, Kai; Wang, Yichen; Gao, Xiaozheng; Shi, Chenrui; Huang, Yuting; Yuan, Hang; Shi, Minwei. *Communications in Space–Air–Ground Integrated Networks: An Overview*. Science Partner Journals – *Space: Science & Technology*, 2025.
- [2] Chen, Jiming; Zhang, Han; Xie, Zhe. *Space–Air–Ground Integrated Network (SAGIN): A Survey*. arXiv preprint, 2023.
- [3] Ray, Partha Pratim. *A review on 6G for Space–Air–Ground Integrated Network*. (Elsevier), 2021.
- [4] Cheng, Nan; He, Jingchao; Yin, Zhisheng; Zhou, Conghao; Wu, Huaqing; Lyu, Feng. *6G service-oriented space-air-ground integrated network*. (Elsevier), 2022.
- [5] An, Jian Ping; Li, Jian Guo; Yu, Ji Hong; Ye, Neng. *Key Technologies of Space-Air-Ground Communication Networks: A Survey*. *Acta Electronica Sinica*, Vol. 50(2): 470–479, 2022.
- [6] Cao, Xuelin; Yang, Bo; Yuen, Chau; Han, Zhu. *HAP-Reserved Communications in Space-Air-Ground Integrated Networks*. arXiv preprint, 2021.
- [7] Dicandia, Francesco Alessio; Fonseca, Nelson J. G.; Bacco, Manlio; Mugnaini, Sara; Genovesi, Simone. *Space-Air-Ground Integrated 6G Wireless Communication Networks: A Review of Antenna Technologies and Application Scenarios*. MDPI – *Remote Sensing*, 2021.
- [8] Sharif, Sana; Zeadally, Sherali; Ejaz, Waleed. *Space-aerial-ground-sea integrated networks: Resource optimization and challenges in 6G*. (Elsevier), 2023.
- [9] Zhang, Peiying; Chen, Shengpeng; Zheng, Xiangguo; Li, Peiyan; Wang, Guilong; Wang, Ruixin; Wang, Jian; Tan, Zizhuang. *UAV Communication in Space–Air–Ground Integrated Networks (SAGINs): Technologies, Applications, and Challenges*. *Drones*, 2025.
- [10] Qian Chen; Zheng Guo; Weixiao Meng; Shuai Han; Cheng Li; Tony Q. S. Quek. *A Survey on Resource Management in Joint Communication and Computing-Embedded SAGIN*. *IEEE Communications Surveys & Tutorials*, 2024.
- [11] Wu, Chenyu; Wang, Xi; Hu, Yi; Han, Shuai; Niyato, Dusit. *Interplay Between AI and Space-Air-Ground Integrated Network: The Road Ahead*. arXiv preprint, 2025.
- [12] Fan, Kexin; Feng, Bowen; et al. *Demand-Driven Task Scheduling and Resource Allocation in Space-Air-Ground Integrated Network: A Deep Reinforcement Learning Approach*. *IEEE Transactions on Wireless Communications* (IEEE), 2024.
- [13] Qiu, Yuan; Luo, Xiang; Niu, Jianwei; Zhu, Xinzhong; Yao, Yiming. *Category Attribute-Oriented Heterogeneous Resource Allocation and Task Offloading for SAGIN Edge Computing*. *Journal of Sensor and Actuator Networks* (MDPI), 2025.
- [14] Huang, Chuan; Li, Ran; Wang, Jiachen. *Task Scheduling in Space-Air-Ground Uniformly Integrated Networks with Ripple Effects*. arXiv preprint, 2025.
- [15] Liu, Xuefang; Mao, Weihao; et al. *A Resource Allocation Algorithm for Space-Air-Ground Networks Using Deep Reinforcement Learning*. *Journal of Electronics & Information Technology* (JEIT), 2024.
- [16] Liu, Jin; et al. *Resource Allocation Strategy of Space Cloud Network Based on Reinforcement Learning*. *International Journal of Communication Systems* (Wiley), 2024.
- [17] Zhang, Hao; Deng, Jingya; et al. *Multi-Agent Deep Reinforcement Learning for Dynamic Routing in Space–Air–Ground Integrated Networks*. *IEEE Internet of Things Journal*, 2024.
- [18] Wang, Ting; Chen, Bo; Gao, Lei; Zhang, Xu. *Energy-Efficient Routing Based on Deep Q-Learning for SAGIN*. *Wireless Networks* (Springer), 2024.

- [19] Feng, Xia; Li, Kun; Zhou, Yan. *Edge Computing Optimization in 6G SAGIN Environments*. Computer Communications (Elsevier), 2025.
- [20] Dabney, Will; Rowland, Mark; Bellemare, Marc G.; Munos, Rémi. *Distributional Reinforcement Learning with Quantile Regression*. AAAI – Association for the Advancement of Artificial Intelligence, 2018.
- [21] Mavrin, Borislav; Yao, Hengshuai; Kong, Linglong. *Deep Reinforcement Learning with Decorrelation*. arXiv preprint, 2019.
- [22] Luo, Yudong; Liu, Guiliang; Duan, Haonan; Schulte, Oliver; Poupart, Pascal. *Distributional Reinforcement Learning with Monotonic Splines*. ICLR – International Conference on Learning Representations, 2022.
- [23] Stan, Paul-Gabriel; Oliehoek, Frans A.; Celikok, Mustafa. *Detecting Environment Changes via Quantile Spread in Quantile Regression Deep-Q Networks (QR-DQN)*. Bachelor thesis, Delft University of Technology, 2025.
- [24] Dabney, Will; Ostrovski, Georg; Silver, David; Munos, Rémi. *Implicit Quantile Networks for Distributional Reinforcement Learning*. PMLR – Proceedings of the 35th International Conference on Machine Learning (ICML), 2018.
- [25] Mavrin, Borislav; Yao, Hengshuai; Kong, Linglong. *Distributional Reinforcement Learning for Efficient Exploration*. PMLR – Proceedings of the 36th International Conference on Machine Learning, 2019.
- [26] Yang, Derek; Zhao, Li; Lin, Zichuan; Qin, Tao; Bian, Jiang; Liu, Tieyan. *Fully Parameterized Quantile Function for Distributional Reinforcement Learning*. NeurIPS – 33rd Conference on Neural Information Processing Systems, 2019.
- [27] Huang, Chong; Chen, Xuyang; Chen, Gaojie; Xiao, Pei; Li, Geoffrey Ye; Huang, Wei. *Deep Reinforcement Learning-Based Resource Allocation for Hybrid Bit and Generative Semantic Communications in Space–Air–Ground Integrated Networks*. arXiv preprint, 2024.
- [28] IPLOOK. *Introducing SAGINs: Space-Air-Ground Integrated Networks*.
<https://www.iplook.com/info/introducing-sagins-space-air-ground-integrated-networks-i00338i1.html>
Truy cập: 10/08/2025. Xuất bản: Không rõ.
- [29] MDPI. *Space–Air–Ground–Sea Integrated Network with Federated Learning*.
<https://www.mdpi.com/2072-4292/16/9/1640>
Truy cập: 10/08/2025. Xuất bản: 2024.
- [30] Elsevier. *A Survey on Resource Management in Joint Communication and Computing-Embedded SAGIN*.
https://www.researchgate.net/publication/381904454_A_Survey_on_Resource_Management_in_Joint_Communication_and_Computing-Embedded_SAGIN
Truy cập: 06/11/2025. Xuất bản: 2025.
- [31] Stable Baselines3 Contrib. *QR-DQN — Quantile Regression DQN documentation*.
<https://sb3-contrib.readthedocs.io/en/master/modules/qrdqn.html>
Truy cập ngày 06/11/2025. Xuất bản năm không rõ.
- [32] OpenReview. *Non-decreasing Quantile Function Network with Efficient Exploration*.
https://openreview.net/forum?id=f_GA2IU9-K-
Truy cập ngày 06/11/2025. Xuất bản năm 2022.